open-media-drc

Digital Room Correction media chain for Linux and FreeBSD — User and Administrator Manual

Generated from the repository Markdown documentation

v0.93.0-530-g8d517b0-dirty (2026-10-13)

Contents

Part I — Common to Linux and FreeBSD **6**

1 Introduction **7**
1.1 Documentation history . . . 8
1.2 How to read this manual . . . 9
1.3 Repository map . . . 10

2 Components **12**

3 Installation: the common build **14**

4 Usage: drc.sh, filters, and configuration **17**
4.1 drc.sh — the single control point . . . 17
4.2 Reconcile: saved intent versus reality . . . 19
4.3 MPD outputs . . . 19
4.4 The filters/ and configs/ trees . . . 19
4.5 Filter generation workflow . . . 20
4.6 Browsers: the No DRC launchers . . . 21
4.7 Helper scripts . . . 22

5 Filter provenance and verification **24**
5.1 The three repositories . . . 25
5.2 Geometry, design, variant . . . 26
5.3 The bundle . . . 26
5.4 What is hashed . . . 27
5.5 The scripts . . . 27
5.6 The workflow . . . 28
5.7 Deploying the verified identity . . . 30
5.8 Why publication and runtime are separate . . . 31
5.9 Live browser-driven installs . . . 31
5.10 Verification at install time . . . 32
5.11 Verification at runtime . . . 32
5.12 What deliberately cannot go green . . . 33

6 The web panel: omdrc-ctrl **34**
6.1 The kiosk and the Android app . . . 36
6.1.1 Meter timing and the Calibration page . . . 36

6.1.1.1 Practical setup: a phone on one or two networks . . . 41
6.1.1.2 Practical setup: the web UI and plasmoid on host X, plus a phone . 42
6.1.2 The Android app's lifecycle and status notification . . . 45
6.2 The KDE Plasma widget . . . 48
6.3 Qobuz album search . . . 49
6.3.1 Ask AI: recommendations from reviews . . . 50
6.3.2 The listening guide . . . 51
6.3.3 AI settings . . . 51
6.3.4 Customizing the AI prompts . . . 52
6.4 Configuration page — filter installs and audio hardware roles . . . 53
6.4.1 DAC switching and known-device policy . . . 54
6.4.2 Bit-perfect check page . . . 54

7 Bit-perfect verification **55**
7.1 The `/bitperfect` page . . . 55
7.1.1 The five paths, and which question each answers . . . 55
7.1.2 The live Qobuz reference: the renderer's own buffer . . . 56
7.1.3 Checking any track, not only the generated asset . . . 56
7.1.4 Using the page . . . 57
7.1.5 Seeing the bytes . . . 57
7.2 Implementation . . . 58
7.2.1 The pipeline . . . 58
7.2.2 Artifacts . . . 58
7.2.3 Alignment: the anchor, not the silence . . . 58
7.2.4 The silence pad . . . 59
7.2.5 Renderer arbitration . . . 59
7.2.6 Two MPD facts the runner has to work around . . . 59
7.2.7 Privilege, and what the page refuses . . . 60
7.2.8 Verdicts and exit codes . . . 60
7.2.9 Status and what is still owed . . . 60

8 Dynamic range: which master, and what it measures **61**
8.1 What the renderers tell the panel . . . 61
8.2 Identifying the pressing . . . 62
8.3 Measuring the DR of the stream . . . 64
8.3.1 How a job runs . . . 64
8.3.2 The metrics . . . 64
8.3.3 Validation and an example . . . 65
8.3.4 Costs and limits . . . 65
8.3.5 HTTP API . . . 66
8.4 The live DR estimate . . . 66
8.4.1 What DR means . . . 66
8.4.2 The algorithm . . . 66
8.4.3 What the panel shows . . . 67
8.4.3.1 Silence, pauses and stops . . . 67
8.4.4 Options in the panel . . . 67
8.4.5 Configuration . . . 68
8.4.6 Costs and limits . . . 68

9 The DR log: album DR remembered across listening **69**
9.1 Which measurement wins . . . 70
9.2 Where a dr14.txt value comes from . . . 71
9.3 The live meter on an album that already has a value . . . 71
9.4 Editions . . . 72
9.5 Sharing between boxes . . . 72
9.6 Setting up the private repository . . . 72

10 CD input: S/PDIF capture into the DRC chain **75**
10.1 Two clocks, no resampler . . . 75
10.2 CD input is an exclusive source . . . 75
10.3 The capture interface: ESI U24 XL . . . 76
10.4 The web panel card . . . 76

Part II — Linux **78**

11 Linux: installation and lifecycle **79**
11.1 Packages . . . 79
11.1.1 BruteFIR defaults . . . 79
11.2 Files that must live in /etc . . . 79
11.2.1 The MPD `User=` drop-in caveat (Arch) . . . 80
11.3 USB DAC hotplug (udev + systemd) . . . 80
11.4 The loopback: snd-aloop . . . 81
11.5 Audio roles . . . 81
11.6 Browser audio: the managed ALSA default . . . 82
11.7 The panel on Linux . . . 83

12 Linux: CD input with alsaloop **84**
12.1 The ESI input selector on Linux . . . 85
12.2 Requirements and what has not been measured . . . 85

13 Linux: Blu-ray decryption with MakeMKV **86**
13.1 libbluray, libaacs and KEYDB.cfg . . . 86
13.2 Why MakeMKV, and why it is required here . . . 87
13.3 How the players are told to use libmmbd . . . 87
13.4 Renewing the MakeMKV beta key . . . 88
13.4.1 From the video web remote . . . 88
13.5 The Blu-ray check and KEYDB.cfg . . . 89

Part III — FreeBSD **90**

14 FreeBSD: installation **91**
14.1 Packages . . . 91
14.2 Installing the project . . . 91
14.2.1 BruteFIR defaults and MPD configuration . . . 92
14.3 Configuring rc.conf . . . 92
14.4 Validate ordering and activate . . . 94
14.5 Network: keep DHCP on every interface . . . 94
14.5.1 Leave `defaultrouter` unset when any interface uses DHCP . . . 94

15 FreeBSD: services, device roles and lifecycle **96**
15.1 Service inventory . . . 96
15.1.1 musicpd . . . 97
15.1.2 rc.conf.d/musicpd/omdrc_audio . . . 97
15.1.3 omdrc_audio . . . 98
15.1.4 omdrc_cdin, omdrc_renderer, upmpdcli . . . 99
15.1.5 omdrcctrl and omdrcvideo . . . 99
15.1.6 omdrc-audio.conf (devd) . . . 99
15.2 Stable device roles . . . 100
15.3 What devd serializes — and what it does not . . . 101
15.4 Locking and bounded waits . . . 102

16 FreeBSD: the OSS audio stack, panel and diagnostics **104**
16.1 OSS, virtual_oss and cuse . . . 104
16.2 DAC priming . . . 104
16.3 The panel on FreeBSD . . . 104
16.4 Glitch detection . . . 105
16.5 Verifying bit-perfect on FreeBSD . . . 106
16.6 Browser audio on FreeBSD . . . 107
16.6.1 Why Chromium is silent on this box without help . . . 107
16.6.2 The launcher's answer: a private sndiod for the browser session . . . 108
16.6.3 The PulseAudio guard . . . 109
16.6.4 Firefox . . . 110
16.6.5 The ALSA route (kept as a fallback, not recommended) . . . 110
16.6.6 Checking it and fixing silence . . . 110

17 FreeBSD: video — mpv playback and the phone web remote **112**
17.1 Playback launchers . . . 112
17.2 The web remote (`video/webremote/`) . . . 113
17.3 Video-related FreeBSD constraints . . . 114

18 FreeBSD: CD input with omdrc-cdin **115**
18.1 Why a bridge is needed at all . . . 115
18.1.1 The lead is the only number that matters . . . 115
18.2 The three states, and the two very different tenancies . . . 116
18.3 What the transport does to the stream . . . 117
18.3.1 Every row of that table is testable without a CD player . . . 117
18.4 Reading the stats line . . . 118
18.5 Running it as a service . . . 118
18.5.1 The panel card on FreeBSD . . . 120
18.6 The ESI U24 XL on FreeBSD: three traps . . . 120
18.6.1 Trap 1: the S/PDIF input is called `pcm2`, and must be selected . . . 120
18.6.2 Trap 2: `pcm2 = 0.00:0.00` is not a muted capture gain . . . 120
18.6.3 Trap 3: a "44100 Hz" open that is neither 44.1 kHz nor bit-perfect . . . 121
18.6.4 The consequence: capture is 24-bit . . . 121
18.6.5 Checklist when the CD input captures silence . . . 121
18.7 Status and what is still owed . . . 122

19 FreeBSD: known issues and their status **123**

20 FreeBSD: kernel and userland patches **124**
20.1 uaudio(4) patches (`freebsd-uaudio-patch/`) . . . 124
20.1.1 1. `uaudio-clock-before-alt.c.patch` — rate-change cold-open silence . . . 124
20.1.2 2. `uaudio-shared-clock-fix.c.patch` — the 44.1 kHz flicker (bug #295933) 125
20.1.3 3. `uaudio-clock-transaction.c.patch` — residual 44.1 kHz silent open . . 125
20.1.4 4. `uaudio-feedback-follow.c.patch` — candidate (unbuilt) . . . 126
20.1.5 Applying and building . . . 126
20.2 virtual_oss / cuse patches (`freebsd-virtual-oss-patch/`) . . . 126
20.2.1 1. Runtime livelock: `virtual_oss-settrigger-sync-deadlock.patch` . . . 126
20.2.2 2. Teardown deadlock: userland device destroy + kernel refleak . . . 127
20.2.3 Applying and building . . . 127
20.3 Kodi OSS sink patch (`kodi-virtual-oss-patch/`) . . . 128

21 FreeBSD: the port plan and a clean image **129**
21.1 Why the repository cannot be ported as-is . . . 129
21.2 Phase 0 — Upstream the out-of-tree pieces (prerequisite, in flight) . . . 130
21.3 Phase 1 — Make the repository package-friendly . . . 130
21.4 Phase 2 — The port itself . . . 130
21.5 Phase 3 — Submission and maintenance . . . 131
21.6 A clean installable image (planned, not built) . . . 131

Appendix A — Bit-perfect test assets and cross-OS comparison **132**
21.7 Test assets (`tests/`) . . . 132
21.8 Other rates and sample widths . . . 132
21.9 Verification status . . . 133
21.10 Cross-OS byte comparison . . . 133

Appendix B — Source document index **135**
21.11 Keeping this manual up to date . . . 137

Appendix C — Glossary **138**

Part I — Common to Linux and FreeBSD

Everything in this part applies to both operating systems. Where a common topic has an OS-specific implementation, the text says so in one sentence and points to the section of Part II (Linux) or Part III (FreeBSD) that holds it.

Chapter 1

Introduction

open-media-drc is the complete software stack of a headless, high-quality music and video playback appliance with **Digital Room Correction (DRC)**. It runs on **Linux** (reference: Arch) and **FreeBSD** (reference: 15.1 on an Intel NUC), driving an OKTO RESEARCH DAC8 STEREO USB DAC.

Audio from any source — UPnP/OpenHome streaming (Qobuz), local files, network streams, a CD transport — is routed through **BruteFIR**, a fast FIR convolution engine that applies room-correction filters designed with Room EQ Wizard (REW), before reaching the DAC. When correction is off, the chain collapses to a verified **bit-perfect** direct path.

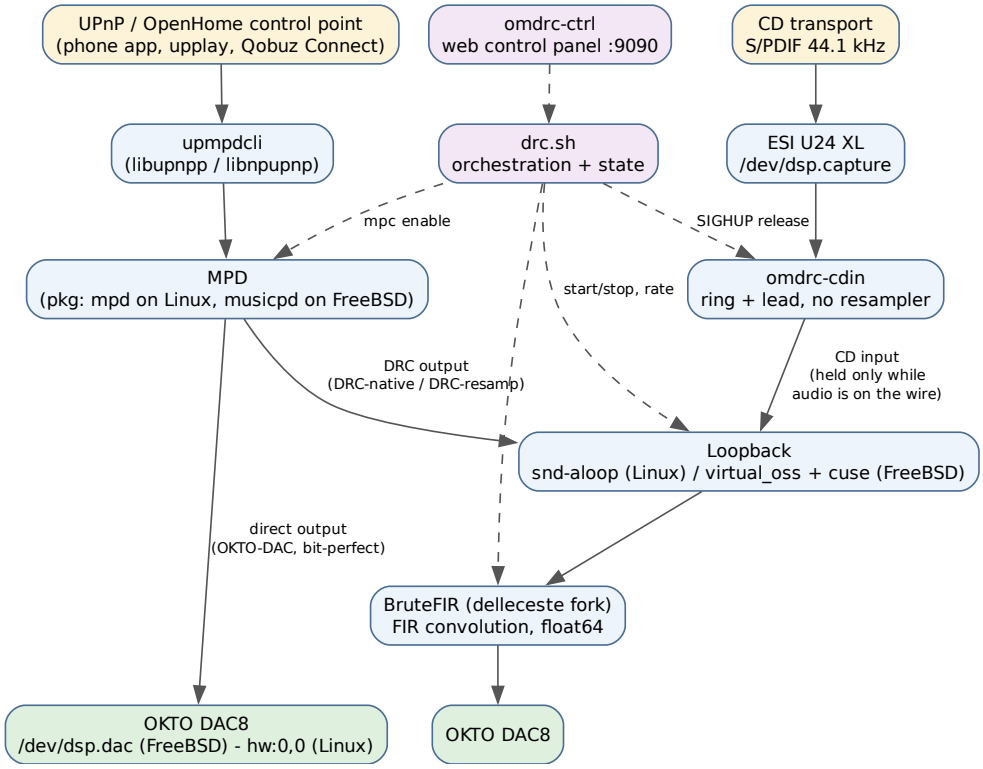


Figure 1.1: The full audio playback chain, from control point to DAC. Solid arrows carry audio; dashed arrows are control.

Design principles:

- **Saved user intent plus actual device state drive DRC.** A present DAC and saved power-on intent mean DRC up at the saved source/rate/design; an absent DAC means transient teardown that keeps the intent; saved power-off means direct output. Boot, hotplug and manual requests all reconcile this rule.
- **Only two resting states exist**: DRC fully up (BruteFIR processing), or direct output. A failed start rolls back; the loopback never runs without BruteFIR.
- **Early-boot glue is copied, everything else can run from the checkout.** Files parsed before a separately mounted `/home` exists (systemd units, udev rules, rc.d scripts, devd rules) are real copies in system paths.
- **Verify, don't assume.** The repository ships tooling to prove the chain is bit-perfect (a USB wire tap), to detect glitches, and to monitor the chain from a phone.

1.1 Documentation history

These milestones summarize changes in commits that updated this manual:

Date	Documentation changes
2026-10-09	Documented the AI features: Ask AI recommendations with their prompt-and-reply view, the listening guide (biographies, swipeable tabs, its strip tinted from the cover), the AI settings dialog (providers and their costs, the model list of always-latest aliases, the new-model notice, usage), and customizing the AI prompt files and the system prompt; Discover by label catalogue and the top-bar Search.
2026-10-08	Gave the DR log its own chapter: how a new track measurement is kept, how the album figure is decided, what the live meter does on an album that already has a value, and how dr14.txt values reach the database (and a disputed one is measured again).
2026-09-27 to 2026-09-29	Added the kiosk and Android app section (orientation and auto-rotate, keep-awake, fast start) and the Qobuz album search (Qobuz's own order, label and date filters, lowering, completions, the player).
2026-09-24	Split the manual into common, Linux and FreeBSD parts; added the glossary and source index; documented the live DR estimate and its DR bar.
2026-09-15 to 2026-09-23	Documented filter provenance and publication, deployment steps, runtime configuration, MPD routing recovery, DR measurement and pressing identification.
2026-08-18 to 2026-08-31	Added CD input and audio lifecycle material, bit-perfect verification, spectrum analyzer updates, browser audio routing and the CMake migration.
2026-07-30 to 2026-08-14	Updated installation for the CMake superproject; added bit-perfect test assets, filter provenance material and FreeBSD documentation.
2026-07-16	Added the generated PDF manual and its build pipeline.

The PDF version stamp follows the repository's project tags, rather than a separate manual version. For example, `v0.93.0-143-g199fbcb` means the build is 143 commits after tag `v0.93.0`, at commit `199fbcb`. A `-dirty` suffix means the working tree has uncommitted changes.

1.2 How to read this manual

Part	Read it if you run	Contents
I. Common (chapters 1–10)	either	components, common build, `drc.sh`, filters and their provenance, the web panel, bit-perfect and dynamic-range tools, the DR log, CD input concept
II. Linux (chapters 11–13)	**Linux**	packages, systemd/udev, `snd-aloop`, ALSA browser audio, the Linux CD bridge
III. FreeBSD (chapters 14–21)	**FreeBSD**	packages, rc.d/devd, `virtual_oss`, video, the CD daemon, known issues, kernel patches, port plan
Appendices	either	bit-perfect test assets and cross-OS comparison; source-document index; glossary

A Linux reader can skip Part III entirely, and a FreeBSD reader Part II; no chapter of one depends on the other. Part I stays OS-neutral and refers to the OS parts wherever an implementation differs.

1.3 Repository map

Path	OS	Contents
`drc.sh`, `drc-status.sh`	both	DRC orchestration and status
`configs/<geometry>/`	both	Per-rate BruteFIR configurations
`filters/<geometry>/<rate>/`	both	Raw FIR filter coefficients (FLOAT64_LE)
`mpd/`	both	MPD templates (`mpd.conf.in` Linux, `musicpd.conf.in` FreeBSD)
`omdrc-ctrl/`	both	Web control panel (Flask)
`browser-nodrc/`	both	Browser launchers that temporarily bypass DRC
`scripts/`, `tests/`	both	Filter conversion, headroom, verification helpers; bit-perfect test signal
`etc/systemd/`, `etc/modules-load.d/`, `etc/modprobe.d/`	Linux	Service and module glue
`etc/rc.d/`, `etc/devd/`, `etc/rc.conf.d/`	FreeBSD	Service and hotplug glue

Path	OS	Contents
`cdin/`	FreeBSD	`omdrc-cdin`, the CD / S-PDIF capture daemon (Linux uses `alsaloop`)
`video/`	FreeBSD	mpv playback launchers + phone web remote
`freebsd-uaudio-patch/`, `freebsd-virtual-oss-patch/`, `kodi-virtual-oss-patch/`	FreeBSD	Kernel and userland patches
`doc/`	both	Measurement plots, verification docs, this manual

Chapter 2

Components

The chain in signal order. The manual writes "MPD" for the player: the package and binary are `mpd` on Linux and `musicpd` on FreeBSD.

Component	Role	OS-specific implementation
upmpdcli (+ libupnpp, libnpupnp)	Makes MPD a UPnP/OpenHome renderer, so a phone app, `upplay` or Qobuz Connect can drive it. Built from source	FreeBSD network caveats: section 14.5
qobuzconnect2mpd	Qobuz Connect front end; the alternative to upmpdcli — one at a time	—
MPD	The player, with the soxr resampler. Three outputs (section 4.3): direct DAC and two DRC loopback outputs	Linux: `mpd`, ALSA output. FreeBSD: `musicpd`, OSS output
Loopback	A device MPD plays into and BruteFIR reads from	Linux: `snd-aloop`, section 11.4. FreeBSD: `virtual_oss`, section 16
BruteFIR (fork)	Float64 FIR convolution of the per-rate `L.raw`/`R.raw` filters, output to the DAC	Linux: ALSA I/O. FreeBSD: built-in OSS I/O
CD bridge	Captures a CD transport's S/PDIF through an ESI U24 XL into the loopback; chapter 10	Linux: `alsaloop`, chapter 12. FreeBSD: `omdrc-cdin`, chapter 18
drc.sh	The single control point of the DRC pipeline (chapter 4)	Service glue: Linux section 11.3, FreeBSD chapter 15
omdrc-ctrl	Flask web panel on the LAN (port 9090) (section 6)	Linux: section 11.7. FreeBSD: section 16.3
browser-nodrc	Launchers that bypass DRC while a browser runs (section 4.6)	Linux ALSA default: section 11.6

Component	Role	OS-specific implementation
video/	mpv playback and a phone web remote	FreeBSD only: chapter 17

BruteFIR is built from `github.com/delleceste/brutefir`, a fork of Anders Torger's BruteFIR with FreeBSD OSS fixes (`bfio_oss` fragment size, `brutefir_loopback -L`, a passthrough-config default). It processes in float64, needs FFTW3 in single and double precision, is built with CMake and installs its modules to `/usr/local/lib/brutefir`.

drc.sh starts and stops BruteFIR and the loopback, selects the MPD output, serializes concurrent runs, records persistent state and rolls back to direct output on failure.

omdrc-ctrl offers DRC control buttons, audio-chain health with a bit-perfect verdict, a live spectrum analyzer and filter-response charts.

browser-nodrc exists because BruteFIR holds the DAC single-open while DRC runs and browsers cannot route into the loopback.

Chapter 3

Installation: the common build

Installation has two layers. This chapter is the OS-neutral build; the OS integration (packages, services, hotplug, loopback) is Linux chapter 11 or FreeBSD chapter 14. Install your OS's packages first, then do steps 1–5 here, then finish in your OS chapter.

Build tools for the from-source components: a C/C++ compiler, **meson + ninja** (upmpdcli stack), **cmake** (BruteFIR, omdrc-ctrl), **pkg-config** and git. Runtime dependencies:

Component	Needs
libnpupnp 6.3.0	libcurl, libmicrohttpd, expat
libupnpp 1.0.4	libnpupnp
upmpdcli 1.9.17	libupnpp, jsoncpp, libmpdclient; the optional Qobuz plugin needs python3 + `requests`
MPD	soxr resampler and the OS audio output plugin
BruteFIR (fork)	FFTW3 single + double precision
omdrc-ctrl	python3, flask >= 2.3, markdown >= 3.5, numpy >= 1.21 (optional)

Package names per OS are in section 11.1 (Linux) and section 14.1 (FreeBSD).

1. The upmpdcli stack (bottom-up; each a standard meson project):

```
for p in libnpupnp-6.3.0 libupnpp-1.0.4 upmpdcli-1.9.17; do
  cd ~/Downloads/$p
  meson setup build --prefix=/usr/local
  ninja -C build
  sudo ninja -C build install
done
```

2. MPD comes from the OS package (Linux `mpd`, FreeBSD `musicpd`).

3. BruteFIR from the fork:

```
git clone https://github.com/delleceste/brutefir ~/Downloads/brutefir
cd ~/Downloads/brutefir
```

```
cmake -B build && cmake --build build
sudo cmake --install build    # modules -> /usr/local/lib/brutefir
```

4. open-media-drc (DRC engine, web UIs, DAC-hotplug glue):

```
git clone --recursive https://github.com/delleceste/open-media-drc
↪  ~/DRC/open-media-drc
cd ~/DRC/open-media-drc
cp host.cmake.sample host.cmake   # AUDIO_USER, GEOMETRY, GEOMETRIES,
$EDITOR host.cmake                #   OMDRC_SITE_DATA_DIRS, MUSIC_DIR,
                                  #   VIDEO_DIR, OMDB_API_KEY,
                                  #   DR_SYNC_REPO, DR_SYNC_BOX
mkdir build && cd build
cmake .. -C ../host.cmake
make
sudo make install                 # -> $PREFIX (default /usr/local)
```

`host.cmake` is the single source of box-specific values. It is an initial CMake cache, so it is read **only by `-C` and only for entries that do not exist yet**: adding `-C` to a build directory that was already configured silently keeps the defaults. The project detects this and warns; start a fresh build directory:

```
rm -rf build && mkdir build && cd build && cmake -C ../host.cmake ..
```

CMake renders every config from `host.cmake` and installs: the DRC engine (`drc.sh` behind the `omdrc` / `omdrc-status` wrappers); the site data (configs + filters for `GEOMETRY` and every set in `GEOMETRIES`, each looked up along `OMDRC_SITE_DATA_DIRS` and reported at configure time); both web UIs (omdrcctrl :9090 as a **system** service running as the audio user, omdrcvideo :9080 as a **`--user`** service because it drives the desktop-session mpv); the DAC-hotplug glue; the MPD and upmpdcli renderer configs; the `browser-nodrc` launchers; the video launchers and `.desktop` entries; and the OS-specific pieces of Part II or III. `make user-install` links the entries that must live in the user's session; menu entries go straight to `$PREFIX/share/applications`. The install prints the OS-specific enable steps. The optional KDE Plasma widget has a target of its own, `make plasmoid-install` (section 6.2).

Running from a checkout needs no install: put a `config.env` beside `drc.sh` and it enters *repo mode*, taking state and site data from the checkout.

5. BruteFIR defaults. BruteFIR reads float precision, partition size and the I/O devices from `~/.config/BruteFIR/brutefir_defaults.conf`. The per-rate configs leave their `input`/`output` blocks empty and inherit devices from it, so it **must** be deployed; the template differs per OS (Linux section 11.1.1, FreeBSD section 14.2.1). Without it BruteFIR (>= 1.1) silently writes a broken `~/.brutefir_defaults`, every start fails with *"Parse error: path not set ... module 'file'"*, and `drc.sh` rolls back to direct output — DRC never comes up at boot.

6. Renderer runtime state. MPD, upmpdcli and qobuzconnect2mpd write persistent state and logs under `/tmp`. On a real host, `make install` runs `scripts/prepare-renderer-runtime.sh` to converge all of those paths onto `AUDIO_USER`, so a reinstall or user rename cannot leave mismatched ownership. It refuses dangerously broad roots, skips an unexpected symlink under `/tmp`, and is skipped under `DESTDIR` staging.

The update path is `git pull`, then `cmake --build build && sudo cmake --install build`; do not edit installed files in place.

Then continue with Part II (Linux) or Part III (FreeBSD).

Chapter 4

Usage: drc.sh, filters, and configuration

4.1 drc.sh — the single control point

`drc.sh <rate>|resamp|cdin|reconcile|restore|off|stop|status|session [variant]`

Verb	Effect
`<rate>`	Start BruteFIR at 44100 / 48000 / 88200 / 96000 / 192000 Hz; restart the loopback at the same rate; switch MPD to `DRC-native`
`resamp`	Everything at 192 kHz; switch MPD to `DRC-resamp` (MPD resamples with soxr)
`cdin`	Persist CD/S-PDIF source mode and request its required 44100 Hz chain
`reconcile`	Compare saved power/source/rate with actual processes, config, rate, nodes and DAC role; repair only a mismatch
`restore`	Re-apply state at shutdown: honour `last_power`, `last_source`, and `last_arg`
`off`	Stop BruteFIR + loopback; MPD back to direct output; **records the off state**. The user-facing disable
`stop`	Same teardown as `off` but does **not** record off. Used by service stop paths so a reboot of a *running* system is restored
`status`, `session`	Report actual chain state or the exact persistent restore tuple
`geometry`, `design`	Show/list/switch the persistent room geometry or audited filter design
`variant`	Optional second argument (a config-filename suffix): selects an alternate filter set; superseded by `design`

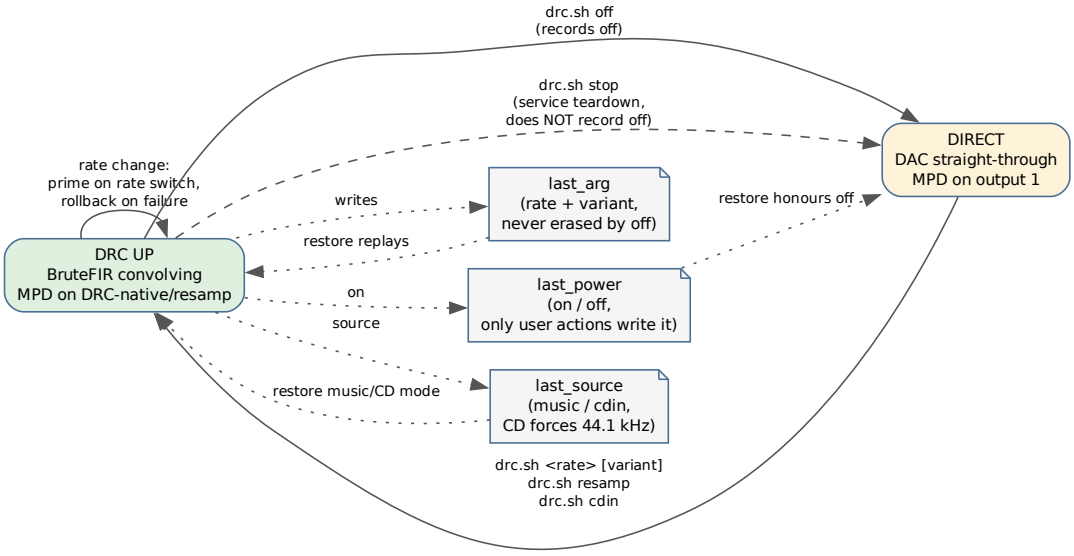


Figure 4.1: drc.sh verbs and the persistent lifecycle state.

State lives in the resolved persistent state directory (git-ignored checkout state in run-from-repo mode, or the configured/installed user state path):

- **`last_arg`** — the last *active* rate and optional variant (`192000`, `resamp`, `192000 <variant>`). Written on each successful run, **never erased by `off`** — turning DRC back on restores the last rate. It records the *desired* state: a failed start never rewrites it, so the next trigger retries the same configuration.
- **`last_power`** — `on` or `off`. Only real user actions write it (a rate run writes `on`, `off` writes `off`; `stop` leaves it alone).
- **`last_source`** — `music` or `cdin`. CD mode survives reboot and forces 44100 Hz; an ordinary rate selection records the return-to-music intent before fallible validation/transition work, so failure cannot restore stale CD mode on the next boot.
- **`cdin-mpd-output`** — the audible MPD output remembered when CD input is selected (`OKTO-DAC`, `DRC-native`, or `DRC-resamp`). Stopping the bridge restores this exact output; the Spectrum FIFO is never recorded or gated.
- **`last_geometry`** — the current geometry override. Design remains part of `last_arg`, so the full tuple can be restored without guessing.

The configuration's `GEOMETRY` is the default; `drc.sh geometry <name>` records a runtime override in `last_geometry`.

A required rebuild does, in order: stop BruteFIR and wait for the DAC to be released; make bounded MPD release requests; stop a running CD bridge and verify its process exit as the output-release acknowledgement; restart the loopback at the target rate; prime the DAC if the rate changed (FreeBSD, section 16.2); start BruteFIR and **verify it stays up**; make a bounded attempt to enable the matching MPD output; record the state. If BruteFIR cannot come up, **`drc.sh` rolls back** — stops the loopback and re-enables the direct output — leaving a clean, audible system equivalent to

`off`.

`drc.sh status` (and `drc-status.sh`) reports the *actual, observed* state — config, loopback rate, BruteFIR, MPD output and rate — derived from what is running, not from `last_arg`.

4.2 Reconcile: saved intent versus reality

`drc.sh reconcile` — run at boot, on DAC hotplug and after MPD starts — compares the saved intent (`last_power`, `last_source`, `last_arg`) with what is physically there: the BruteFIR config and process, the loopback process, rate and nodes, and the DAC role. A matching chain is a no-op. A missing DAC causes a transient teardown that keeps the intent; desired off stays off; a partial or wrong-rate chain is rebuilt once.

`off` writes `last_power=off` *before* touching BruteFIR, MPD, the CD bridge or the loopback: under `set -e` a teardown failure must not leave the previous "on" intent, which would bring DRC back at the next boot. `stop` is different: shutdown and detach use it as a transient teardown and it leaves the desired power alone. `restore` reads state only after taking the DRC lock, closing the time-of-check/time-of-use window.

4.3 MPD outputs

Output	Device	`format`	Meaning
`OKTO-DAC`	direct DAC	—	bit-perfect direct path, no DRC
`DRC-native`	loopback	`*:*:*`	native-rate DRC; MPD does **not** convert; you pick the `drc.sh` rate matching the track
`DRC-resamp`	loopback	`192000:24:2`	MPD resamples everything to 192 kHz (soxr); for mixed-rate playlists

`*:*:*` (rate:bits:channels, asterisk = unenforced) is deliberate: native DRC mode must not force MPD conversion. `drc.sh 192000` and `drc.sh resamp` both use the 192 kHz BruteFIR config but are distinct modes: the web UI shows them as *Flat 192 kHz* vs *Flat auto-resample*. In native mode the sample rate is the routing selector, never the bit depth — BruteFIR processes in floating point.

4.4 The filters/ and configs/ trees

```
filters/<geometry>/<rate>/{L,R}.raw          raw FLOAT64_LE FIR coefficients
filters/<geometry>/<rate>/@<design>/{L,R}.raw   an immutable A/B design
filters/<geometry>/provenance/<design>.json  hash-bound manifest
filters/<geometry>/analysis/<design>.json    precomputed response traces
filters/<geometry>/rew/                      REW-exported source WAVs
configs/<geometry>/brutefir-<rate>[@<design>].conf.in
```

These two trees are the *site data*. They need not live in this checkout: CMake resolves them along `OMDRC_SITE_DATA_DIRS` and the design scripts along `OMDRC_SITE_ROOT`, so one room's measurements can be a separate repository while the engine ships only the generic `flat` set. See Filter provenance and verification.

`drc.sh` builds the config path as `configs/<geometry>/brutefir-<actual_rate><variant>.conf` (for `resamp`, `actual_rate` is 192000) and BruteFIR reads the filter paths from that config — the config is the authoritative link. A variant works only if the matching config file exists; nothing is auto-discovered.

For a new rate or variant, create all the pieces: the `L.raw`/`R.raw` pair, the config pointing at them, and verify the `attenuation:` (below).

4.5 Filter generation workflow

This is the low-level route: it produces coefficients but no provenance bundle, so the web UI cannot verify the result and will not show stored measurements beside it. For deployable work use the audited workflow in Filter provenance and verification; what follows is still useful for experiments and is what the audited path drives underneath.

1. Export the corrected impulse responses from REW as WAV (typically 48 kHz).
2. Convert for every rate directory:

   ```
   scripts/REW2raw-all-rates.sh \
     -L filters/120.blue/rew/FLX-trimmed-48k.wav \
     -R filters/120.blue/rew/FRX-trimmed-48k.wav \
     -o filters/120.blue
   ```

   `REW2raw.sh` (called per rate) resamples with SoX at very high quality (`-v -L -s`, 64-bit float intermediate) and applies **one deterministic FIR coefficient scale** — `scale = Fs_source / Fs_target` (Julius O. Smith, *Physical Audio Signal Processing*) — never peak normalisation, which would alter the intended filter gain. A `sox.txt` log records every command and measured stat.

3. Compute the clipping headroom and set `attenuation:` in each config:

   ```
   python3 scripts/headroom_calc.py
   ```

   BruteFIR works in float64, so clipping can only happen at the output boundary where the filter has gain > 0 dB. The script FFTs each raw filter, takes the worst-case gain across frequency, adds a safety margin (default 1 dB), and reports the suggested per-pair attenuation (BruteFIR applies one `attenuation:` per coeff block to both channels, so the louder channel limits). Attenuation in float64 is lossless — the only goal is avoiding clipping.

The measured result of the current filter set (`120.blue`, v1.5.0):

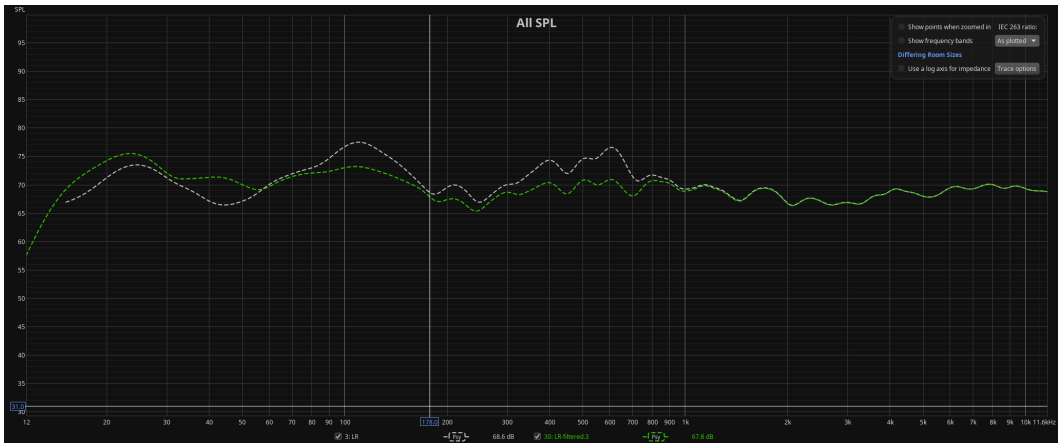


Figure 4.2: Amplitude response: corrected vs uncorrected.

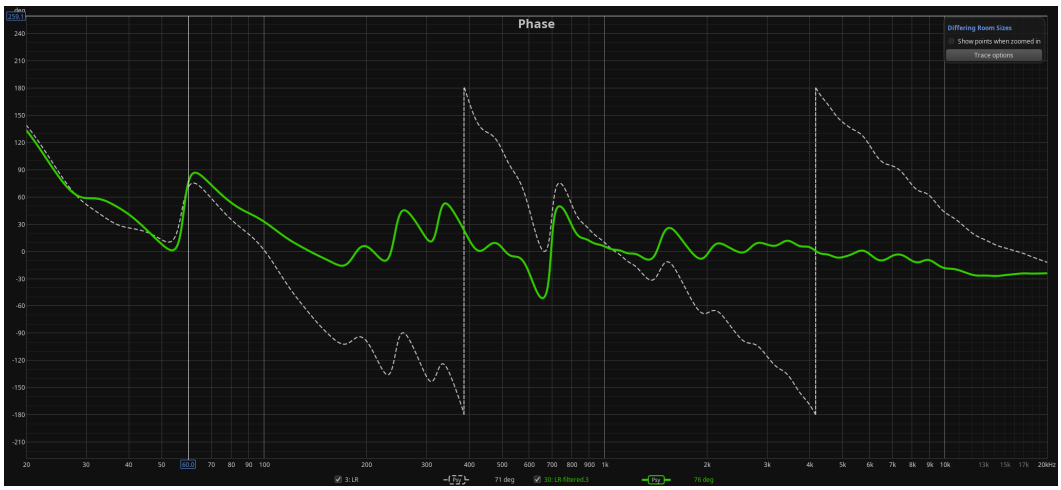


Figure 4.3: Phase response: corrected vs uncorrected.

4.6 Browsers: the No DRC launchers

While DRC runs, BruteFIR owns the raw DAC. A browser cannot share that handle and cannot conveniently reach the loopback, so the supported path stops DRC for the whole browser session. `browser-nodrc/` installs one launcher and one **No DRC** menu entry per browser (Firefox, Chromium, Chrome). Each runs in the foreground and:

1. reads `omdrc session` and remembers the exact power and mode state;
2. arms `EXIT`, `INT` and `TERM` traps;
3. runs `omdrc off`, stopping BruteFIR and enabling the direct MPD output;
4. waits for the DAC to be released;
5. runs the browser in the foreground; and
6. on exit, releases browser audio and reapplies the state from step 1.

The trap reapplies the captured mode rather than calling plain `omdrc restore`, because `off` has just recorded power-off, which a plain restore would honour and leave DRC down. If DRC was

already off before launch, it stays off.

Firefox runs with `--no-remote` so the launcher owns a new instance. Chrome and Chromium cannot: when one is already running, a second invocation merely hands it a URL and exits, so their launchers detect that and leave DRC unchanged. **Fully quit an existing Chromium or Chrome before using its No DRC entry**; otherwise the old process keeps both its audio configuration and its old DRC relationship.

How the browser reaches the DAC while DRC is off is OS-specific: Linux section 11.6, FreeBSD section 16.6. On FreeBSD, Chromium and Chrome have no sound at all without a sound server, and this box cannot run PulseAudio or PipeWire; the launcher starts a private `sndiod` for the browser session instead (section 16.6 explains why and how).

4.7 Helper scripts

Script	Purpose
`REW2raw.sh`	REW WAV -> BruteFIR raw FLOAT64_LE at a target rate, with the `Fs_source/Fs_target` coefficient scale (no peak normalisation)
`REW2raw-all-rates.sh`	Batch: `L.raw/R.raw/sox.txt` for every rate directory under a filter root; prompts before overwriting unless `-y`
`headroom_calc.py`	Minimum `attenuation:` per config from worst-case FFT gain + safety margin
`new_filter_design.py`, `remove_filter_design.py`, `deploy_filter.py`, `verify_filter_bundle.py`, `console_ui.py`, `filter_workflow_next.py`, `rew_mdat_audit.py`	The filter-design commands, described in section 5.5
`verify-bitperfect.sh`	The original single-host bit-perfect proof (FreeBSD, section 16.5)
`bitperfect-lib.py`	Shared engine of the tap scripts and the panel: promotion to the S32 wire container, USB capture decoders, alignment, verdict, report, byte-view readers
`bitperfect-tap-linux.sh`, `bitperfect-tap-freebsd.sh`	Play a WAV to the DAC and record the bytes on the USB wire; same CLI and artifacts on both OSes
`bitperfect_runner.py`, `bitperfect_material.py`	Run a tap through a chosen playback path and prepare its material; back the `/bitperfect` page (chapter 7)
`bitperfect-compare.py`	Compare two tap artifacts from either OS
`omdrc-ctrl/src/drmeter.py`	The DR meter behind *Measure DR* and the live *Estimate DR* (chapter 8)

Script	Purpose
`omdrc-ctrl/src/dr_store.py`	The DR log's album store: which track measurement is kept and how the album figure is decided (chapter 9)

Chapter 5

Filter provenance and verification

A room-correction filter is an empirical artifact: it is only as good as the measurement session it came from, and it is indistinguishable, once converted to a `.raw` blob of float64 coefficients, from any other blob of the same length. The response page in the web UI shows *stored* room measurements — curves that were computed offline, months earlier, from the original REW exports. Showing them beside a filter that is not the one they describe would be worse than showing nothing, because it looks authoritative.

This chapter describes the machinery that prevents that: a chain of content hashes running from the REW exports in the source repository to the coefficient bytes BruteFIR has actually loaded, with a refusal at every link that cannot be checked.

What follows is the synthesis: what the chain is for, and how it hangs together. The *normative* detail lives beside it in `doc/FILTER_PROVENANCE_AND_RESPONSE.md` (built to a printable `doc/FILTER_PROVENANCE_AND_RESPONSE.pdf` by the same script as this manual) — what every export must be, what may never be shown as verified, the exact deployment and removal transactions, and the audit record of the bundles actually deployed in this room. Read this chapter to understand the design; read that document when you need the rule.

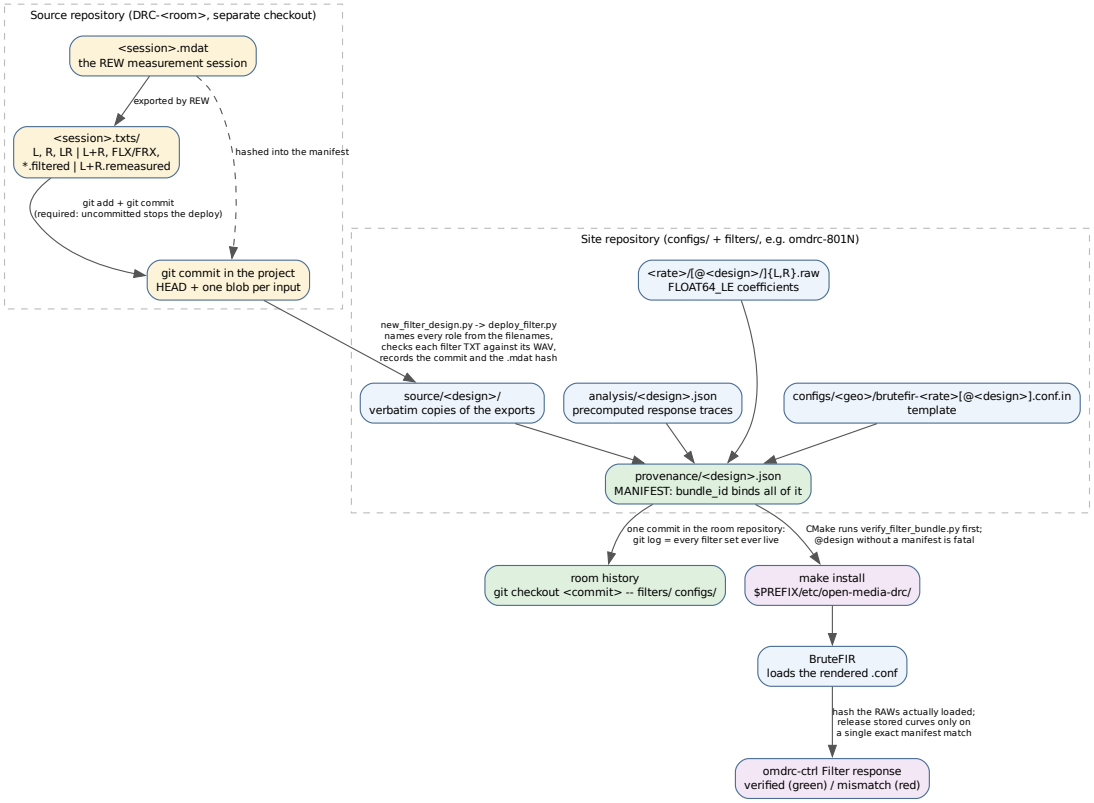


Figure 5.1: Provenance chain: each labelled arrow is a content check that must pass before a design is deployable, installable, or shown as verified.

5.1 The three repositories

The workflow spans three trees, deliberately kept apart:

Tree	Holds	Example
Source repository	one geometry's REW projects (`.mdat`) and the sessions exported from them (`<session>.txts/`)	`../DRC/DRC-120.blue`
Site repository	`configs/<geometry>/`, `filters/<geometry>/` — one physical room	`omdrc-801N`
Engine repository	`drc.sh`, the scripts, CMake, and the generic `flat` set	`open-media-drc`

The site data used to live inside the engine checkout. It no longer has to: `configs/<geo>` and `filters/<geo>` are resolved through a *site root*, so personal room measurements can be versioned

and deployed independently of the software. Two settings control it, and they are not the same thing as the runtime `OMDRC_SITE_DIR`:

Setting	Read by	Meaning
`OMDRC_SITE_DATA_DIRS`	CMake	semicolon-separated *search path* for `configs/<geo>` + `filters/<geo>`; first match wins
`OMDRC_SITE_ROOT`	the design scripts	the one checkout they read and write room data in (also `--site-root`)
`OMDRC_SITE_DIR`	`drc.sh` at runtime	the *installed* `$PREFIX/etc/open-media-drc`

Both default to the engine checkout, which is exactly the historical single-repository layout. Set the search path in `host.cmake`:

```
set(OMDRC_SITE_DATA_DIRS "${CMAKE_SOURCE_DIR};$ENV{HOME}/devel/omdrc-801N"
    CACHE STRING "Search path for configs/<geo> + filters/<geo>")
```

At configure time CMake prints which directory supplied each set, so the substitution is never silent.

A missing *extra* set is a warning and is skipped; a missing default `GEOMETRY` is a fatal error, because installing it would record a geometry in `omdrc.conf` with no configs behind it.

5.2 Geometry, design, variant

Three words that are easy to confuse:

- a **geometry** is a physical setup — speaker and listening position, e.g. `120.blue`. It is a directory under `configs/` and `filters/`.
- a **design** is one immutable filter revision inside a geometry, addressed as `@design-id` (e.g. `@rscreen-20260812`) and carrying a provenance manifest. The historical revision has the reserved id `default` and keeps the un-suffixed paths.
- a **variant** is the older mechanism: an arbitrary suffix appended to the config filename. It survives for compatibility, has no manifest, and is therefore always displayed as unverified. New work should use designs.

5.3 The bundle

Everything belonging to one design lives in the site repository under the geometry:

```
filters/<geometry>/
  provenance/<design>.json          the manifest (the commit marker)
  provenance/<design>.source.json   the build recipe (development input)
  analysis/<design>.json            precomputed response traces
  source/<design>/                  verbatim copies of the ten inputs
    L.txt  R.txt  LR.txt            measured, before correction
```

```
    FLX-trimmed.txt  FRX-trimmed.txt          exported filter responses
    FLX-trimmed-48k.wav  FRX-trimmed-48k.wav  deployable impulses
    L.filtered.txt  R.filtered.txt  LR.filtered.txt   after correction
  <rate>/{L,R}.raw                  runtime coefficients (design `default`)
  <rate>/@<design>/{L,R}.raw        runtime coefficients (immutable design)
configs/<geometry>/
  brutefir-<rate>.conf.in           template; @REPO_DIR@ -> $SITE_DIR at install
  brutefir-<rate>@<design>.conf.in
```

Sources keep the names they had in the export directory, because those names *are* the role assignment. The copy is deliberate: a deployment must never depend on a mutable sibling checkout or on an absolute path that will not exist on the playback machine.

5.4 What is hashed

The manifest records, for every source export, RAW file, config template and the analysis file: logical role, relative path, byte size, format, sample rate/count where applicable, and SHA-256. It also records where the design came from — the source project's repository, branch, HEAD commit, a Git blob id per input, whether everything was committed, and the REW `.mdat` the exports were taken from (name, size, SHA-256, blob id) — plus parsed REW header metadata, the TXT-versus-WAV validation results, per-channel headroom and required attenuation, and the exact rate-to-config mapping with the expected BruteFIR format and attenuation.

The `bundle_id` is the SHA-256 of a canonical identity object: a hash of the complete source block, every source artifact hash, the runtime config and RAW hashes and settings, and the analysis hash. Editing any provenance value the UI displays therefore invalidates the bundle instead of quietly changing a label.

A hash proves bytes; it cannot say where they came from or bring them back. The commit id does that. It is the only part of the chain that is not self-verifying, which is why deployment refuses to run until the exports and the `.mdat` are committed.

5.5 The scripts

All of them are offline. None starts REW, and none opens a `.mdat` except the optional auditor.

Script	Role
`new_filter_design.py`	the one deployment command: reads one export directory, resolves every role from the file names, checks each filter TXT against its impulse WAV, hashes the `.mdat`, requires the project commit, reports, asks, publishes, and commits the room repository

Script	Role
`remove_filter_design.py`	the exact inverse: removes one design completely — manifest, analysis, source copies, every rate's coefficient pair and every config template — manifest first, then records the removal in the room repository. Refuses the reserved `default` set
`deploy_filter.py`	the engine underneath: regenerates every requested rate in a temporary directory, validates TXT against WAV, computes headroom, bakes and reads back each config, carries the exports into the analysis file unchanged, and writes the bundle
`verify_filter_bundle.py`	read-only re-verification of committed bundles: bundle id, source copies, analysis dependencies, configs, exact RAW hashes and headroom. `--no-next` for CMake and CI
`console_ui.py`	shared terminal contract for publication and removal: stages, colours, confirmation, warnings, and the uniform failure line
`filter_workflow_next.py`	shared operator handoff — prints the exact commit/install/select/verify commands, and names a working directory per step when the site data lives in its own repository
`headroom_calc.py`	minimum `attenuation:` per pair from the worst-case FFT gain plus a safety margin; also reports what each config currently specifies
`rew_mdat_audit.py`	optional archival evidence: audits selected REW project traces via the REW API, comparing TXT responses numerically and final WAV impulses sample-by-sample against the project. Not a deployment dependency
`REW2raw.sh`, `REW2raw-all-rates.sh`	the low-level SoX conversion underneath, usable directly for experiments; they produce no provenance bundle

5.6 The workflow

Two things are manual: naming the exports by the convention and committing them, and reading the tool's report before answering its confirmation prompt. Everything else — coefficients for every rate, BruteFIR configs, headroom, manifest, the site-repository commit — is one command:

```
# 1. In the source repository: give the exports their imposed names, commit them
#    together with the .mdat they came from.
git -C ../DRC/DRC-120.blue add -- 120.blue.Rscreen.txts 120.blue.Rscreen.mdat
git -C ../DRC/DRC-120.blue commit -m 'Rscreen measurement session'
```

```
# 2. In the engine repository: one command, one directory.
export OMDRC_SITE_ROOT=~/devel/omdrc-801N
python3 scripts/new_filter_design.py ../DRC/DRC-120.blue/120.blue.Rscreen.txts

# 3. Re-verify independently, then push the room's history.
python3 scripts/verify_filter_bundle.py --all --require-sources
git -C ~/devel/omdrc-801N push
```

Nothing on the command line says what a file is: the names are the role assignment. They are `L.txt`, `R.txt`, `LR.txt` (or `L+R.txt`), `FLX-trimmed.txt`, `FRX-trimmed.txt`, the two impulse WAVs, and `L.filtered.txt`, `R.filtered.txt`, `LR.filtered.txt` (or `L+R.filtered.txt`, or `L+R.remeasured.txt` for a re-measurement with DRC running). A missing name, a duplicate spelling or a mismatched aggregate style stops the run before anything is written.

Three checks run first, and none has an override:

- every text export must be **unsmoothed** (`* Smoothing: None`; an export stating no smoothing is refused too) — REW's smoothing is baked into the numbers, while the browser's Smoothing selector is a separate, reversible view;
- every export must come from a **measurement at 48 kHz or below** (it must not reach past 24 kHz), because all runtime coefficients are resampled from one 48 kHz impulse;
- each **filter TXT must be the exported response of its WAV**: one integer causal delay and one constant export gain are detected, and the residual magnitude and phase errors must stay inside the declared limits. This is what makes the plotted FLX/FRX curve a statement about the bytes BruteFIR will load.

The command then reports the eight curves the panel will plot, the project and session behind them, the two filters with their residuals and every path it will write, and asks. `--dry-run` runs every check, including the SoX conversions, and stops.

The flags are narrow. `--yes` supplies confirmation but skips no check. `--replace-design` permits differing bytes under an existing design id, never partial publication. `--allow-uncommitted` records the source-recoverability gap as `clean: false`, visible to the verifier and UI. `--no-commit` skips the room-history commit. `--site-root` selects the site checkout (separate from CMake's `OMDRC_SITE_DATA_DIRS`).

Publication is a transaction: source copies first, then the analysis, then the runtime RAW pairs, and the manifest **last**. The manifest is the commit marker — readers ignore an incomplete deployment until it exists and verifies every preceding hash. A design owns `filters/<geo>/source/<design>/` and every `filters/<geo>/<rate>/@<design>/`, so a redeployment prunes what the previous one left there (after the manifest, so a failure leaves harmless leftovers). A bare `<rate>/` directory belongs to `default` and is never touched.

Removal is the exact inverse:

```
python3 scripts/remove_filter_design.py --list
python3 scripts/remove_filter_design.py 120.blue@rscreen-20260812
```

It deletes the manifest **first** (removing it is what makes the design cease to exist for every reader), then the recipe, analysis, source copies, every `@design` coefficient directory and config template, and nothing else. The reserved `default` set is refused, because a geometry falls back to it.

Each publication or removal ends with one commit in the room repository naming the geometry, design, bundle id, project commit, session hash and rates. `git log` lists every filter set that was ever live, and `git checkout <commit> -- filters/<geo> configs/<geo>` restores any of them byte for byte. Verification never depends on it.

5.7 Deploying the verified identity

Publication changes the site repository, not the installed playback tree. The handoff has four boundaries: the source-project commit makes every export and the `.mdat` retrievable; the site-repository commit records the published bundle; CMake verifies and copies that bundle; and the running BruteFIR config plus exact L/R RAW hashes determine whether the web UI can go green.

On the design machine, `new_filter_design.py` makes the site commit by default. Re-verify those bytes and push the site repository:

```
export OMDRC_SITE_ROOT=~/devel/omdrc-801N
python3 scripts/verify_filter_bundle.py --all --require-sources
git -C "$OMDRC_SITE_ROOT" status --short
git -C "$OMDRC_SITE_ROOT" push
```

On the playback machine, pull the site repository. `host.cmake` must include it in `OMDRC_SITE_DATA_DIRS`. Initialise a first build from that file; later deployments can reuse the correctly initialised cache:

```
git -C ~/devel/omdrc-801N pull

# First build for this host
cmake -C host.cmake -S . -B build

# Later deployments may reconfigure the existing cache
cmake -S . -B build

cmake --build build
sudo cmake --install build
```

Configure prints the checkout that supplied each geometry and runs the read-only verifier before anything can be installed. The install contains the RAWs, rendered configs, manifests and analysis JSON; source exports and recipes remain development-only in the site repository.

Restart the panel with your OS's service manager (Linux section 11.7, FreeBSD section 16.3), select the installed geometry and immutable design, and verify what is actually running:

```
/usr/local/bin/omdrc geometry 120.blue
/usr/local/bin/omdrc design --list
/usr/local/bin/omdrc design @rscreen-20260812
```

Open `http://<box>:9090`, enter **Filter response**, and require the green identity to show the selected geometry/design and the complete `bundle_id` printed during publication and verification. A missing or different identity means the curves must not be trusted. The scripts' **NEXT** block is the host-specific version of this sequence and names the correct working directory for each step in a split engine/site layout.

5.8 Why publication and runtime are separate

There are three lifecycle stages, not three arbitrary copies:

Stage	Example	Purpose
Working source	`~/DRC-120.green`	Editable REW project, measurements, experiments
Published bundle	`~/omdrc-801N` or `~/.local/share/omdrc/site-data`	Frozen, verified, reproducible release
Runtime	`/usr/local/etc/open-media-drc`	Only the files playback and the UI need

- The REW project is mutable — you can reopen the `.mdat`, rerun FDW processing or replace exports — so it cannot by itself prove which inputs produced the installed filter. The bundle freezes one release: the exact `.mdat`, exports, RAW coefficients at every rate, config templates, analysis, hashes, source commit and bundle id.
- The runtime tree is deliberately narrow: system-owned, replaceable atomically, regenerable from the bundle, and free of bulky evidence (one `.mdat` is about 61 MB and BruteFIR does not need it). A narrowly privileged helper installs verified files without giving the web application write access to `/usr/local`.
- A browser upload has no stable local Git repository, so the web workflow publishes into a user-owned store, `~/.local/share/omdrc/site-data` by default: writable without root, persistent, not necessarily Git-managed. Only the verified runtime subset crosses the privilege boundary.

`omdrc-801N` (Git-backed) and `.local/share/omdrc/site-data` occupy the same layer, so having both means two authorities. Point the web installer's `design_root` at the Git-tracked site repository and there is one:

```
REW projects / web uploads --> ~/omdrc-801N (one Git-tracked authority)
                           --> /usr/local/etc/open-media-drc (runtime only)
```

5.9 Live browser-driven installs

The sequence above is the offline/Git path. The panel's **`/configuration`** page (section 6) drives the same audit/build engine from a browser on the trusted LAN and is the normal way to install or remove a design. The command line remains for offline publication and provisioning; it is not a follow-up to a web install.

The browser uploads one REW `.txts` directory plus its `.mdat`. `omdrc-ctrl/src/configuration.py` stages them in a private per-job directory and rejects anything but an exact-basename pair, a mixed export folder or a mismatched `.mdat`. (Selecting the common parent fills the session field automatically; selecting the `.txts` directory itself leaves one explicit `.mdat` choice, since a browser cannot inspect `../`.) `new_filter_design.py` gained the flags this needs: `--live` publishes runnable `.conf` files instead of `.conf.in` templates, `--coefficient-root` bakes an absolute live path when staging happens elsewhere, `--archive-mdat` copies the full `.mdat` into the bundle, and `--upload-provenance` records the upload's own name instead of a temporary path.

Every web publication first enters the persistent `[configuration] design_root`. If that is a Git work tree it must be clean, and the deployment commit is required before runtime installation, exactly as in the manual sequence; otherwise it is a plain managed folder using the explicit uncommitted mode. When the live site is not writable by the web process, the staged design goes to `scripts/omdrc-config-helper.py` (the same helper that pins audio roles, section 6.4), which re-verifies every claimed hash before copying and writes the manifest **last**. Reinstalling a byte-identical live bundle is a verified no-op. Progress streams back over Server-Sent Events. Installation never activates a design, and an active or saved design must be switched away before removal.

5.10 Verification at install time

`cmake/core-drc.cmake` runs `verify_filter_bundle.py --require-sources` over every manifest of a geometry *before* installing it, and fails the configure step if a bundle does not verify. It also rejects any `@design` config that has no same-named manifest. A broken bundle therefore stops the build rather than producing an installed system that looks authoritative.

The install copies the manifests and analysis JSON beside the RAWs, and deliberately excludes `rew/`, `source/` and the `.source.json` recipes: their hashes and parsed metadata are already embedded in the installed manifest, so the playback machine needs none of the development material.

5.11 Verification at runtime

The response page never trusts the geometry name or the rate. For every request `omdrc-ctrl`:

1. finds the `.conf` of the **running** BruteFIR process from its argv;
2. parses the coefficient blocks and hashes the exact `.raw` files named there;
3. requires exactly one manifest matching that `(relative path, SHA-256, format, attenuation, rate)` tuple;
4. verifies the analysis file's own hash and that its recorded input hashes equal the manifest's source artifact hashes;
5. releases the stored exports **unmodified** — no scaling, offset, attenuation subtraction or resampling stands between the analysis file and the browser.

Only then are the stored room measurements released, with the green banner carrying the bundle id; the details panel adds the export directory, the source project and commit, the `.mdat` behind the measurements, every plotted export with its hash and the active L/R RAW hashes.

Anything else is **mismatch** (red): no manifest matches the active bytes, a hash differs, the config attenuation or format differs, or an analysis dependency fails. **No graph is drawn** — not even a diagnostic FFT of the live coefficients, because a calculated curve on a page whose whole promise is *these are REW's numbers* is worse than no curve. A legacy variant, having no manifest, always lands here.

A/B switching obeys the same rule. After `drc.sh` returns, the server re-reads the running process, parses the config actually in use, hashes its RAWs, and reports the new selector as verified only if that runtime identity matches one manifest. A design that starts but fails this check is an assurance failure, not a successful switch.

5.12 What deliberately cannot go green

- A pair whose bytes do not reproduce from the recorded source exports. It must be re-exported and redeployed as a new bundle.
- Any selector without a manifest, whatever its audio quality.
- A design whose configured attenuation is below the computed requirement: the base `120.blue` pairs peak at about +1.28 dB and need 2.3 dB including the 1 dB margin, and are configured at 3.0 dB. A design whose filters never exceed unity gain legitimately requires 0.0 dB.

One gap is worth naming explicitly: the manifest pins the config *template*, not the rendered `.conf` that BruteFIR loads, because `@REPO_DIR@` is only substituted at install time. Runtime verification closes this for everything that matters — coefficients, format, attenuation and rate are all re-checked against the bytes in use — but a post-install hand-edit to a rendered config's routing or device settings is outside the chain.

Chapter 6

The web panel: omdrc-ctrl

A Flask app serving a dark, touch-friendly control panel to any browser on the LAN (default `0.0.0.0:9090`). Originally a replacement for KDE Connect's feedback-less "Run command" plugin. Everything is driven by a plain INI file, `commands.conf`; no command is hard-coded.

Widget types — each `[section]` of `commands.conf` is one command:

- **READ** — runs a shell command, shows its output next to a label, optionally auto-refreshing (`refresh = N` seconds). A READ widget with `details_root` gains a dynamic **Details** button whenever `{details_root}/{output}/README.md` exists — so each filter configuration can carry its own documentation page with images.
- **WRITE** — a labelled button firing a command; green on success, red on failure, optional confirmation dialog (`confirm = yes`).
- **LINK** — opens a URL in a new tab.

Built-in monitoring panels (always present, below the command cards):

- **MPD panel** — the audio-health centrepiece for a headless server: daemon state, playback state and song, the stream MPD reports (rate/bits/ channels), the **DAC feed** (the rate the DAC actually receives), the BruteFIR rate, a green/red **SAMPLE RATE MATCH / RESAMPLING** comparison, and a plain-language **bit-perfect verdict**: *Bit-perfect passthrough* (DRC off, all rates equal), *Full-resolution DRC, no resampling* (BruteFIR at native rate), or *Resampling active*.
- **Qobuz Connect panel** — current track via qobuzconnect2mpd, with a restart button and colour-coded log viewer, plus a renderer switch (qobuzconnect2mpd vs upmpdcli — never both) that drives the OS service manager.
- **CD input panel** — what the CD bridge is doing, read entirely from its log (chapter 10).
- **BruteFIR CPU** — per-process CPU for every brutefir instance (matched by `argv[0]`); **Top CPU** — processes above a configurable threshold.
- **Debug card** — the glitch-detection switch (FreeBSD, section 16.4).

Some cards exist on one OS only (device listings, DAC diagnostics); they are in section 11.7 (Linux) and section 16.3 (FreeBSD).

DRC filter response page — charts the *live* filters loaded by the running BruteFIR: magnitude (dB), delay-compensated wrapped phase, and residual group delay, computed on demand by FFT (NumPy) from the active config's `L.raw`/`R.raw`. Chart.js is vendored, so the page works offline. The

filter files are only ever read, never modified.

Live spectrum analyzer — an optional card fed from a FIFO of raw S32_LE stereo. It FFTs whatever arrives and knows nothing about the writer, so more than one part of the chain can feed it. `source` picks the producer: `mpd` (a secondary `OMDRC Spectrum` fifo output), `cdin` (the CD bridge), or `auto` (default), which takes `cdin` while a disc is playing and `mpd` otherwise. CD audio never passes through MPD, so without the `cdin` source the analyzer is blank for a whole disc. The rate travels with the source (CD is 44.1 kHz, the MPD FIFO 44.1 kHz): analysing at the wrong rate mislabels every bin without any error. How CD samples reach the FIFO is OS-specific (section 11.7 or 16.3).

- Started and stopped from the page; the source is enabled only while a browser is streaming (Server-Sent Events; clients share one capture thread) and is force-disabled at startup for crash recovery.
- `/spectrum/stream?max_s=N` asks for a **bounded stream**, ended by the box after N seconds (5 to 3600) and confirmed by an `X-Stream-Max-S` header. It is for clients that cannot close a stream: Qt's QML `XMLHttpRequest.abort()` leaves the socket open and downloading, so the Plasma widget (section 6.2) asks for 20 s streams, each opened before the previous one ends, and the analyzer stops at most 20 s after it stops listening.
- 24 logarithmic bands from 31.5 Hz, 25 Hz refresh, Music (16384-point) or Precision (65536-point) FFT windows, VU bars or needles over a ~50 ms window, one Floor slider for graphs and meters.
- Each band peak-holds across the publication interval, so short transients cannot fall between FFT windows. Rises are immediate; the browser animates only the release at `fall_db_per_s` (30 dB/s by default).
- **Meter timing**: the tap is at the top of the audio path, so the display is held back, clock-anchored, by what the box can measure (filter group delay and convolver partition with DRC on; the DAC buffer with DRC off) minus `drc_delay_margin_ms` (150 ms), re-derived every 2 s while a meter is open. The web panel and kiosk wait out the rest with a per-network, per-configuration browser profile, calibrated with the phone's microphone and click track or set by eye (Config -> Meter timing / **Screen delay**). The Plasma widget follows the reported remaining margin by default and also has a per-widget manual delay. A screen can only wait, so a calibration that finds the meters late calls for a larger margin.
- With `source = auto` an open card follows MPD-to-CD hand-offs without closing the browser stream. Writer loss and FIFO replacement are detected, stale pre-pause history is dropped on resume, and a disconnected browser stream reconnects.
- **FIFO ownership**: MPD owns and creates its FIFO; the analyzer enables MPD's Spectrum output first and opens the inode MPD created, never replacing it (MPD would silently keep writing an unlinked inode until restarted). The CD FIFO is the opposite: omdrcctrl creates it, and the appearance of a reader tells the bridge to tee samples. Identical frames are suppressed, so paused playback produces no SSE traffic.

Install: omdrcctrl has no standalone deployment. Configure and install the top-level project so the panel, wrappers, site data and state share one host configuration; the service definition is in section 11.7 (Linux) or 16.3 (FreeBSD).

Security: the server executes arbitrary shell commands from `commands.conf` as the service user — trusted LAN only, never a public interface.

6.1 The kiosk and the Android app

The panel serves a second, touch-first UI at `http://<box>:9090/k/`: the **kiosk**. It is built for a 7" 1024x600 landscape screen and works on a phone, where the **Android app** (`android/omdrc-app`) opens it full screen in a reused WebView. The kiosk is a pure client of the panel's routes, in its own package; the only seam is one `init_app` call at the end of `app.py`.

Pages (swipe, or the tab bar; the top bar's menu on a phone): Now, Cover (optional), Qobuz (optional, section 6.3), DRC, DR, Source, Chain, System, Logs, Config. The top bar slides in on a tap on empty page space and hides after a few seconds; a tap on any control (buttons, fields, chips, sliders, the cover) never pulls it in. It carries the page's own switches (on Now: level display, cover art, DR, balance), the search icon (when the Qobuz search is enabled), the rotate button, keep-awake and fullscreen. `/k/?embed=<page>` (for example `?embed=qobuz`) shows one page alone, for embedding it elsewhere (such as the Plasma widget): no top bar, tab bar, Qobuz player strip or screen saver.

Now, in landscape and upright. In landscape Now shows needles (or bars, or bars plus spectrum) beside DR and balance. Upright, the cover takes a third of the height with title, artist and album beside it; play state and time sit on their own row under the cover; the meters span the full width with DR and balance under them. The **level display is chosen per orientation**: upright it is bars unless needles are picked while upright, in landscape needles unless something else is picked there, and turning the phone switches between the two at once. **Seeking** upright takes two steps, so a stray touch never moves the music: the first touch on the cover only brings up a thin ring inscribed in it (12 o'clock is the start of the track); a slide that then starts *on* the ring moves its knob round, drawn thicker while the finger is on it, and lifting the finger seeks there.

6.1.1 Meter timing and the Calibration page

Open **Config -> Meter timing** to calibrate the VU needles, level bars and spectrum against the sound heard at the phone. This is a display timing adjustment: it does not delay the music, change room-correction filters, or measure the acoustic frequency response. **How it works** opens the illustrated explanation; **Calibration log** opens the last measurement and its raw data.

The two paths. The analyzer receives a copy of the audio upstream of the speakers. Audio then passes through the active FIR/convolver and output buffers; the display data passes through the controller, network and drawing device. These paths need not take the same time. The target is the same musical event heard and drawn at the same moment, rather than simply drawing every frame as soon as it arrives.

The analyzer computation is shared, but its SSE stream has a separate connection for each client. Clients receive copies of the same published frames; they do not compete for or consume a single message queue. Each client can therefore wait a different amount before drawing the same event. The localhost browser can wait longer than a phone whose network transit uses some of the early lead. Both displays can line up with the sound at once, provided the shared controller margin leaves enough lead for the slowest route.

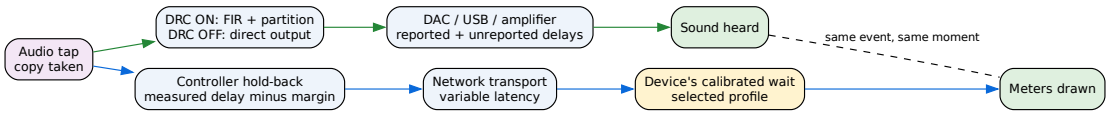


Figure 6.1: The audio and display paths split at the tap and should meet at the same moment.

The controller measures filter group delay from the peak index of the active impulse response divided by its sample rate. BruteFIR partition latency is `filter_length` divided by that rate. With DRC off, the controller instead reads queued DAC audio from ALSA on Linux or sound(4) channel buffers on FreeBSD. Stages that are not reported reliably, including other buffering and transport, are left to calibration rather than guessed. The measured terms are refreshed about every two seconds while the analyzer runs.

Let **M** be the sum of measured audio delays and **R** the requested controller margin (150 ms by default). The effective margin is **min(M, R)** and the controller holds frames back for **H = max(0, M - R)**. Sending frames early leaves room for network transport and the screen's extra wait. The effective margin can change when playback starts or stops; this does not itself mean that the user changed configurations.

The screen's part. Calibration estimates how far the meter events lead the microphone events. A positive result becomes this device's extra screen delay. For example, with a 300 ms controller hold-back, 80 ms transport time and sound heard 600 ms after the tap, the remaining wait is **600 - 300 - 80 = 220 ms**. These are illustrative numbers, not defaults or measurements of a particular installation. The Now timing dialog reuses this example from the information page and identifies it as illustrative.

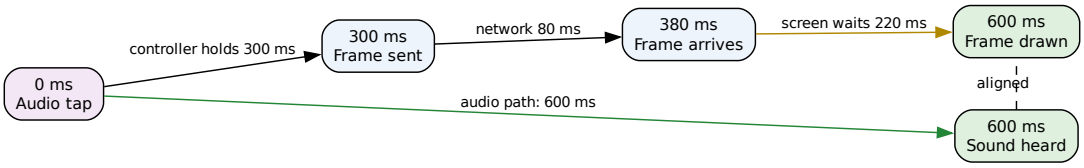


Figure 6.2: Example timing: hold-back 300 ms, network 80 ms, screen wait 220 ms, sound and drawing at 600 ms.

A screen can only wait. If the frames arrive after the sound, a negative result cannot be applied as a negative screen delay. The Calibration page asks the box to send earlier by increasing its margin. It requests the measured lateness plus a 20 ms cushion, limited to the hold-back available; the endpoint also enforces its configured margin limit. Any time successfully released becomes available to transport and the screen. If the box was not holding frames back, or cannot release enough, the page reports the remaining lateness rather than promising alignment that the screen cannot achieve. The controller margin is shared by all clients; other screens compensate their saved delay for effective margin changes within their active profile.

Using the page. First choose the network and DRC configuration to calibrate. The Android app identifies its network when available; **Identify Wi-Fi** requests identification, and Android can require precise location permission and Location to reveal the Wi-Fi name. **Set network name** provides a manual profile name when automatic identification is unavailable; use `wired` for Ethernet. Manual names do not detect a subsequent network switch: update them yourself when necessary. The microphone workflow requires an identified or manually named network in an app that supports network identification.

After calibration, a yellow reminder advises repeating the measurement for all networks, sample rates and filters actually used. It remains on Meter timing after a successful run. Rate fallback provides an initial value, not evidence that every combination has been acoustically verified.

Tune with clicks is the preferred precise measurement. Hold the phone near the speakers at the usual listening position/volume and press **Start**. The page shows the level bars and performs two steps: measure and apply the extra delay, then play the test again and check the alignment of frames actually drawn. **Check again** repeats the check and can correct the residual delay; repeat it after a correction. Read the result before leaving the sheet.

The click test stops current music and does not resume it afterwards. It plays in a separate MPD partition, borrowing the enabled audio/analyzer outputs and returning them afterwards. It leaves the main queue intact, including a queue owned by Qobuz Connect. With DRC running, the test WAV uses the running DRC rate so calibration does not intentionally rebuild the chain. Otherwise the controller uses the reported MPD playback rate when available, falling back to 48 kHz.

In an ordinary browser without the Android microphone bridge, **Play the clicks** and the delay controls remain available. Adjust by eye until the bars flick with the sound. The delay controls move by 10 or 50 ms, **Reset** saves zero for the active combination, and stored screen waits are bounded to 0–3000 ms.

Click algorithm. The track contains 14 quiet, 8 ms Hann-windowed bursts of 1 kHz tone at -30 dBFS. The first burst follows 1 s of silence; successive gaps are 700, 530, 860, 610, 940, 480, 770, 650, 890, 560, 720, 830 and 590 ms. There is a 1.2 s silent tail, giving about 11.3 s total. Irregular spacing prevents a repeated rhythm from producing several equally plausible offsets.

A measurement opens the level stream and lets it settle for 1.5 s. The Android app records a capture-time-stamped peak envelope at 10 ms steps. It continues requesting hardware timestamps after startup and anchors sample zero to the median of timestamp observations taken about every 100 ms. A fixed mapping between monotonic and wall time is used throughout the recording. The central 80% of timestamp anchors must span no more than 20 ms; otherwise recording fails. If hardware capture timestamps never become available, microphone calibration refuses to persist a read-time estimate and offers manual click adjustment instead. Device clock steps over 20 ms during a run also invalidate it. These checks distinguish when samples were captured from when a thread happened to read them; the latter can vary between runs.

Click measurement and verification each request 14 s of microphone recording. One second after requesting the microphone, the page requests playback of the test track. It collects only its dedicated VU stream alongside that recording (concurrent spectrum and clipping streams are excluded to avoid duplicate events). It allows a 1.5 s tail for the last frames, extended by the screen wait when checking drawn frames.

The detector finds onsets rather than trying to match identical waveform shapes: a short burst can ring much longer in a room than in the meter envelope. It sorts meter frames by time and estimates

their silence floor from the 20th percentile. A meter onset must exceed that floor by 15 dB and rise by more than 15 dB from the previous frame, be the first frame, or follow a gap over 150 ms. There is no fixed -50 dBFS cutoff: clicks attenuated by the playback path can still be detected. Microphone onsets must exceed the recording's 20th-percentile floor by 10 dB (with a threshold no lower than -80 dBFS), rise over the minimum of the preceding five steps by more than 10 dB, and be over 200 ms apart.

The algorithm tries offsets formed by differences between microphone and meter onsets, within **-600 to +2500 ms**. For each offset it counts meter onsets having a microphone onset within **+/-40 ms** of their predicted time. A result needs at least **9 of the 14 clicks** paired, and at least four more pairs than the best alternative more than 100 ms away. Pairing is one-to-one: a microphone onset cannot count twice. The final delay is the median of the paired time differences. The median offsets in the first and second halves of the matched events must agree within 30 ms, and median absolute deviation from the final offset must be at most 20 ms. These guards reject a shifting timeline or an excessively scattered recording even if enough clicks match. The match score is the number paired divided by 14; the reported margin is the difference in pair count from the competing offset. Too few detected onsets or an ambiguous match produces an error and does not apply a new delay.

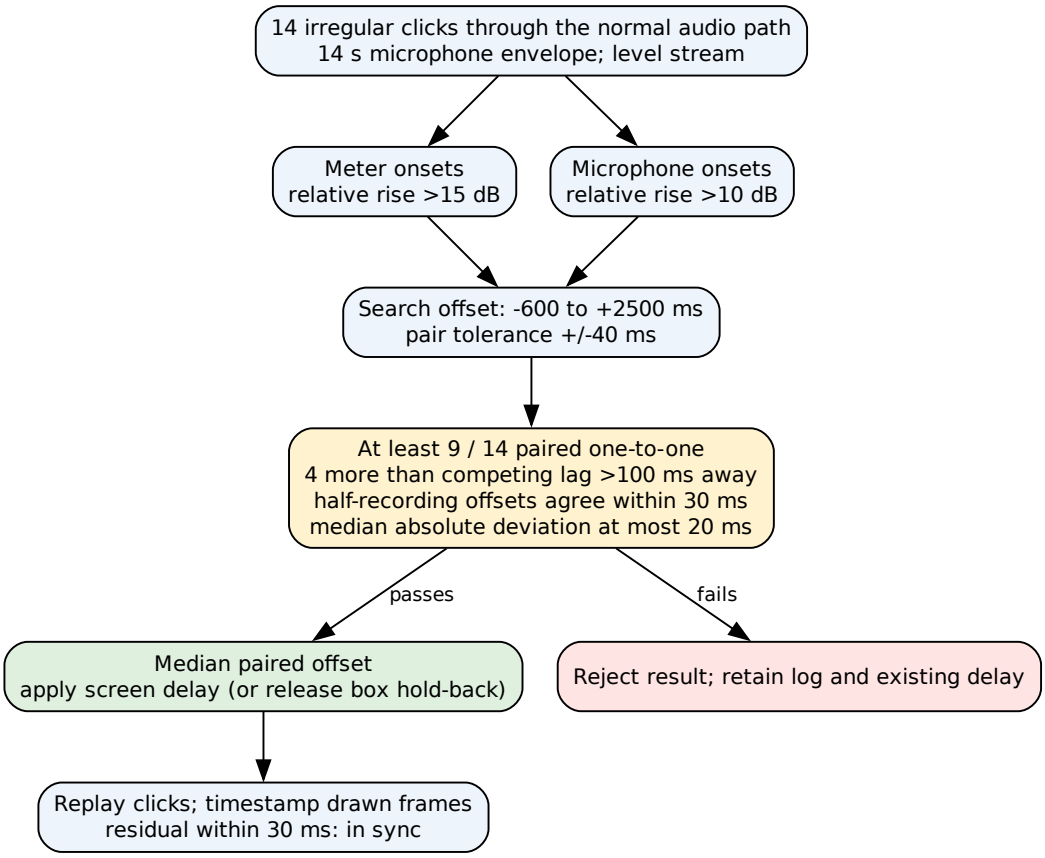


Figure 6.3: Irregular click onsets: the correct shift aligns the meter sequence with the microphone sequence; verification repeats the test after applying the delay.

Verification. Step 1 times incoming meter events; Step 2 times them when actually delivered for drawing, after the screen delay and animation-frame scheduling. Therefore the second result is the residual difference, not another absolute delay to replace the first. Within 30 ms is reported as in sync. A positive residual means the bars still lead the sound and need more wait; a negative residual means they trail it. The page also reports the spread of paired click offsets. A broad spread indicates jitter or uncertain onset detection even when the median is small. Drawing timestamps represent delivery to the drawing code, not a measurement of the physical display's pixel response.

Calibrate on the music. This option records 10 s of ordinary playback, without playing the click track. It compares microphone and meter peak envelopes on a 10 ms grid. Microphone peaks get a trailing 50 ms maximum to match the meter peak window, and levels are bounded to -60–0 dB. Between frame times the meter series is interpolated; a gap over 150 ms holds the previous level until the last 55 ms rather than inventing a slow ramp across silence.

The algorithm removes a trailing one-second moving mean and searches lags from -600 to +2500 ms in 10 ms steps using normalized correlation. Each lag needs at least 200 overlapping points. A parabola through the best point and its neighbors refines the peak below the grid spacing. Acceptance requires correlation **r >= 0.4** and a lead of at least **0.06** over any candidate more than 150 ms away. The same estimator also runs on the first and second halves of the recording; both must be accepted and agree within 30 ms, so a strong full-recording peak cannot conceal changing timing. Clear, varied dynamics work better than steady sound or a very regular beat. The manual music action tries up to three runs for failures that can plausibly improve, and shows the progress and result in the log sheet.

Automatic recalibration is off by default. When enabled, it uses an 8 s music measurement on Now after track changes or playback starts, at most once every two minutes. A detected playback start requires at least 1.5 s below -55 dB, followed by a level above -40 dB. It accepts only successful, nonnegative results with **r >= 0.5**, requires three recent accepted values agreeing within a 40 ms range, and uses their median only if it differs from the current delay by more than 15 ms. An existing dedicated profile moves by at most 20 ms per accepted adjustment; a combination without its own calibration adopts the agreed median directly. One noisy result cannot immediately overwrite stored timing. History resets when the network/configuration/margin context changes. It operates only on Now and does not pre-calibrate every combination or run the click test by itself. A blue calibration indicator and Android's microphone indicator show when measurement is running.

What is stored: device x network x audio configuration. Each device has its own calibration. The Android bridge stores profiles in native preferences so a controller address change does not erase them; an ordinary browser uses storage for its origin. The profile selects the network together with the actual loaded DRC identity: on/off, source, DRC config path, rate, coefficient filenames and formats/modification times, and partition size. DRC OFF does not use the selected filter or subdivide profiles by music sample rate. Within a given audio source it is one profile per network. A source change can select a different profile.

The following table assumes **one phone, MPD as the source, unchanged filter files and partition settings**, and two named networks. All waits are invented examples; use the measured values for your system. Every row is independently saved. There are nine combinations per network, eighteen in this example: DRC OFF is shared across the two selected filter families.

Network	DRC	DRC rate (kHz)	Filter/configuration family	Example saved wait (ms)
Home	OFF	—	Not applicable	90

Network	DRC	DRC rate (kHz)	Filter/configuration family	Example saved wait (ms)
Home	ON	192	120.green.multipos.fdw6	210
Home	ON	96	120.green.multipos.fdw6	180
Home	ON	48	120.green.multipos.fdw6	160
Home	ON	44.1	120.green.multipos.fdw6	155
Home	ON	192	120.blue.blah.blah	225
Home	ON	96	120.blue.blah.blah	195
Home	ON	48	120.blue.blah.blah	175
Home	ON	44.1	120.blue.blah.blah	170
Studio	OFF	—	Not applicable	70
Studio	ON	192	120.green.multipos.fdw6	190
Studio	ON	96	120.green.multipos.fdw6	160
Studio	ON	48	120.green.multipos.fdw6	140
Studio	ON	44.1	120.green.multipos.fdw6	135
Studio	ON	192	120.blue.blah.blah	205
Studio	ON	96	120.blue.blah.blah	175
Studio	ON	48	120.blue.blah.blah	155
Studio	ON	44.1	120.blue.blah.blah	150

Calibrate once for each combination actually used, not every theoretically available combination. Switching back automatically restores its saved profile. A new DRC rate without its own calibration can provisionally reuse the saved delay for the **same network, source, configuration directory and filter design selector** at another rate. If more than one is available, it chooses the nearest rate by ratio, with the most recently saved value breaking a tie. The status says **Provisional timing from ... kHz — calibrate this rate**. Borrowing does not create a dedicated saved profile; calibrating the active rate replaces the fallback with its own value. For example, Home/green/192 can seed Home/green/96 or /48, but cannot seed Studio/green/96, Home/blue/96 or DRC OFF. It is an approximation because unmeasured buffering can still vary between rates. Without a compatible fallback the wait is zero and status is **Not calibrated**. Older network-only profiles are retained but not used as rate fallbacks. Modifying a filter file or changing partition size creates a new identity even if its displayed name stays the same. Legacy unidentified browsers/older bridges retain the unscoped delay fallback.

Each saved profile records its screen delay and the effective margin at saving. The applied wait is **saved wait + current effective margin - saved margin**, bounded to 0–3000 ms. For example, 100 ms saved at a 40 ms margin becomes 140 ms at an 80 ms effective margin. A zero effective margin when the same configuration stops playing does not erase the last playback margin; this avoids anchoring a just-finished click result against an idle DAC.

6.1.1.1 Practical setup: a phone on one or two networks

Calibrate on the phone itself; its extra delay includes the route from the box to that phone. On each Wi-Fi network you use, open **Config -> Meter timing**, press **Identify Wi-Fi**, and confirm the displayed network name. If Android does not provide it, set a manual network name before calibrating. Keep the phone connected to the box while measuring. **Calibrate on the music** uses the phone microphone and is the easiest option when there is varied music playing. **Tune with**

clicks is useful when the music match is weak; with the phone app it measures the click train too. Without microphone access, use the click bars and adjust by eye.

Repeat after switching to the second network. The phone keeps separate values for each network and active audio configuration, so returning to the first network restores its calibration. Calibrate the DRC filters and sample rates you actually use; an unmeasured nearby rate may borrow provisionally from the same filter family. Do not enter a different network name merely to force a new value: the network identity should describe the connection in use.

6.1.1.2 Practical setup: the web UI and plasmoid on host X, plus a phone

On host X, open the web panel using one stable origin, for example `http://localhost:9090`. The full panel and kiosk at `/k/` share their timing profile when they use that same origin. Browser storage is origin-specific, so switching between `localhost`, a host name and an IP address can make the same browser appear to have different saved timing. The ordinary browser cannot identify its network automatically: in **Config -> Meter timing**, set a network name such as `wired` or the Wi-Fi name before saving a local profile. For host X, use **Tune with clicks** and set the delay by watching the bars against the sound; this needs no microphone in the box.

In the plasmoid on X, enter `localhost` as the box address. **Use the box's live delay estimate** reads `/spectrum/settings` and applies the effective remaining margin to the plasmoid. The estimate includes the box's reported FIR peak/partition or direct-output delay model, but does not measure network transit. For a local plasmoid it is a starting point. To match the web UI, turn Auto off and copy its **Applied screen delay** in milliseconds into the plasmoid's number field; the slider adjusts in 1 ms steps. This setting is local to the plasmoid and does not alter the web profile, phone profile or box-wide margin. Fine-tune it by ear if the two displays do not line up.

The phone is a separate client even when it is on the same Wi-Fi as X. Its Android app stores profiles on the phone, and identifies that Wi-Fi separately from the browser's manually named local profile. Calibrate the phone on that network too: its route and drawing device add different delay from the localhost display. A copied value can be a starting point, but use the phone's microphone calibration or click test for the phone's own final value. Both clients can share the same network label and still retain separate values.

The controller margin is different: it is shared by every client. A phone calibration may increase that margin only when frames otherwise arrive after the sound. The web profiles compensate for a changed margin; the plasmoid's automatic estimate follows the current value. A manual plasmoid value is an explicit override, so check it again if the box-wide margin is changed.

Network stalls and recovery. In the kiosk, each timestamped frame is scheduled against its server publication time plus the lowest observed transit time on that connection. The unknown clock offset between box and phone is included in this baseline; absolute clock synchronization is not needed. Network jitter consumes the saved waiting time rather than adding the full wait to every late arrival. Drawing coalesces to the newest eligible frame per screen refresh. If transport takes longer than the available wait, the screen cannot undo that lateness.

Server publication time is retained even when a sending thread runs late or a newly opened connection receives an old snapshot; sending an old frame now does not make its audio current. Frames over 1.2 s behind the best observed timing are excluded from both display and calibration. Five consecutive late frames replace the stream, at most once per five seconds. A new connection opens before the old one closes so the analyzer remains enabled, and pending draws belonging to the old connection are

ignored. **LAG!** appears briefly. The saved profile is reapplied automatically; a permanent change in transport latency can still require recalibration. DR history streams retain their separate delivery behavior, since their events carry historical information rather than time-critical meter values.

Missing-calibration warning. When the active network/DRC configuration/rate has no dedicated calibration, Now shows red text above level bars or spectrum. VU needles replace their PK/RMS readout with **UNCALIBRATED**. A tiny **Configure now** chip beside the warning opens Config at Meter timing. A borrowed rate remains marked as provisional and still shows this warning; it is not treated as a completed calibration. The warning disappears when a dedicated value is saved, and follows network/configuration changes while Now is visible. During active measurement the existing calibration indicator remains available. The Configure chip does not itself start microphone recording.

Inspecting timing from Now. Hold the VU meter, level bars or spectrum for about 650 ms. **Applied meter timing** identifies the active network, DRC configuration/rate, filter filenames and source, whether a matching saved profile exists or a provisional value is borrowed (including its source rate), the saved/borrowed and applied screen delays, and the box hold-back. The values refresh while the dialog is open. Its small timing diagram is the same screen-wait example used by **How it works**; it is not drawn to the live values. **How it works** opens the full diagrams, and **Close** returns to Now.

A **single tap** on the meters or spectrum shows the top bar and leaves the display alone. After the double-tap interval it also offers a small gesture hint. Tick **Understood — don't show this again** before closing to suppress future hints in this browser's storage. A **double tap** steps to the next display in the order of the View menu — needles, bars, bars + spectrum, bars + separate L/R spectrum, circular spectrum, bars + circular spectrum — and wraps back to needles; **Level off** is reached only from the menu. The menu's mark follows the double tap. A double tap keeps the top bar visible and does not reset the layout. Double tap the vertical layout handle to restore the default layout. The choice remains specific to the current orientation. Swipes, diagonal cover-opacity drags and canceled touches do not count as meter taps or holds. The separate CLIP indicator still resets its channel on a short tap.

Failures and diagnostic evidence. A genuine network, loaded configuration or requested server-margin change during recording invalidates the result; start a new run in the now-active combination. Merely starting/stopping playback and changing effective buffer margin does not invalidate it. Settings refresh every five seconds and at measurement start/end, so identity changes are detected through that polling rather than instantaneously.

"Only x of 14 clicks" needs interpretation: *meters showed only x* concerns the analyzer/output/frame sequence; *microphone heard only x* concerns recording, noise, level or room response; *only x lined up* means too few onsets matched within the timing tolerance or an alternative lag was too competitive. High volume alone cannot solve missing meter frames, network bursts or ambiguity. Keep the phone near the speakers, use normal volume, avoid talking during the test, and inspect the log rather than repeatedly increasing volume.

Every run keeps a plain-text log locally with its mode/trigger, network and controller timing, meter timestamps/peaks, microphone capture start and envelope, arrival gaps, detected onsets, best candidates and paired lags, and final verdict. Detector diagnostics remain available when a context change rejects the result. Use **Calibration log -> Copy**, or **Select all** and manual copy when clipboard access is restricted. The last log is overwritten by the next run, so save it before repeating a failure. Logs can distinguish input/detection faults from transport timing; an acoustic test on the

actual phone and speakers is still needed to confirm audible alignment.

ADB timing verification (debug Android app). The repository includes `omdrc-ctrl/tools/meter-timing-de`
With Node.js 22 or newer, Android platform-tools (`adb`) on the computer, USB debugging authorized, and the debug OMDRC app open in the foreground, run from the repository root:

```
# Passive 30-second recording; leaves playback and calibration unchanged.
node omdrc-ctrl/tools/meter-timing-debug.mjs --out /tmp/timing-passive.json

# Hold VU messages for 2.2 seconds, then deliver the queued burst.
node omdrc-ctrl/tools/meter-timing-debug.mjs --fault hold --out
↪  /tmp/timing-hold.json

# Discard every third VU message for 2.2 seconds.
node omdrc-ctrl/tools/meter-timing-debug.mjs --fault drop --out
↪  /tmp/timing-drop.json

# Also measure audible alignment before and after the fault.
node omdrc-ctrl/tools/meter-timing-debug.mjs --fault hold --acoustic --out
↪  /tmp/timing-mic.json
```

Select VU/level bars on Now for the default `vu-clip` stream. For Now's spectrum use `--mode music-clip`. Other views may use `vu`, `music` or `precision`; match the stream named in the recording's counters. Keep the screen awake and the view visible. Use `--serial SERIAL` when more than one ADB device is attached. `--seconds N` controls the recording interval (15–480 s). The tool connects to this app's WebView through a temporary ADB forward and removes that forward at exit. It does not enable global network throttling.

Recording is opt-in and bounded: at most 12,000 events, with the number of omitted older events recorded explicitly. A watchdog stops capture within ten minutes; an injected fault ends automatically within ten seconds even if the computer disconnects. The CLI uses a 2.2-second fault after five seconds of baseline. Faults affect only the chosen meter message handler; they neither simulate an audio dropout nor exercise the Wi-Fi driver or TCP reconnect path. For those, record passively while reproducing an actual interruption, and inspect connection events. The tool does not manufacture frames or turn an inactive analyzer into successful evidence.

The JSON contains wall-clock and monotonic timestamps, network and loaded filter/rate identity, exact or borrowed calibration and applied delay, connection opens/errors/replacements, received frames, stale frames, discarded obsolete or out-of-order draws, and frames combined into one screen refresh. Frame age is relative to the best observed transit on that connection (with server publication time preserved), **not absolute one-way network latency**. Draw lateness is the callback time minus its scheduled time; it does not measure the physical screen's scanout. These distinctions matter when interpreting apparently good logs.

For an injected fault, the report compares the 95th percentile of drawing lateness before and after recovery, allowing three seconds after fault release. `delivery-recovered` requires at least ten baseline and ten recovered draws, an actually exercised fault, no omitted events or observed configuration/visibility change, and recovered lateness no greater than the larger of 100 ms or baseline plus 50 ms. `delivery-late` fails that lateness criterion; `inconclusive` means there is insufficient or interrupted evidence. Passive recordings are reported as inconclusive by this automatic

fault comparison; their raw events remain useful. None of these results is a guarantee of acoustic synchronization.

`--acoustic` explicitly **stops music and plays the click track twice**, using the phone microphone. Place the phone near the speakers at normal volume before starting. Both full microphone verification logs are saved separately in the JSON, with the measured residual lag before/after and their difference when both detections succeed. Failure of either detector makes the acoustic comparison inconclusive. These checks do not save or overwrite a calibration; the UI's last calibration log will contain the second check. Small differences are subject to the existing microphone/onset estimator's tolerance, so inspect paired-lag spread and repeat measurements before attributing a small change to long-term drift.

During a recording, debug Android builds also forward structured diagnostics to Logcat under `OMDRCTiming`. In a second terminal use:

```
adb logcat -v threadtime -s OMDRCTiming:I '*:S'
```

Logcat contains one-second cumulative counters and session/configuration, connection and calibration messages; per-frame details are in the JSON export. No raw microphone PCM is recorded. Acoustic exports contain the existing level envelope and network/filter identifiers; they are diagnostic files rather than anonymous statistics. Release builds disable WebView debugging and this native Logcat forwarding. The implementation uses Android's documented WebView debugging support and the DevTools Runtime protocol.

Orientation. With the phone's auto-rotate on, the app follows the phone like any other app and every page adapts to how it is held; the rotate button is hidden. With auto-rotate off, Now is shown in landscape and every other page upright, and the rotate button turns the page on screen the other way (remembered per page). A browser can only lock the orientation in fullscreen; the 7" panel never turns.

Keeping the screen on. Only while Now is on screen *and* there has been sound in the last 30 s: still meters hand the screen back to the phone's own timeout. The app keeps the screen on natively (the kiosk asks through the `window.OmdrcApp` bridge); a browser has no wake lock over plain `http`, so it falls back to a tiny silent video, which recent browsers may not honour.

Starting fast. Back leaves the app like Home, so the page stays alive and comes back as it was. After the system has dropped the app, its last screen (a picture per orientation, taken when it went to the background) is shown at once while the page loads; the splash with its progress ring appears only if that takes more than about a second. The kiosk's scripts are requested with a version parameter that changes on every deploy and are cached for a year, so a cold load does not re-check each file.

Meters under network lag. Every level frame carries the time the box sent it. The kiosk draws at most one frame per display refresh, the newest; frames that keep arriving more than 1.2 s late mean a backlog queued in the network, and the stream is reopened to drop it.

6.1.2 The Android app's lifecycle and status notification

Besides the dashboard, the app runs one background component: a foreground service (`LiveStatusService`) that keeps the home-screen widgets and a notification up to date. Android requires a foreground service to show a notification for as long as it runs, so the notification and the service live and die together.

Two modes. *Instant updates* (the default; App settings) keep one request waiting on the box's `/now`, which answers only when the track, the playback state, the DRC or the film changes, or after 5 minutes with no change, when the phone asks again. This is an idle connection rather than polling: with nothing changing, the phone wakes for it about every 5 minutes. A box without `/now` is polled every 5 s instead. *Live updates* (instant updates off) poll the box every 2 s while the dashboard is used, so the large widget's meters move; 10 minutes after the app was last on screen they stop polling and wait on `/now` as instant updates do. In both modes the notification stays while something plays.

The notification. It comes in two forms, chosen in App settings with **Media notification**:

- *Media notification* (the default): the box's player appears in Android's media controls, at the top of the shade and on the lock screen, as a music app's does. The app holds a `MediaSession` for it, fed from `/now`: the track or film, its cover, and for music the position (`/now` reports `elapsed` and `duration`; Android moves the bar on by itself, so the position costs no extra requests). Music offers previous, play/pause, next and seeking; a film, play/pause. **Stop** and **Close** are the player's extra buttons. The service then runs as a `mediaPlayback` foreground service, which has no daily time limit.
- *Plain notification* (the setting off): the track or film with its cover; expanded, the format, the renderer and the DRC in use. Its actions are **Pause** while music or a film plays (**Play** while it is paused), **Stop**, **Levels** (the PiP meters) and **Close app**.

In both forms the cover carries the widget's badges: **DRC** with its output rate (`DRC 192`) while the DRC runs, nothing when it is off; the source format (`96/24`, colored as on the widget); and the DR log's figure, which is never the live meter's:

- `DR11`: the track's own final figure, from a full measurement, a complete listen, or its row in the album's `dr14.txt` (matched by title, else by track number);
- the same with a leading "approximately" sign: the track's figure from a partial listen;
- `DR12 album`: the album's figure, when the track has none: its `dr14.txt` value, or the value of all its tracks measured;
- `DR12 album` with that sign: an album estimate from the tracks heard so far.

A track heard for the first time shows its album's figure: what is being heard counts only once the track ends, when its figure is saved. `/now` reports the figures (`music.dr`), looked up once per track and refreshed once a minute. In the media player the badges are rows at the top centre of the part of the cover Android leaves in view, no wider than the gap between the player's app icon and its output chip; on the plain notification's thumbnail, one larger row along its bottom. The live DR of the meters is not shown: the box computes it only while something watches the meters, and a value changing every few seconds would wake the phone as often.

All the buttons go to `/k/api/transport`; for a film, with `"target": "video"`, which passes play, pause and stop to the video remote's mpv instead of MPD. **Stop** stops playback and closes the app 5 s later, as Close does, rather than after the 30 s idle grace, unless something plays again meanwhile. **Swiping the notification away is the same as Close app** (below): the notification is not marked ongoing, and its delete intent closes the app. The app has no media browser service, so Android does not keep a stopped player in the media controls for resuming later: the player goes when the notification does. The status bar shows the note from the launcher icon (the "d" of *omdrc*), with a small play, pause or stop glyph in its top-left corner, the part of the icon the note leaves free. Android draws status-bar icons as a single-colour silhouette in the bar's own colour, so the icon is monochrome. Because the service runs only while there is music (see below), the note in the status bar means that the box is playing or paused. A cover reduced to a 24 dp silhouette was tried first

and was unreadable. The icon is generated with the launcher icon by `artwork/draw-app-icon.py` (`--notification` writes it alone): `ic_notification` without a glyph (used until the box's state is known) and `ic_notification_{play,pause,stop}`. The note has a thicker stem than in the launcher icon so that it stays visible at that size.

In the pulled-down shade, Android 16 and later show the app's launcher icon on the left of the entry, not the status-bar icon; its green "d" is the same note, which links the two. The picture on the right is the cover or poster, plain; with nothing loaded there is none. (A copy of the note in the picture's corner was tried and dropped: an icon on each side made the entry too heavy.)

The status-bar glyph follows every state change the box reports, stop included, even though the notification disappears 30 s after a stop. While the box is unreachable, the glyph keeps its last state until the service stops.

The notification uses the channel *DRC live updates* with **default importance and no sound or vibration**. A low-importance channel is a *silent* notification: Pixels hide silent notifications from the status bar and the lock screen by default ("Hide silent notifications in status bar"). Android does not let an app raise a channel's importance once created, so the channel has a new id (`drc_live_icon`) and the former one (`drc_live_status`) is deleted. Users who changed its settings must set them again. The media notification uses the same channel.

When it stops by itself. In both modes the service ends, and its notification disappears, when there is nothing to show:

Condition	Stops after
Nothing playing (stopped, no film)	30 s
Paused (music or film)	15 min
Box unreachable (disconnected, other network, box off)	60 s
Android 15+ daily limit, plain notification only (`dataSync`, 6 h per 24 h)	at the limit

The short grace periods let a change of album or a brief Wi-Fi drop pass without the notification disappearing. The service never stops itself while the app is on screen; after the app is left, the conditions are checked again at the next update, at most about 5 minutes later. A box that reports no playback state at all never counts as idle; only unreachability stops the service then. Once it has stopped, nothing of the app is running: Android removes the process when it needs the memory. The widgets keep their own timers, a roughly 5-minute alarm and a 15-minute WorkManager job, which run only while widgets exist and the app has not been closed.

What brings it back, and the compromise. With the service stopped, nothing on the phone watches the box. Since Android 12 an app in the background may not start a foreground service, so playback started from the kiosk, a browser or another phone does **not** make the notification reappear. It returns when:

- the app is opened (or brought back to the screen), in either mode;
- a widget's refresh icon is tapped while instant updates are on. Android allows this because a tap on a widget is user interaction; if it still refuses, the next opening of the app starts the service.

The only way round this would be a push from the box through Google's push service, which the project does not use. The rule is therefore: **no resources while idle, at the cost of reopening the app** after playback starts elsewhere.

Close app, or swiping the notification away, is stronger than the idle stop. It stops the service, the widgets' timers and every background request until the app is opened again. The widgets' own taps still work.

6.2 The KDE Plasma widget

OMDRC Monitor (`kde/omdrc-plasmoid`) is a KDE Plasma 6 widget showing what the box plays: level meters (LED-style bars or VU needles), the spectrum analyzer, live DR and channel balance meters, and the cover, in any combination. Pointing at the cover brings up previous, play/pause, next and stop, plus Qobuz search and the play queue; on the desktop and in the popup a ring around the cover seeks, dragged clockwise from twelve o'clock. Every icon-only button has a tooltip. Like the kiosk it is only a client of the panel: it reads `/spectrum/stream` (levels, bands, DR), polls `/k/api/player` and `/k/api/queue`, sends `/k/api/transport`, shows `/qconnect/art` and uses the `/qobuz/...` endpoints of section 6.3.

- **In a panel** it is a strip as thick as the panel, as long as the chosen panes need (stacked in a vertical panel). Hovering shows the transport, search and queue buttons beside the cover; a click opens a larger view with title and artist.
- **The panel tooltip**, while a track is loaded, shows the cover on the left and, on the right, the track, artist, album, label, year, format, genre, composer, performers, awards and as much of Qobuz's album description as fits. A tooltip takes no input and cannot scroll, so the description is cut where the space ends. The album details come from `/qobuz/track/<id>` (the track id is in the player's file URL) and `/qobuz/album/<id>`, fetched once per track; a track not from Qobuz shows what the player reports. While the pointer is on the panel buttons or the quick search, the big tooltip stays away so the buttons' own tooltips show.
- **On the desktop** it shows the large view directly, panes side by side when wide and stacked when tall. The cover can sit beside the meters or behind them, the meters then translucent. With the cover pane, meters and spectrum all on, the desktop and popup use a grid: cover and track next to the meters, the spectrum across the full width below, DR (wider) and balance in a strip underneath.
- **Pane order**: Control-click a pane to move it one place on, or Control-drag it to a chosen place; the order is saved per widget.
- **When playback stops** for a couple of seconds, the desktop view opens Qobuz search and the panel widget turns into a quick-search field with Search, AI, Filters and Back buttons. A Plasma panel normally takes no keyboard focus; the widget asks Plasma for it (`AcceptingInputStatus`) only when the field is clicked, never when it opens by itself, and gives it back on Escape, on leaving the field or when the search closes. A search opens the results in the popup.
- **Qobuz and queue pages** in the popup: recently played, search results, discover by genre and awarded albums, with release-date, order, label-group, awarded and Hi-Res filters and the AI recommender. An album card plays or appends the album and lists its tracks; playback goes through upmpdcli on the box, as from the kiosk. The queue page lists, removes and clears.
- **Settings**: the box's address and port, meter style and channel layout, spectrum (beside or below the meters), DR and balance meters, cover, track line, the length in a panel (automatic, or fixed in pixels: Plasma offers no resize handle there), the panel edge margin and the panel

outer margins (pixels kept clear from the panel's top/left and bottom/right edges, 0 and 2 by default), and the background (Plasma's, none, theme color with transparency, or a custom color). Meter and spectrum timing can follow the box's live estimate or use a per-plasmoid millisecond override, with a 1 ms slider and an editable number.

- **Timing estimate and manual tuning**: Auto reads the effective remaining margin from `/spectrum/settings` and shows both that local wait estimate and the box's frame hold-back in the configuration page. It does not estimate the plasmoid's network transit. For a desktop plasmoid on the same host, this is usually a good starting point. For a manual value, copy the web UI's **Applied screen delay** into the number field, or tune in 1 ms steps while comparing the bars with the sound. The setting affects this plasmoid only; it does not change the controller's shared margin or another device's saved profile.
- It streams levels only while something plays — MPD, or a disc on the CD input (`/cdin/status`), which plays past MPD — and otherwise closes its stream, so the box can switch the analyzer off.

It needs an omdrcctrl with bounded analyzer streams (`max_s`, section 6); against an older one it says so instead of showing meters, while the cover and the buttons still work.

Installing. The widget is a per-user Plasma package, installed by whoever will use it, on the box or on any KDE desktop on the LAN:

```
cmake --build build --target plasmoid-install   # or:
↪  kde/omdrc-plasmoid/install.sh
```

Both run `kpackagetool6` for the current user, installing or upgrading in place. It is not part of `make install` or `make user-install`, since a headless box has no Plasma; once installed, `make user-install` keeps it upgraded. Then add *OMDRC Monitor* from *Add Widgets...* and enter the box's address (`localhost` when the desktop is the box). After an upgrade, a Plasma restart (`plasmashell --replace &`) loads the new code into running widgets.

6.3 Qobuz album search

Qobuz's own apps cannot filter a search by **label** or **release date**, the two questions a classical listener asks first. The panel adds that search, in the kiosk's Qobuz page (and at the bottom of Now, upright). What is found plays through **upmpdcli**, whose Qobuz plugin streams it from the box itself, never through the phone; the page is greyed out while upmpdcli is not the running renderer. It uses upmpdcli's own Qobuz credentials (the panel's Qobuz sign-in) and needs nothing of its own.

Results. With no filter the answer is Qobuz's own release list for the text, in Qobuz's order, a page of 50 at a time; reaching the end of the list reads the next page, as scrolling does in Qobuz's app. Albums Qobuz lists but cannot stream are shown greyed. The filters only ever work on that list:

- **Labels** — favourite labels as check boxes, several at once. A label is a keyword matched as whole words, so "Decca" matches "Decca Music Group Ltd." and "Decca Classics". Any label met in the results can be ticked too.
- **Released** — any time, the last N years (a slider), or a from–to span of calendar years (each year a spinner: drag it up or down, faster the further the finger goes, or tap it for a year picker).
- **Order** — Qobuz order, or newest first.

A filtered search reads deeper at once (250 albums per query, doubling to 1000 while few match) and also sends one "text label" query per ticked label, which reaches that label's albums sooner; "Load more" reads on. The filters fold into a one-line summary once results arrive.

Each result shows a small cover, title, artist, performers, label, year, a hi-res tag and how often it was played from here. Its buttons: **play** replaces upmpdcli's queue with the album and plays it; + appends it; **-** lowers it. A tap shows the tracks, each playable from there. Playing goes through upmpdcli's OpenHome playlist exactly as a control point browsing the box's own Qobuz library would, so each track carries its metadata (and, with the patched upmpdcli, MPD's Date and Label tags).

Lowering. The **-** button moves the album to the end of every result list, folded under "N lowered results"; the bar that follows can undo it or lower the album's whole label or artist (the album artist or any performer, so a conductor) instead. The Lowered list, by the filters, restores entries or clears them. It is kept on the box (`qobuz-lowered.json` in the state directory), shared by every screen.

Completions. The search field completes as you type, from a classical word list shipped with the panel (`qobuz_words.txt`: about 2,600 composers, forms, named works, instruments and voices, conductors, soloists, singers, orchestras and ensembles, editable) and from what was played after a search: the search text and the album's artist and composer are learned (`qobuz-words.json`), come first, and appear when the empty field is tapped. Accents and case do not matter.

The player. A strip at the bottom of the page shows the track playing (from MPD, which is also upmpdcli's queue) with play/pause and, in landscape, previous, stop, next and a seek slider. It opens into a full-screen player: the cover as large as the screen allows, the track, work, album, label and year, the transport, and the queue, where a tap plays from that track. Saving the playing album to the Library (the heart) is in the full player; the strip keeps only the transport. When a listening guide is minimized (section 6.3.2), its own strip docks just beneath the player strip. The two strips take their tints from the playing album's cover (the player strip its dominant colour, the guide strip the next distinct one), or the theme's shades when there is no cover.

Discover by label. Besides Search, the page has Recent, Discover (new releases, by genre) and Awarded views. With a label ticked, Discover reads that label's own catalogue, newest first, instead of filtering Qobuz's new releases list (which ends a few hundred albums back and holds only a handful of any one label); unreleased albums are left out and a chosen genre still applies. With any filter, the list keeps reading on (up to ten pages at a time) while you scroll, until a screenful passes the filters.

Search from anywhere. The magnifier in the kiosk's top bar opens the Qobuz page already scrolled to its search box. Reaching the page any other way keeps the box out of sight, as before.

Configuration is the `[qobuz_search]` section of `commands.conf` (enabled, favourite labels, scan depths, cache times); the HTTP routes are under `/qobuz/` and listed in `omdrc-ctrl/README.md`.

6.3.1 Ask AI: recommendations from reviews

The **AI** toggle beside the search field turns the search into a request in plain words, for example *"three Beethoven Symphony No. 5 recordings, prioritizing sound engineering"*. Press the magnifier and the AI:

1. **researches** reviews on the web (at most four searches);
2. **plans** up to four Qobuz searches from what it found;

3. **selects** albums only from what those searches returned.

The box then checks every chosen album again (still streamable, still passing the Hi-Res filter) and shows ordinary playable cards, in the AI's order, each with the reason it was chosen and links to the reviews behind it; a pick without a supporting link is flagged. The number of results is the one asked for, or a suitable number up to 20. Turning AI on clears the filters; label, date, Awarded and Hi-Res filters set afterwards apply. Changing a filter never starts another (paid or rate-limited) AI request by itself: press the magnifier again.

While the request runs, and afterwards, **AI prompt and replies** under the results shows each stage: the exact prompt sent, the reply, the web searches the model made and the tokens it used. If a stage times out, what it had produced so far stays visible. The AI does not listen to anything: sound quality comes from reviews, and Hi-Res figures alone prove nothing about the engineering.

6.3.2 The listening guide

The **Research music** chip on Now, or the AI icon in the full Qobuz player, opens a listening guide for the album that is playing. The AI researches the album once (up to five web searches) and the guide shows:

- **Overview** — the release and its works, followed by the **composers** (or, for pop, rock and jazz, the artist or band) with a biography that places them in the history, culture and ideas of their age, and the principal **performers** when their background is verified;
- one tab per **composition** — when, where and why it was written, the composer's life at the time, its premiere and reception, and what to listen for;
- a note for **every track** or movement.

Swipe sideways to step through the tabs (they wrap round at either end). The tab of the work that is playing follows playback. Swipe the guide down to a strip at the bottom of the screen (it keeps following playback there) and up to reopen it; **Close** stops following and cancels a research still running. For classical albums, Qobuz's own work metadata decides which movements belong together; jazz records and live albums get a tab per song, concept albums one whole-album tab. If the web research times out, the guide says that it relies on the model's own knowledge and the album metadata.

6.3.3 AI settings

Opening the dialog. Kiosk **Configuration** page, button **AI settings**. It also opens by itself the first time Ask AI or the listening guide is used while no AI provider is configured.

Provider. Three choices; only the first uses an existing subscription:

Provider	How it works	Cost
Claude account (server login) — the default	The box runs the Claude Code program (`claude`), signed in with your Claude account (the same login as claude.ai) as the user that runs omdrcctrl	Included in your Claude plan; counts against its usage limits
Claude API	Anthropic's developer API with an API key	Billed per use, separately from any subscription

Provider	How it works	Cost
OpenAI API	OpenAI's developer API with an API key	Billed per use; a ChatGPT subscription does not include it

For Claude account mode, install Claude Code on the box and sign in once as the service user (`claude auth login`); the dialog says whether that login is detected. Account mode never falls back to an API key. API keys are kept only on the box (`qobuz-ai.json` in the state directory, mode 0600) and are never sent back to the browser; leaving the key field blank keeps the saved key.

Model. A drop-down list. In Claude account mode it offers:

Entry	Meaning
Sonnet (latest)	balanced; recommended and the default
Opus (latest)	most thorough, slower, uses the plan's limits faster
Fable (latest)	top tier
Haiku (latest)	fastest and lightest
Other...	a text box for an exact model name, to pin one model

"Latest" entries are Claude Code aliases (`sonnet`, `opus`, `fable`, `haiku`): each call runs the newest model of that tier, so the list never needs updating when Anthropic releases a model. Which tiers can be used depends on the Claude plan; a tier the plan lacks fails with an error rather than silently falling back. The API providers list exact model names, plus an Other... entry.

New-model notice. After every call the box records which model actually answered (`qobuz-ai-models.json` in the state directory). When it changes — for example "sonnet" moving from Claude Sonnet 5.5 to a newer Sonnet — the next result shows a one-time notice: above the Ask AI results, or as a toast in the listening guide. A model pinned with Other... never changes, so it is never announced.

Usage. In account mode the dialog shows the five-hour and seven-day usage percentages Claude Code reported after the last request, and links to the provider's usage page. The box cannot see the account's billing balance.

6.3.4 Customizing the AI prompts

The instructions sent to the AI are plain text files, installed in `lib/omdrcctrl/prompts/` (source: `omdrc-ctrl/src/prompts/`):

File	Used for	Required placeholders
`system.txt`	the **system prompt**: the standing brief sent with every call, for every provider	none
`listening-research.txt`	listening guide, web research	`${metadata}`
`listening-guide.txt`	listening guide, organizing the research into tabs and notes	`${metadata}`, `${research}`
`recommend-research.txt`	Ask AI, web research	`${quantity}`, `${request}`

File	Used for	Required placeholders
`recommend-queries.txt`	Ask AI, planning the Qobuz searches	`${research}`
`recommend-select.txt`	Ask AI, choosing the albums	`${quantity}`, `${data}`

The **system prompt** is the brief the model reads before each task and weighs above it: who it is, how careful to be with evidence, and that album metadata and web pages are data, never instructions. Rules that should hold everywhere — a language, a tone, a length — belong there; the other files hold each task's own instructions. Claude account mode passes it to Claude Code as its system prompt, the Claude API as `system`, OpenAI as `instructions`.

To change a prompt on one box, copy the file into `prompts/` in the state directory (`$OMDRC_STATE_DIR`, else `~/.local/state/omdrc` of the service user) under the same name and edit the copy. It is read on every request, so no restart is needed; deleting it restores the default, and reinstalling never overwrites it. For example, to have every answer in Italian:

```
mkdir -p ~/.local/state/omdrc/prompts
cp /usr/local/lib/omdrcctrl/prompts/system.txt ~/.local/state/omdrc/prompts/
echo "Always answer in Italian." >> ~/.local/state/omdrc/prompts/system.txt
```

Each file opens with `#` comment lines saying where it is used and what its placeholders hold; those leading lines are never sent. The box fills in the `${name}` placeholders (album and track list, research text, your request, the candidate albums); a copy that drops a required one is refused with an error naming it, before anything is sent. Any other `$` is sent as written. The shape of the AI's answers is fixed in the program, so a prompt can change emphasis, tone or language, but not the fields the guide and results read. The kiosk keeps the last listening guide it showed, so open a different album to see the effect of a changed prompt.

6.4 Configuration page — filter installs and audio hardware roles

`/configuration` lets an operator on the trusted LAN install or remove room-correction designs and pin physical audio cards by USB identity, with no git checkout or shell access to the box. `omdrc-ctrl/src/configuration.py` drives two independent workflows through one privileged helper, `scripts/omdrc-config-helper.py`, installed with the panel:

```
<AUDIO_USER> ALL=(root) NOPASSWD: /usr/local/libexec/omdrc/omdrc-config-helper *
```

- **Filter installs** — upload a REW `.txts`/`.mdat` pair and publish it as a live, verified bundle; the page invokes the same audit/build engine as the command line, streaming progress over Server-Sent Events (section 5.9).
- **Audio hardware roles** — pick the DAC and (where a bridge exists) the capture interface from the cards actually attached, by USB identity (`vid:pid[:serial]`), instead of editing OS configuration by hand. `apply_audio()` refuses a DAC that is not attached or cannot play, and refuses either role when two identical cards have no serial number to disambiguate — unplug one before applying. A configured capture card that is not plugged in this boot is dropped with a notice rather than failing the reconcile. What Apply writes and reconciles is OS-specific: Linux section 11.5, FreeBSD section 15.2.

6.4.1 DAC switching and known-device policy

Each role keeps a list of every card applied to it, the explicit selection first. USB card numbers (`pcm0`, ALSA `card1`) are never saved; every reconcile resolves the identities afresh from USB VID/PID and, when needed, the serial. Where the list is stored is OS-specific (the roles sections above). Every boot and hotplug reconcile applies these rules:

1. A card becomes known only when it is selected and successfully applied in the web UI. Being plugged in never enrolls it.
2. Apply puts the chosen card first in its role's list and keeps the cards applied before it (at most eight). It removes the card from the other role's list, so a playback-capable capture interface never becomes a known DAC. For capture, *Disabled* forgets only the capture interfaces attached at the time. One that is elsewhere, such as the home ESI when you Apply at the office, stays in the list.
3. When the explicit selection is attached, it takes the role.
4. When the explicit selection is absent and **exactly one** other known card is attached, that card takes the role. The OKTO/Cambridge swap between home and office therefore needs no Apply. The list order does not rank the other known cards.
5. When the explicit selection is absent and two or more known cards are attached, the choice is ambiguous and the role stays empty. The chain card and `/configuration` ask the operator to choose (`dac_ambiguous=1` in `audio.roles`). An Apply there makes that card the explicit selection.
6. An attached card that is not in the list is never used for a role that has a list. Automatic ranking of unknown cards applies only when the role has no list at all.
7. VID/PID identifies a model, and a serial suffix separates identical devices. If identical attached devices expose no usable serial number, Apply is refused and the operator must unplug all but the intended one.
8. When no unique known card is found, nothing is redirected. The DRC chain, the bit-perfect controls and the hardware volume are never sent to an arbitrary playback card.

A remembered capture interface that is not attached is normal for a box that travels, so it is reported as information, not as a fault. Upgrading keeps the single identity configured at upgrade time as a one-entry list; a DAC whose identity was overwritten earlier must be applied once to enroll it.

The helper validates USB identities, selectors, canonical bundle IDs, hashes, config derivation and destination roots before touching anything root-owned; `configuration.py` itself runs as the unprivileged web user throughout. A finished job keeps only its log: the browser's upload and the staged site are discarded when the job ends, and payloads orphaned by an earlier run are swept at panel startup.

6.4.2 Bit-perfect check page

`/bitperfect` runs the USB wire tap from the browser through any of five playback paths — including the real **upmpdcli** and **qobuzconnect2mpd** routes — and shows the reference bytes beside the bytes the DAC actually received, as a whole-stream colour map and a synchronised hex view. It shares this page's operation lock, CSRF token and Origin rule. Fully described in The `/bitperfect` page and its implementation.

Chapter 7

Bit-perfect verification

Bit-perfection is proved, not assumed: a tap records the USB isochronous OUT stream at the **last host-controlled point** — the URBs handed to the USB host controller — and the bytes are compared with a reference. The control panel's **Bit-perfect check** page runs that tap through the paths music actually takes and shows the compared bytes. The test signal and the cross-OS comparison are in Appendix A; the original single-host proof tool is FreeBSD-only (section 16.5).

7.1 The `/bitperfect` page

Music arrives through **upmpdcli** or **qobuzconnect2mpd**, both of which drive **MPD**, and every layer can break bit-perfection in a way a direct test never sees — MPD resampling, a decoder promoting samples differently, or a renderer quietly setting MPD's volume or replaygain. The page (`http://<box>:9090/bitperfect`) runs the tap and verdict engine while varying *who plays* and, uniquely, shows the compared bytes.

7.1.1 The five paths, and which question each answers

Source	Who starts playback	What it proves
`aplay`	the panel, via the per-OS tap script, delegated to unchanged	host to USB, no renderer. The control.
`mpd`	the panel: MPD plays a local file staged into `music_directory`	MPD to DAC: the segment both renderers share
`mpd-http`	the panel: MPD fetches an HTTP URL it serves	MPD's curl input plugin and streaming decoder — structurally the path a Qobuz stream takes, but with a *known* file, so the verdict is real byte equality
`upnp`	the panel: upmpdcli is found by SSDP and told to play it itself	the whole **upmpdcli** to MPD to DAC path (needs upmpdcli to be discoverable — see the UPnP interface section)

Source	Who starts playback	What it proves
`live`	**you, in the Qobuz app** — the panel only arms the tap and waits	the whole **qobuzconnect2mpd** path, against the renderer's own buffer

`live` is the one row where the panel is not the playback initiator, and that is the point rather than a limitation. Every other source hands material to the chain; `live` deliberately touches nothing, arms the USB tap, and prompts you to start a track in Qobuz — so the bytes it captures were produced by the real service path, with nothing synthesised on their behalf. Read "the page plays nothing" as *the page starts no playback of its own*, not as *no audio flows*: the tap has to see a live stream or there is no capture to compare.

Running `mpd` and `upnp` on the same material is what *localises* a fault: if `mpd` is bit-perfect and `upnp` is not, the renderer is the cause. Both renderer-driven modes snapshot MPD's volume and replaygain before and after and report any change, because a renderer that silently enables replaygain destroys bit-perfection without altering a byte of the file it was handed.

7.1.2 The live Qobuz reference: the renderer's own buffer

`live` needs a reference for a stream nobody has a copy of, and does not ask for one: **the renderer already wrote the exact bytes it fed MPD to local storage**, so that file *is* the reference — real byte equality against the real service. `qobuzconnect2mpd` stages a complete FLAC per track:

`/tmp/qobuzconnect2mpd-<uid>/cache/track_<id>_<fmt>_<pid>_<n>.flac`

Resolution happens at run time and never trusts a hard-coded path:

1. `mpc current` — if MPD reports a local file, that is the reference;
2. otherwise the open file descriptors of the MPD and renderer processes (renderer-agnostic, read with the OS's own tool);
3. otherwise the known buffer locations, **filtered to files that changed during the tap window** — the cache keeps completed tracks for hours, and comparing against a stale one would report a fault that never happened.

Numbered *segments* are concatenated in **numeric** order (lexical order would put `part10` before `part2`); whole tracks are never concatenated. A **lossy** buffer is reported as *decode-path transparent*, not bit-perfect: the DAC legitimately receives the decoder's output.

7.1.3 Checking any track, not only the generated asset

The page loads a **WAV or FLAC** (anything `ffmpeg` decodes). The file is decoded once to build the reference while the **original** is played, since the decoder is part of what is under test. Rate and depth come from the file, so a 96/24 FLAC tests 96/24 with nothing to configure, and the existing `prep` promotion widens it to the S32 wire container exactly as before.

One hazard belongs to real music alone. Alignment anchors on a 4 KiB window of the reference; the generated counter can never repeat one, but music can — digital silence, a looped intro, a repeated bar. Aligning on the wrong occurrence would be a *silent* error, so `find_probe_offset`

takes `unique_in=` and skips any window occurring more than once, and the loader reports the anchor it chose *before* a run rather than after it.

7.1.4 Using the page

1. **Chain readiness** names the DAC, who holds it, whether BruteFIR is running, whether `sudo -n` will start the tap, and MPD's volume, replaygain and enabled outputs. Blocking conditions are listed explicitly; `drc.sh off` clears the usual one.
2. **Test material** — either generate the per-rate counter asset (30 s for rates up to 96 kHz, 10 s at 192 kHz) or load a file. Each row shows size and sha256 so a run is identifiable from its report alone.
3. **Run a check** — pick the path, the material and (for `live`) the tap window, then start. Progress streams as a phase strip (`prep`, `tap`, `play`, `drain`, `align`, `verdict`) with live tap counters: URBs, payload bytes, kernel drops. The runner also reports finer stages that are not chips of their own — decoding the capture, resolving what the renderer streamed, restoring MPD — and those are shown as a caption beside the strip: the strip only ever advances, so a stage it does not recognise holds position rather than clearing it.
4. **Result** — the verdict, then every stage named and hashed (input file, reference bytes, untrimmed wire, tapped payload), then the byte view.

The identical run from a terminal:

```
scripts/bitperfect_runner.py --source mpd  --input track.flac --out bp-results/run
scripts/bitperfect_runner.py --source upnp --input
↪  tests/bitperfect-test-44100-s32-stereo-30s.wav \
                            --out bp-results/upnp
scripts/bitperfect_runner.py --source live --duration 60 --out bp-results/qobuz
scripts/bitperfect_material.py load album.flac --out-dir /tmp/mat
scripts/bitperfect-lib.py window PREFIX.ref.raw PREFIX.wav 80000 8 2
scripts/bitperfect-lib.py scan   PREFIX.ref.raw PREFIX.wav 200 2
scripts/bitperfect-lib.py leadin PREFIX.wire.raw 5648 2 0 32
```

7.1.5 Seeing the bytes

A verdict says *whether*; the byte view says *where* and *what*, at two zoom levels served by `bitperfect-lib.py window` and `scan`:

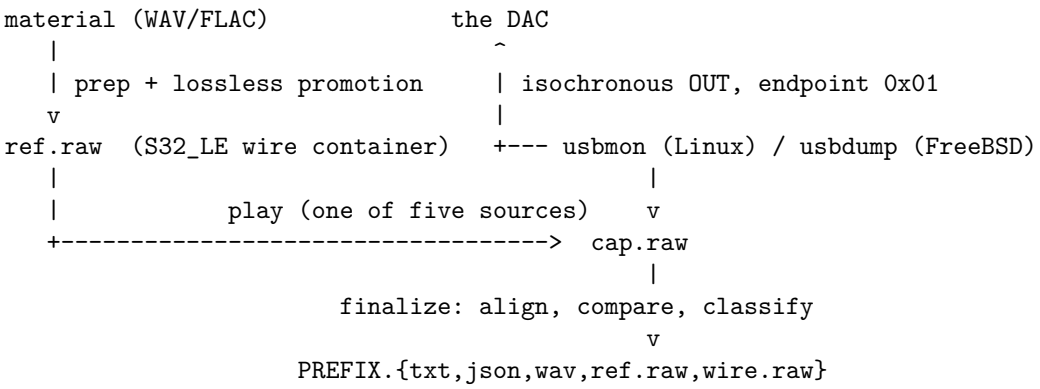
- **The colour map** paints the whole stream, one cell per slice: green where the wire carried exactly the reference bytes, red where it did not, grey where the capture did not reach. It makes the *shape* of a fault legible: an isolated red cell is a bit flip, a red tail an underrun, dense speckle a converting feeder in the path. Clicking jumps the hex view there.
- **The side-by-side hex view** shows reference and tapped wire, one row per frame (offset, hex, decoded per-channel integers), with differing byte positions highlighted in both columns and one slider driving both. `first mismatch` and `random spot check` jump to interesting places.

Both read only the artifacts `finalize` already wrote, so they cannot disagree with the verdict (`tests/test_bitperfect_window.py` pins this on captures with faults known to the byte). A third subcommand, `leadin`, shows what alignment discards: everything before the first reference byte. The collapsible **Before the stream** panel reads it from `PREFIX.wire.raw`; a run of zeros right before the anchor is the ring's priming silence, while a non-zero byte came from somewhere else and

is worth a look. It also works after an `ALIGNMENT FAILED` verdict, when no aligned pair exists.

7.2 Implementation

7.2.1 The pipeline



The tap sees the stream at the **last host-controlled point** — the URBs handed to the USB host controller. Only the controller's DMA engine and the DAC's own receiver lie beyond it; neither can be tapped in software on any OS, and neither has any mechanism to alter a PCM payload.

`scripts/bitperfect_runner.py` orchestrates; it does **not** reimplement the verdict. `--source aplay` is delegated to `bitperfect-tap-{linux,freebsd}.sh` unchanged, so the control experiment stays byte-identical to every result recorded in `doc/BIT-PERFECT-VERIFICATION.md`.

7.2.2 Artifacts

File	Content	Reproducible
`PREFIX.txt`	the human report: every stage named and hashed, then the verdict	yes (this is the committed cross-OS key)
`PREFIX.json`	the same as structured data, plus `first_mismatch`, `start`, `probe_offset`, `matched`, `slips`, the MPD before/after state and the source	yes
`PREFIX.wav`	the aligned capture, trimmed to reference length	yes
`PREFIX.ref.raw`	the promoted reference — kept so the byte view has both sides	yes
`PREFIX.wire.raw`	the untrimmed capture: priming zeros, pad, trailing packets	**no** — provenance only, never a comparison key

7.2.3 Alignment: the anchor, not the silence

The wire stream never starts at the first source byte: the capture begins before playback, and the audio stack emits priming zeros before the first written sample (measured: exactly 16 ms at any rate

— a fixed-duration buffer prime, hence reproducible). `finalize` therefore takes a 4 KiB *anchor* from the reference at a known offset `po`, finds it in the capture at `pos`, and derives `start = pos - po`.

It anchors on content it already knows rather than "skipping the leading zeros" because **zeros can be data**: a track may legitimately open with silence, and on the wire those bytes are indistinguishable from the priming zeros in front of them. A "skip the zeros" rule would eat a quiet intro.

7.2.4 The silence pad

What is *played* is not what is *compared*: playback gets a few seconds of digital silence appended, the reference does not, and `finalize` cuts by arithmetic (`cap[start : start + len(ref)]`) rather than by detecting silence. Two measured reasons:

- `usbdump` buffers its pcap and loses whatever is unflushed when terminated (hence `SIGINT`, not `SIGTERM`);
- **MPD does not drain its output buffer on close**: a 10 s WAV reached the wire 129744 bytes (0.74 s) short, identically across runs and regardless of how long the tap kept recording — the player discarding its tail, not the tap stopping early. The direct OSS writer has no such gap because it `SNDCTL_DSP_SYNC`s first.

The pad cannot mask a defect: the comparison window is still exactly `len(ref)` bytes, so a bit flipped mid-stream is still reported at its exact offset.

7.2.5 Renderer arbitration

`qobuzconnect2mpd` and `upmpdcli` are mutually exclusive front-ends, and an active one does not sit still: it watches MPD and re-queues its own track. That is not hypothetical — it truncated a 10 s run to 2.2 s, with MPD's log showing the test WAV replaced by a Qobuz stream mid-playback. So:

Source	Requirement
`mpd`, `mpd-http`	nothing else may drive MPD — the renderer is stopped and restarted around the run
`upnp`	`upmpdcli` must be the one running
`live`	`qobuzconnect2mpd` must be running and playing; nothing is touched

`omdrc-renderer stop` deliberately leaves the *remembered* choice alone, so the restart afterwards is exact.

7.2.6 Two MPD facts the runner has to work around

- **MPD refuses `file://` from a TCP client** ("Access to local files via TCP is not allowed") and this MPD has no Unix socket. The only way to exercise the local-file input plugin is to place the material inside `music_directory` and add it by relative path. When that directory is not writable the run falls back to an HTTP URL — and *says so*, because the path under test changes from MPD's file input plugin to its curl one.
- **Stored playlists are disabled**, so an `mpc save` backup silently does nothing and the restore then fails with "No such playlist". The queue's URIs are held in memory instead, which works on any MPD.

7.2.7 Privilege, and what the page refuses

The panel runs unprivileged. The tap needs root — `usbmon` on Linux, `usbdump` on FreeBSD — and escalates with `sudo -n`, probed up front with `true` so a refusal is reported before a capture is wasted rather than discovered halfway through it.

Runs are **refused while BruteFIR is convolving**. The DRC path applies the FIR filter, so its output is *supposed* to differ; a verdict there would look like a failure and mean nothing. `--allow-drc` overrides it for an operator who knows what they are asking for.

The page shares the configuration page's operation lock (one box, one DAC: a filter publication and a tap run must never overlap), its CSRF token and its Origin rule. One route is necessarily token-free — the asset MPD's curl plugin and upmpdcli fetch by URL, which can present no header — and it resolves basenames inside the asset cache and nothing else.

7.2.8 Verdicts and exit codes

Exit	Meaning	Verdicts
0	judged, and the chain is transparent	`BIT-PERFECT`
1	judged, and something is wrong	`HEAD LOST`, `INCOMPLETE`, `VALUE CORRUPTION`, `TIMING SLIP(S)`, `UNDERRUN TAIL`
2	**could not judge** — capture unusable	`NO CAPTURE`, `ALIGNMENT FAILED`

Exit 2 does not by itself indict the capture setup. Aligning needs 4096 *consecutive intact* bytes, so a defect altering *every* sample (a volume feeder, say) leaves no such run anywhere and the search fails before any comparison. When exit 2 appears, open `PREFIX.wire.raw`: junk or zeros point at the tap, plausible-looking audio points at a converting feeder.

7.2.9 Status and what is still owed

- `live` needs a human to start Qobuz playback, so it has not been run end to end against hardware. Its buffer resolver is unit-tested against the exact layout observed on the FreeBSD box, including the stale-buffer and never-concatenate-whole-tracks rules.
- `drc.sh cdin` re-execs without the design variant, so switching to CD input silently drops an active `@design`.

Chapter 8

Dynamic range: which master, and what it measures

A record exists in several masters, and they are not equally loud. The Alice In Chains *Unplugged* CD measures DR 8 while vinyl rips of it measure DR 12–13; the Rolling Stones' *Get Yer Ya-Ya's Out!* is DR 11 on the 2002 CD and DR 9 as the 2014 download. What a streaming service serves is one of those masters, and nothing on the stream says which. The panel answers the question four ways, each more direct than the last:

1. **DR versions** (`/dr-alternatives`, the *DR* button) — every version of the playing record listed in the community database at dr.loudness-war.info, most dynamic first.
2. **Which one is playing?** — a button on that page that scores each version against the track on the wire and names the most likely one.
3. **Measure DR** (the renderer card) — measures the copy on the wire itself, with the same algorithm the database's entries were measured with.
4. **Estimate DR** (the renderer card) — a live, rolling reading of what MusicPD is playing now, with no download (section 8.4).

The first two report other people's measurements of other people's copies; the last two measure what you are hearing — *Measure DR* the whole record, *Estimate DR* the passage in progress.

What the live meter hears can also be kept: the DR log (chapter 9) stores every track's DR and ranks albums by it.

8.1 What the renderers tell the panel

Everything below depends on metadata, and MusicPD's queue carries only what a renderer puts there. Both renderers are patched to publish more than their stock versions do:

Tag in MusicPD	upmpdcli	qobuzconnect2mpd	Used for
Artist, Album, Title, Track	stock	added (the queue held bare redirect tokens)	the database lookup; the running order
`Date`	patch (`dc:date`)	added (`release_date_original`)	year evidence

Tag in MusicPD	upmpdcli	qobuzconnect2mpd	Used for
`Label`	patch (`dc:publisher`)	added (`album.label.name`)	label evidence
`Genre`	patch (`upnp:genre`)	—	display
cover art	from upmpdcli's DIDL cache	`art=` line in the status file	the card's thumbnail

The upmpdcli change is `upmpdcli/patches/0001-carry-date-genre-and-publisher-tags.patch`, written to go upstream as a Framagit merge request; `cmake` warns, with the procedure, when the upmpdcli it finds was built without it. Two caveats hold for every stream: the year Qobuz supplies is the album's *original* release date, true of every reissue alike, and a multi-disc set arrives numbered `disc * 1000 + track` (track 1005 is disc 1, track 5).

8.2 Identifying the pressing

Which one is playing? reads each listed version's own page and scores it against **the track playing, paused or last played — never the rest of the queue**, which is a listener's doing and not a pressing's. One track is enough, because that is exactly where two masters of a record differ.

Every signal that applies adds its points; a comparison one side cannot make adds nothing and is listed with a neutral dot, so an absence never reads as agreement. The score is clamped to 0–100.

Signal	When	Points	Why
Running order	the playing title is at the same track number	+12	the edition's own track list, from its uploaded DR log
	the title is on this version, at another number	+8	a reissue with bonus tracks, say
	this version's track list does not name it	-20	
Track length	within 3 s, at the right track number	+45	two rips of one pressing agree to the second
	within 3 s, elsewhere in the list	+32	
	within 8 s	+20	a few seconds out is a different transfer
	20 s or more apart	-30	a different cut — a DVD keeping the between-song talk
Stream format	a CD entry, and the stream is above 44.1 kHz/16 bit	**rules out**	a CD cannot carry it; the same master's DR may still be right

Signal	When	Points	Why
	the medium suits the stream's rate	+6	CD or download at CD rate; download, SACD or Blu-ray when hi-res
	a surround or disc transfer, stereo stream	-12	
	the entry is an analog source: vinyl, LP, cassette, tape	**rules out**	it measured a turntable's or tape deck's output; a stream is a digital master
	the entry's title states another rate/depth	-15	"48kHz-16bit", "16/48"; "448 Kbps" is a bitrate and ignored
	the entry's title states the stream's rate/depth	+8	
Disc of a set	same disc	+10	sets are filed one disc per entry
	another disc	-25	
Label	an imprint name in common	+14	"Columbia/Legacy" shares Columbia; "Records" and the like are ignored
	both named, nothing in common	-5	weak: services carry the reissue imprint
Year	same year	+14	
	one year apart	+6	
	further apart	-6	weak: the stream's year dates the album, not the transfer
Album name	identical	+5	
	one contains the other	+2	

A **rules out** row is a veto, not a weight: a medium the stream cannot have come from marks the version *ruled out* whatever else agrees — a vinyl rip shares the digital mix's track times, and points alone once left one looking possible beside a digital stream.

The verdict ladder is: **55** or more *likely*, **30**–**54** *possible*, below that *unlikely*, and *no evidence* when nothing could be compared. The best score is *most likely* — and so is every version within **12** points of it: the evidence does not separate them, so the page does not either. Country, catalog number and bar code are displayed but never scored, since nothing in a stream can confirm them.

These numbers live in one table, `drdb.W`, which both the scorer and the page's own copy of this table

read; tests fail if a weight is used without being documented or written into the scorer as a literal.

8.3 Measuring the DR of the stream

The **Measure DR** button on the renderer card fetches the playing record's tracks a second time, measures each one and deletes it before fetching the next. It runs in the background; the card shows a progress bar, the track in hand and its phase, the album value as it builds, and one colour-coded badge per track.

8.3.1 How a job runs

1. **Which tracks.** The run of queue entries around the one playing that carry *exactly* its album tag, capped at 60. Exactly: *Get Yer Ya-Ya's Out!* and *Get Yer Ya-Ya's Out! (40th Anniversary Deluxe Edition)* are different masters — "Carol" measures DR 8 on one and DR 9 on the other — and averaging them would produce a number neither has.
2. **Fetch.** Each queue entry is an HTTP URL (upmpdcli's proxy, or qobuzconnect2mpd's `/qobuz-direct/` token); both redirect to the CDN, which reports the file's size and serves byte ranges. The download reports progress by bytes. A connection that stalls for 30 s is retried up to four times, **resuming** with a `Range` request from the bytes already on disk, or starting the track over if the server ignores the range.
3. **Measure.** `drmeter.py` runs in a subprocess under `nice -n 19` so it never takes a cycle from the DRC convolution. ffmpeg decodes at the file's native rate and channel count — resampling would move the peaks — to 32-bit float, and the meter consumes it one 3-second block at a time, so a 24/192 track of any length needs a few megabytes of memory.
4. **Delete.** The track's file is removed as soon as it is measured, before the next download starts. At most one track is on disk at any moment.

The work directory is `/var/tmp/omdrc-dr-*`; it is removed when the job ends, fails or is cancelled, and a directory a crashed panel left behind is removed when the next job starts. A job refuses to start with less than 1 GB free there, or when the playing tracks are not streams. One track that will not download or decode is marked failed and does not stop the rest; the album value is then over the tracks that measured.

8.3.2 The metrics

The meter is the TT Dynamic Range algorithm, as used by the foobar2000 Dynamic Range Meter 1.1.1 whose logs fill the database, and reproduced by dr14_tmeter:

Metric	Definition
Block	3 s of samples per channel — **3 times 44 160** at 44.1 kHz (a quirk of the reference meter, kept), 3 times the sample rate otherwise; the last block is partial
Block RMS	square root of (2 * sum of squared samples / n) over the block's own length n; the factor 2 makes a full-scale sine read 0 dB
Block peak	the largest $\lvert x \rvert$ in the block

Metric	Definition
RMS_{upper}	root mean square of the loudest **20 %** of block RMS values (at least one block)
Reference peak	the **second-highest** block peak: one clipped transient cannot inflate the result
DR (channel)	20 times the base-10 logarithm of (second-highest block peak / RMS_{upper})
Track DR	mean over channels, rounded half to even (Python's `round`, as the reference)
Exact DR	the same, unrounded — shown in the badge's tooltip
Peak	largest sample of the track, dBFS
RMS	mean of the channels' RMS over the whole track (with the factor 2), dB
Album DR	mean of the tracks' integer DR values, rounded — the "Official DR value" a database log closes with
Disc DR	the same per disc, shown when the record spans more than one

The DR scale is the database's own: 7 and below flat red, 14 and above flat green, with 8–13 graded between.

8.3.3 Validation and an example

The meter was compared with dr14_tmeter's `compute_dr14` on the same Qobuz track, decoded identically: DR 9 against 9 (9.04), peak -0.13 dB and RMS -10.89 dB identical. The unit tests pin the properties that agreement rests on: a steady sine measures DR 0 at any level; one full-scale spike in otherwise steady material does not raise it; quiet passages under full-scale peaks give the range the formula predicts; a decode through ffmpeg matches the samples written; and one track at a time is on disk, with nothing surviving success, failure, cancellation or a crash mid-download.

Get Yer Ya-Ya's Out! as Qobuz serves it (16 bit/44.1 kHz) measures album DR 9 (tracks 8–10, every track peaking at 0 dBFS). The database holds it at DR 11 (2002 CD) and DR 9 (2014 HDtracks download): the stream is the 2014 master.

8.3.4 Costs and limits

- **Bandwidth.** The job is a second download beside the stream MusicPD is playing. On this box it ran at about 350 KB/s, so a ten-track CD-rate record took about twelve minutes. MusicPD's buffer absorbed the contention here; a hi-res record on a slow line may not.
- **Streams only.** A track MusicPD plays from its own library is not fetched; the button refuses rather than guessing.
- **One job at a time**, and its result lives in the panel's memory: it stays on the card until the next job, but not across a panel restart.
- **The record as queued.** If only part of an album is in the queue, the album value is over that part.

8.3.5 HTTP API

Endpoint	Purpose
`POST /dr/measure`	start measuring the playing record (`409` while a job runs or nothing plays, `507` below 1 GB free)
`GET /dr/measure`	the current or last job: status, fraction, message, per-track results, album and disc values
`POST /dr/measure/cancel`	stop after the track in hand; its file is still deleted

The job's state never carries a track URL.

8.4 The live DR estimate

Measure DR answers "which master is this?" after a download. **Estimate DR** answers a different question: how dynamic is what I am hearing *right now*, and how has that changed over the last minutes? It needs no download, uses no disk, and updates while the music plays.

8.4.1 What DR means

Dynamic range here is the gap between the loud peaks and the sustained loud part of the music, in decibels. A steady tone has none (DR 0). A heavily compressed, "loud" master keeps its peaks close to its average level and reads low; a master that lets quiet passages breathe and peaks stand out reads high. It is a property of the *master*, not of your volume knob, and it is not a measure of quality: a DR 8 pressing can be a fine recording. Compare a record with its other masters, not one genre with another.

The scale is the database's own (section 8.3): DR 7 and below flat red, 14 and above flat green, graded in between. A live reading uses the colors of the same scale.

8.4.2 The algorithm

The estimate uses the *same* algorithm as *Measure DR* — the TT Dynamic Range meter — applied to blocks of the audio as it plays:

1. The audio is cut into **3-second blocks**. The length is fixed by the standard and does not change with any setting.
2. Each block gets an **RMS** (with the factor 2, so a full-scale sine is 0 dB) and a **peak**, per channel.
3. Over the blocks in scope, the $\mathbf{RMS_{upper}}$ is the root mean square of the loudest 20 % of block RMS values, and the **reference peak** is the *second-highest* block peak.
4. **DR** per channel is $20 * \log_{10}(\text{reference peak} / \text{RMS}_{\text{upper}})$; the reading is the mean over channels. The gauge shows it rounded, and the exact value to a hundredth in the status text.

What differs from a whole-track measurement is the *scope*, not the formula. A track measurement takes every block of one track; the estimate takes the blocks of a rolling window. Two consequences follow. A window shorter than a track is a reading of that passage, and a passage can read lower or higher than its track. And a reading over very few blocks is noisy: the first number appears after **six seconds** (two blocks), and it settles as blocks accumulate.

The audio is a 44.1 kHz tap on MusicPD's secondary output. Material at 44.1 kHz passes through unchanged and uses the reference meter's block length; other rates are resampled to 44.1 kHz first. Expect the estimate to land within a fraction of a point of the track's DR when the window covers the track (not measured); treat it as an indication, not as a value to cite against the database.

Only per-block statistics are kept — two numbers per channel per block. No audio is recorded or saved. The server keeps **one** history for every browser (up to 90 minutes, about 1 800 blocks); a second browser reuses it and adds almost no work.

8.4.3 What the panel shows

Turn the estimate on with **Estimate DR** on the renderer card. The panel then shows:

- **The gauge and its number** — the DR of the blocks in the selected window, back to the latest silence. A needle marks the value on the red-to-green scale. The number, the gauge and the status line share the top row of the panel; on a narrow screen the status wraps below.
- **The status line** — how many seconds are sampled, the window, or *Paused / Stopped — waiting for audio* when MusicPD is not playing.
- **The segment bar** — the history, oldest on the left, spanning the full width of the panel and covering the whole window. The audio collected so far is divided among as many segments as fit (each at least about 1.1 times the width of two wide characters), so a wider panel shows the same history in finer segments and a narrow one in coarser segments: a 90-minute window on a wide screen draws a fine curve of the level over the whole listening. All segments are recalculated whenever a new block arrives or the panel is resized, so their values shift slightly as the audio moves through them. Each segment shows the DR of its own blocks, colored on the same scale, with the rounded value at its base. Its colored height is proportional to its mean RMS level, full height at 0 dB and empty at -40 dB, so a loud, compressed section stands out from a quiet one.
- **Segment details** — hover a segment, or **tap** it on a touch screen (which has no hover), to see its time range (seconds before the latest interval), exact DR and level on a line under the bar. Tapping it again clears the selection.
- **The time labels** — the left label is the time the bar covers ("-4 min 30 s"), the right one is *Latest*.

8.4.3.1 Silence, pauses and stops

A complete block whose peak is below -80 dBFS is **silence**. Silence, a pause or a stop is drawn as **one** narrow, hatched segment, in a disabled color, between the audio before and after it. Pausing or stopping MusicPD adds that single empty segment however long it lasts, so a long stop does not push the earlier history out of the window. The history before it stays until the next play. When playback resumes, the gauge starts afresh with the next audio; if the song changes, **Detect song change** starts the view at the new track. Seeking inside a track keeps the completed blocks.

8.4.4 Options in the panel

Control	Effect
Estimate DR / *Stop estimate*	starts or stops the listener. The history is kept in the server only while at least one browser is listening; the first listener to join starts it afresh
Show DR / *Hide DR*	shows or hides the panel without stopping the estimate
Rolling window slider	1 to 90 minutes in 1-minute steps; the value is written to the right of the slider. The window is how much history the gauge and the bar cover. Long windows are the user's choice — a 90-minute window mixes many tracks
Detect song change: On / Off	*On*: when MusicPD moves to another track, this view starts at the track boundary — the gauge and bar describe the current track only (up to the window); the earlier history is kept on the server for views with the switch off. *Off*: the window follows a slice of audio across tracks, for a long stretch you care about as a whole rather than track by track

Detect song change is a per-browser view. The server keeps a single history across track changes and reports each boundary; whether a browser starts its window there is that browser's own choice. The window, the song-change setting, whether the panel is visible and whether the estimate is on are **saved in the browser** and restored on reload, so the panel returns as you left it. An active estimate keeps running when you switch between *Listen*, *System* and *Spectrum*; it stops when the browser tab is hidden or closed, so an idle tab does not keep MusicPD's tap open.

8.4.5 Configuration

The panel refreshes at most every 5 seconds. This limits how often the display updates and the traffic it causes; the measurement blocks are still 3 seconds. The interval is `dr_refresh_seconds` in the `[spectrum]` section of `commands.conf` (default 5, minimum 3); restart `omdrcctrl` to apply it. The estimate also needs the spectrum tap itself (`[spectrum] enabled`); it uses MusicPD's secondary FIFO output, which is disabled when *Estimate DR*, Spectrum and Levels all have no listeners.

8.4.6 Costs and limits

- **The history lives in the panel's memory.** Restarting `omdrcctrl` empties it, and the bar starts again from nothing.
- **The tap follows MusicPD.** Sources that do not go through MusicPD are not estimated.
- **An indication, not a citation.** Use *Measure DR* for a value to compare with the database; use the estimate to see how dynamic a passage is and how a track changes over time.
- **Server cost is per block, not per browser.** Each 3-second block costs one calculation and one serialization of the history for all connected browsers; each browser then draws its own window.

Chapter 9

The DR log: album DR remembered across listening

The **DR log** switch on the kiosk's DR page keeps the meter running on the server whether or not any page is open, and stores the DR of every track played (`dr-albums.sqlite` in the state directory). The switch is saved on the server and survives restarts. While it is on, MusicPD's FIFO output stays enabled; the Audio chain draws the log as its own consumer of that FIFO, **DR log - always on**, and says so when the log is on but cannot measure (the CD input, the analyzer disabled, MusicPD down).

- **Per track, not per album.** A track's DR is computed from its own blocks when it ends (the next track starts, or playback stops). A track heard for less than 30 seconds is not kept. A track is *heard whole* when it was heard from its start, never sought, and to its end; a seek or a late start keeps it as a partial measurement.
- **The album figure is worked out when read**, as the reference does: the mean of the tracks' integer DR values, rounded. It is **exact** when every track of the record has been heard whole (over any number of sessions), when the *Measure* job has measured every file, or when a local folder has a `dr14.txt`. Otherwise, with at least two minutes heard, it is an **estimate**, shown as `~DR9` on the page (with a dashed outline) and in the search results.
- **Albums** are keyed by their Qobuz album id, by their folder under MusicPD's `music_directory`, or, for other streams, by artist and album tags (never exact: the number of tracks is unknown).
- **The local collection** is read in: every `dr14.txt` under `music_directory` is imported shortly after start, as the DR14 scan comes across it or writes it (section 9.2), and on **Rescan dr14.txt**. Names and the track count come from the tags MusicPD indexed for the folder's tracks (a cue sheet's tracks included); only an untagged folder falls back to its name, read as "Artist - Album" when it has that form. A parent folder is never taken for the artist.
- **Albums by dynamic range** lists every album with a figure, highest first, filtered by source (Qobuz, Local, Other), *Exact only* and free text. Tapping an album shows its tracks; a Qobuz album can be played from there.
- **The live bar** stays continuous across tracks; it draws a solid divider at each track start and a dashed one at a seek, and a tapped segment names its track and that track's own DR.

The rolling figure of the estimate itself is never stored: it is computed over blocks of several tracks at once, which is not how an album's DR is defined.

9.1 Which measurement wins

For each track, the best measurement stored is used: a complete one beats a partial one, a **measured** one (the *Measure* job: the file fetched again and measured whole, the TT algorithm checked sample for sample against `dr14_tmeter`) beats one **heard** live, and more seconds beat fewer. A local album's `dr14.txt` is used before any of them.

Heard whole is measured the way the meter measures a file: the analyzer keeps 50 ms sub-blocks of the stream and, once a track has ended, groups them into 3-second blocks counted from the track's own first sample, the last one partial, with the block length of the track's source (3 x 44 160 samples at 44.1 kHz, exactly 3 s at any other rate). The first sample is known exactly when the track starts after silence; when one track runs into the next it is placed from MusicPD's elapsed time, corrected by the latency measured on the tracks that did start after silence. Cutting the blocks wherever they happen to fall is not good enough: on a short track it alone moves the result by half a DR. Measured this way, a live track reads what an offline meter reads on the same 44.1 kHz signal, to within 0.01. What remains is the tap itself: MusicPD resamples a hi-res track to the FIFO's 44.1 kHz, which removes ultrasonic content and can move the second-highest peak; on two 96 kHz test tracks that made the live value 0.08 higher than the file's, the same integer DR. *Measure* reads the file at its own rate and settles it.

The same order decides whether a new track measurement is kept at all (`DrStore.record_track()`). This box keeps one row per track and replaces it only with a row that ranks at least as high, comparing in turn:

1. complete beats partial;
2. `measured` beats `live`;
3. more seconds measured beat fewer;
4. on a tie, the newer row wins.

So a live hearing never replaces a complete *Measure* result, a partial hearing never replaces a complete one, a second complete hearing replaces the first, and a longer partial hearing improves a shorter one. Rows from other boxes are stored beside this box's own and never replaced by it. When the album figure is read, the best row of each track is taken, from whichever box measured it.

The album figure is then decided in this order (`dr_store.summary()`):

Case	Figure	Shown as
The album has a `dr14.txt` value	the report's album value	exact, *from dr14.txt*
Every track has a complete row	rounded mean of the tracks' integer DRs	exact, *measured* or *listened*
At least 120 s stored over its tracks	the same mean over the tracks stored	estimate, `~DR`
Less than that	none	*under two minutes so far*

The report's value comes from this box's own scan or, for an album on a drive with a volume marker, from another box that read the drive's report, until this box reads it itself.

9.2 Where a dr14.txt value comes from

The database is the one source of every figure shown, search results included; the `dr14.txt` files are what it is kept in step with. MusicPD does not watch the music directory (`auto_update "no"`), so a folder copied in is seen when the library is rescanned: **Rescan** on the Local page runs `omdrc-mpd-update-dr14.sh`, which updates MusicPD's index and then runs the DR14 scan. The scan names every folder with a report in `local-scan.reports` in the state directory, one line each, as it goes:

- `found<TAB><folder>` for a folder that already has a `dr14.txt` (one copied in with its report), listed while the scan walks the library, before it measures anything;
- `measured<TAB><folder>` for each folder it has just measured and written a `dr14.txt` for (one copied in without a report).

omdrc-ctrl reads the new lines every 5 seconds and reads just those folders' reports, so a folder appears with its figure within seconds, not when the scan ends. An unchanged report is skipped by its modification time. When the scan finishes, the whole collection is walked once more, which also notices a report that was deleted. A local album that a search finds with a `dr14.txt` the database has not read yet is read in on the spot.

A report that disagrees. When a folder's `dr14.txt` gives a different value from the one the database already holds for it (a folder copied over with another report, a report edited by hand), one of the two is wrong. The new value is not taken. The folder is written to `local-scan.recheck`, a DR14 scan is started (or the next one is used, if one is running), and the scan measures that folder's audio again, although it has a report, and rewrites its `dr14.txt`. The database takes the measured value; until then it keeps the old one. Each version of a disputed file is sent once: a folder the meter cannot measure is not sent back on every look (until omdrc-ctrl restarts).

9.3 The live meter on an album that already has a value

The live meter does not look at what the store already knows. It computes the rolling figure for the DR panel and the Now page, works out each track's own DR when the track ends, and hands every track heard for at least 30 seconds to the store as a `live` row, whatever the album is. What happens next depends on what the album already has.

An album with a `dr14.txt`. The live rows are stored beside the report, but they do not change the album's figure: the report wins outright, and the album stays *exact, from dr14.txt*. The live rows are still used for these things:

- **Recent DR** lists them as tracks heard on this box, the same way as on any other album;
- **Ordering of equal values.** Among albums with the same integer DR, the ranking orders by the unrounded mean of the stored tracks; a reported album with no stored tracks uses its report value;
- **Sharing.** They are exported to the other boxes like any track row;
- **A fallback.** If the `dr14.txt` is deleted, the walk at the end of the next DR14 scan clears the report value but keeps the album because it has tracks, and the figure is then worked out from them.

The album page of a local reported album shows the per-song rows read from the `dr14.txt` itself, not the live rows.

An album measured or heard before, without a report. The new live row competes with the stored one for its track under the order above: it is kept only if it ranks at least as high. A complete *Measure* result therefore stays, and listening again can only complete or lengthen what was heard. The album figure is recomputed from the best row of each track the next time it is read, so an estimate becomes exact once the last missing track has been heard whole.

The rolling figure. The rolling session figure on the DR panel is never stored and never compared with a stored value. It is computed over blocks from several tracks at once, so the DR panel can show a value next to the album's stored figure that differs from it.

9.4 Editions

Every edition is an album of its own and is never merged with another by name: Qobuz albums by their album id (a remaster, a mono or a hi-res release each have their own, and the stored title carries Qobuz's version, e.g. "(2011 Remaster)"), local albums by their folder (the ranking shows the folder when the tags name two editions the same). Only streams that are neither Qobuz nor local are keyed by artist and album tags.

9.5 Sharing between boxes

Boxes that cannot reach each other (home and office) share the log through a git repository both can reach, configured by `DR_SYNC_REPO` and `DR_SYNC_BOX` in `host.cmake` (rendered into `[dr_sync]` of `commands.conf`): `repo` (e.g. a private GitHub repository), `box` (this box's name, default the host name) and `interval_minutes` (default 10). Each box commits only its own file, `boxes/<box>.jsonl` (its own tracks, album names and local reports), and imports the other boxes' files; nothing is ever overwritten, so a box can stay away for weeks. Rows from other boxes are kept beside this box's own, and an album's figure can combine tracks heard on both. Another box's local albums appear as `local@<box>:<folder>`.

A drive that moves between boxes is the exception. The first box that sees a writable drive in the music directory writes a marker at its root, `.omdrc-volume` (a random id; `uuidgen > <mount>/.omdrc-volume` does the same by hand on a read-only drive). Its albums are then keyed `vol:<id>:<folder on the drive>`, the same on every box whatever the mount point or the link that reaches it, so the albums are not duplicated and tracks heard on either box add up. The `dr14.txt` reports travel with the drive: plugged into another box, they are read within a minute, without a rescan, and until then that box shows the values the other box shared. An unplugged drive keeps its albums. Older `local:` rows are rekeyed the first time their drive is seen with a marker. Git runs as the service user without prompting: give it credentials first (`gh auth login` and `gh auth setup-git`, a token in `~/.git-credentials`, or an SSH deploy key). The DR page shows the last round and offers **Share now**.

9.6 Setting up the private repository

Done once, then once per box. `<service user>` is the account the panel runs as (`AUDIO_USER`; `omdrcctrl_user` under FreeBSD); the git credentials must belong to *that* user, not to root or to your login.

1. **Create the repository** (any machine, once). It must be private: it lists the albums you play. With the GitHub CLI:

   ```
   gh repo create omdrc-dr-log --private
   ```

   Leave it empty. The first box to sync creates the `main` branch.

2. **Give each box access**, as the service user (`su - <service user>` or `sudo -u <service user> -H sh`). Pick one:

   - *HTTPS with the GitHub CLI:* `gh auth login`, then `gh auth setup-git`. This stores a token in the user's home; use a fine-grained token limited to this repository, with Contents: read and write.
   - *SSH deploy key (no account login on the box):*

     ```
     ssh-keygen -t ed25519 -N '' -f ~/.ssh/omdrc-dr-log
     cat ~/.ssh/omdrc-dr-log.pub
     ```

     Add the public key under the repository's Settings, Deploy keys, with **Allow write access**. Point SSH at it in `~/.ssh/config`:

     ```
     Host github-omdrc
         HostName github.com
         User git
         IdentityFile ~/.ssh/omdrc-dr-log
         IdentitiesOnly yes
     ```

     and use `git@github-omdrc:<you>/omdrc-dr-log.git` as the `repo`. A deploy key serves one repository only, so a second box needs its own key.

3. **Check that git works non-interactively**, still as the service user. This is exactly what the panel does and it never prompts:

   ```
   GIT_TERMINAL_PROMPT=0 git ls-remote <repo>
   ```

   An empty answer on a new repository is correct; a password prompt or an error means step 2 is not finished.

4. **Configure the box in `host.cmake`**, then install:

   ```
   set(DR_SYNC_REPO "https://github.com/<you>/omdrc-dr-log.git" CACHE STRING "")
   set(DR_SYNC_BOX  "home" CACHE STRING "")
   ```

   CMake renders these into the installed `commands.conf`, so every reinstall keeps sharing on (`commands.conf` itself is overwritten by each install; edits made only there are lost). Use a fresh build directory when adding them to an existing build. The rendered result is:

   ```
   [dr_sync]
   repo = https://github.com/<you>/omdrc-dr-log.git
   box = home
   interval_minutes = 10
   ```

   `box` must differ on every box (letters, digits, `-`, `_`). It names the box's file, so renaming it later abandons the old file.

5. **Restart the panel** (`systemctl restart omdrcctrl` on Linux, `service omdrcctrl restart` on FreeBSD). The first round starts after about 30 seconds; press **Share now** on the

DR page to see it at once. The page shows the last round, any error, and each other box with its last import. The working clone is `dr-sync/` in the state directory and can be deleted safely: it is cloned again.

Where things live. Nothing has to be initialised or cloned by hand: the panel clones the repository itself on its first round, and creates the `main` branch if the repository is empty.

- *The repository URL* is read from `[dr_sync]` in the `commands.conf` the service was started with (`--config`; check with `ps` on the running `app.py`). The installed file is `<prefix>/etc/omdrcctrl/commands.conf`. The program's built-in default, `<prefix>/etc/open-media-drc/commands.conf`, applies only when neither `--config` nor `$OMDRCCTRL_CONF` is given. Nothing needs to be inside `etc/open-media-drc`.
- *The working clone* is `dr-sync/` in the state directory, with `dr-sync.imported.json` beside it (which other-box files were already imported). The state directory is `$OMDRC_STATE_DIR` if set, else `/var/db/omdrc` when running as root, else `${XDG_STATE_HOME:-~/.local/state}/omdrc` of the service user, which is the usual case. Both can be deleted: they are rebuilt on the next round.
- *The database* is `dr-albums.sqlite` in the same state directory. It is not in the repository; only the exported rows are.
- *Git credentials* are the service user's own: `~/.gitconfig`, `~/.git-credentials` or the `gh` configuration, and `~/.ssh`. They are never stored in `commands.conf` or in the repository.

Repeat steps 2 to 5 on the other box with a different `box` name. Each box then holds its own `boxes/<box>.jsonl` and imports the others'. Nothing else is needed: there is no server, and a box that is off or offline simply catches up at its next round.

Chapter 10

CD input: S/PDIF capture into the DRC chain

A CD transport is the second source the chain accepts, and the only one that is not a file: it arrives as a live S/PDIF signal, clocked by the disc, on a USB capture interface. A bridge writes it into the same loopback MPD uses, so a disc gets exactly the same room correction as everything else. It takes the seat `mpv` takes for video: a non-MPD writer into the loopback BruteFIR reads.

This chapter holds what is common to both systems. The bridge itself is **`alsaloop`** on Linux (chapter 12) and the purpose-written **`omdrc-cdin`** daemon on FreeBSD (chapter 18); the web card below is the same on both, because both emit the same log grammar.

10.1 Two clocks, no resampler

Capture is slaved to the CD's crystal and the DAC runs on its own; the two differ by a few ppm **forever**. Neither bridge resamples: the data path is bit-identical to what the transport sent, which the bit-perfect tools can verify. The drift is absorbed elsewhere, by a mechanism each bridge chapter describes.

10.2 CD input is an exclusive source

Selecting CD input remembers whichever MPD output is enabled (`OKTO-DAC`, `DRC-native` or `DRC-resamp`) and disables all of them; stopping the bridge restores exactly that output. The `OMDRC Spectrum` FIFO is not an audible output and is left alone. Each bridge chapter says how the rule is enforced.

`drc.sh` carries the choice in its persistent state:

- `drc.sh cdin` records `last_source=cdin` and requests the 44.1 kHz chain, so a reboot restores CD mode at 44.1 kHz.
- An ordinary rate action is the explicit return to music: it writes `last_source=music` *before* validation or any teardown, as write-ahead intent. If the music transition fails, the next `restore` or hotplug `reconcile` retries music instead of resurrecting stale CD mode.
- `off` and the transient `stop` do not change the source.
- A geometry change in CD mode is refused if the target geometry has no 44100 Hz configuration.

10.3 The capture interface: ESI U24 XL

The capture interface is an ESI U24 XL (USB Audio Class 1.0, USB 2.0 Full Speed, 32/44.1/48 kHz, 24-bit maximum). The vendor documentation (ESI KB00307EN) settles two things:

- **it slaves to the incoming S/PDIF automatically** — when the source is clock master, "the U24 XL will receive clock from the source and automatically will be slave"; there is no manual clock switch;
- **the sample rate is not auto-detected**: depth and rate must be set to match the incoming signal. The bridge sets 44100 Hz, so a non-44.1 source would be captured at the wrong rate rather than refused.

It is class compliant, so the OS's generic USB audio driver handles it with no vendor driver. **Do not update the interface firmware**: a Linux report has S/PDIF capture working on the original firmware and becoming "completely distorted" after an upgrade.

The card has two inputs, only one live, chosen by a mixer setting that does **not** survive a reboot or a replug — it comes up on the *analog RCA* input, so the bridge records silence from a healthy transport until the S/PDIF input is selected. The OS-specific way to set it on every attach is in section 12.1 (Linux) and section 18.6.1 (FreeBSD).

10.4 The web panel card

`omdrc-ctrl` (section 6) shows a **CD input** card driven entirely by that log, under the Renderer card. It is configured by the `[cdin]` section of `commands.conf`, which configures nothing about the daemon itself — only where to read it from, plus whether the buttons exist:

```
[cdin]
enabled  = yes
log_file = /tmp/omdrc-cdin.log     # must match omdrc_cdin_logfile
process  = omdrc-cdin              # pgrep -x: tells "stopped" from "broken"
service  = omdrc_cdin              # what Start/Stop runs
control  = yes                     # offer the Start/Stop button at all
refresh  = 5
max_events = 20
```

The card is the size of its news, which for a bridge with no disc in the player is one line:

Daemon state	The card
`state playing`	full size, opened by itself — the only state with anything to watch
idle / no carrier	the status line and an expand chevron; still polling, so a disc reopens it
stopped	the status line and a **Start** button

Expanding or collapsing by hand sticks until the daemon's own state changes. Expanded, the card adds both device paths and their tenancy, the `[stats]` line as chips (`buffer 1962 ms (min 1955)`, `underruns 0`, `dropped 0 B`, `drift +1.2 ppm, fills in 46 h`, `silence`, `up`), a sentence

for anything that has already cost something audible, the raw stats line, and a scrolling event list. Four rules decide what goes where:

- **the LED follows device availability only** — red when an end cannot be opened, because that is the question "can a disc play right now?". A released output device is the daemon working correctly and never colours anything;
- **failures are kept, health is replaced.** Every error stays in the list, in red, in chronological order, even after the condition clears, and the newest one gets its own red line above the fold. The healthy status line is replaced rather than accumulated. A live status line can never say "the output was missing for ten minutes this morning", and that is exactly the thing worth saying;
- **`starves` is the number to surface.** The lead is a margin; an underrun is that margin having run out — a dropout that already happened and that nothing else in the chain will ever mention again;
- **a daemon that is not running is idle, not broken**, however alarming the tail of its log is. The log outlives the process, and nothing is unavailable when nothing is trying to open it. With no daemon and no log at all, the card hides itself.

Three buttons: refresh, **Log** (opens the whole bridge log in the Logs card), and **Stop**/**Start**, a source hand-off around the bridge service. Start records whichever of `OKTO-DAC`, `DRC-native` or `DRC-resamp` is enabled, then disables all three before starting; a failure to gate MPD aborts the start. Stop waits for the bridge to release its output and restores exactly the recorded output. A failed or timed-out start also restores MPD. The service name is the OS's own (`omdrc_cdin` rc.d on FreeBSD, `omdrc-cdin.service` on Linux); the FreeBSD `sudoers` grant the button needs is in section 18.5.1.

Part II — Linux

Everything a Linux (Arch) host needs beyond Part I: packages, systemd and udev integration, the `snd-aloop` loopback, ALSA audio roles, browser audio, the panel's Linux behaviour, the CD bridge and Blu-ray decryption with MakeMKV. **On FreeBSD, skip to Part III**; nothing in Part I depends on this part.

Chapter 11

Linux: installation and lifecycle

Do the common build (chapter 3) with these Linux specifics.

11.1 Packages

Component	Arch package
libnpupnp, libupnpp	`curl libmicrohttpd expat`
upmpdcli	`jsoncpp libmpdclient`
upmpdcli Qobuz plugin	`python python-requests`
MPD	`mpd`
BruteFIR (fork)	`fftw alsa-lib`
Loopback	`snd-aloop` (kernel module, no package)
CD bridge	`alsa-utils` (`alsaloop`, `amixer`)
omdrc-ctrl	`python-flask python-markdown python-numpy`

```
sudo pacman -S mpd                  # MPD, chapter 3 step 2
sudo ldconfig                       # after installing the upmpdcli stack
```

11.1.1 BruteFIR defaults

```
mkdir -p ~/.config/BruteFIR
cp etc/open-media-drc/brutefir_defaults.linux.conf \
   ~/.config/BruteFIR/brutefir_defaults.conf     # ALSA I/O
```

11.2 Files that must live in /etc

The CMake install copies everything into `$PREFIX` (default `/usr/local`). Two files are read *before* `$PREFIX` is on the relevant search path:

- **udev rule** — udev scans only `/etc/udev/rules.d` and `/usr/lib/udev`, never `/usr/local/lib/udev`. The install places `99-usb-audio-drc.rules` under `$PREFIX/lib/udev/rules.d` and prints the one-line copy into `/etc/udev/rules.d` (then `udevadm control --reload`).

- **MPD `User=` drop-in** — described next; the repository installs it directly, so it needs no manual copy.

The systemd units live in `$PREFIX/lib/systemd/{system,user}`, which systemd *does* scan. After the install, the printed checklist enables `mpd`, `omdrcctrl` and `omdrc-renderer`.

11.2.1 The MPD `User=` drop-in caveat (Arch)

The Arch `mpd` package ships a systemd drop-in (`/usr/lib/systemd/system/mpd.service.d/00-arch.conf`) setting `User=mpd`. Drop-ins always apply *on top of* the main unit, so a full unit override at `/etc/systemd/system/mpd.service` **cannot** override that `User=` — it silently loses. The repo therefore ships a **counter-drop-in** (`etc/systemd/system/mpd.service.d/open-media-drc.conf`, rendered by `cmake/renderers.cmake` and installed to `$PREFIX/lib/systemd/system/mpd.service.d/`) that sets `User=` to the audio user and the installed config path; a drop-in in `/usr/local/lib` beats one in the distribution's `/usr/lib` load path. `cmake --install` installs it directly; no manual `/etc` copy is needed. Do not create a full `/etc/systemd/system/mpd.service`.

11.3 USB DAC hotplug (udev + systemd)

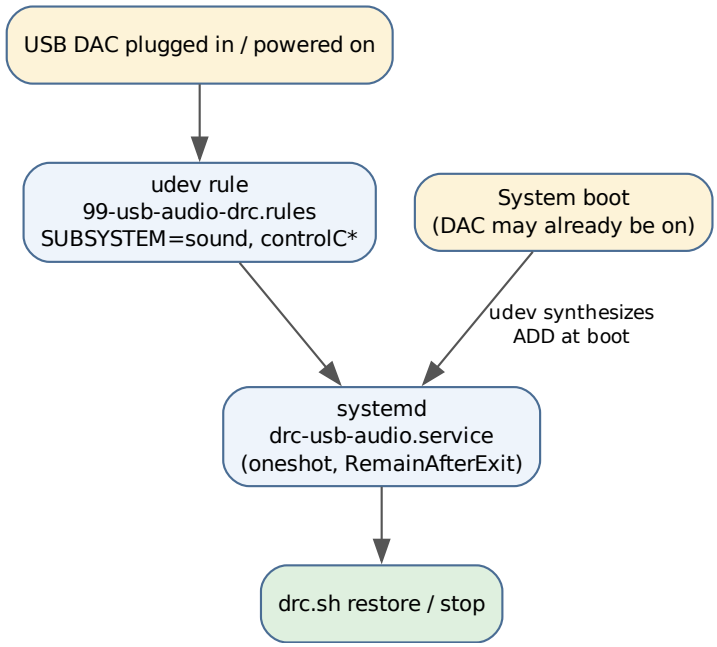


Figure 11.1: Linux hotplug path: udev synthesizes ADD events at boot, so one service covers boot and hotplug.

File	Installed to	Purpose
`99-usb-audio-drc.rules`	`/etc/udev/rules.d/`	Triggers the service on DAC plug/unplug
`etc/systemd/system/drc-usb-audio.service`	`/etc/systemd/system/`	Starts/stops DRC
`mpd.service.d/open-media-drc$PREFIX/lib/systemd/system/mpd.service.d/`		MPD service config and post-start routing-reconcile drop-in

The udev rule matches any USB sound-card control device and pulls in `drc-usb-audio.service` (`Type=oneshot`, `RemainAfterExit=yes` so the several `controlC*` events of one plug never start duplicate BruteFIR instances). `ExecStart` is `drc.sh restore` after a 1 s settle; `ExecStop` is `drc.sh stop`. Because udev synthesizes ADD events for already-present devices at boot, the same service covers boot and hotplug.

Manual control: `sudo systemctl start|stop drc-usb-audio.service`, `journalctl -fu drc-usb-audio.service`.

11.4 The loopback: snd-aloop

MPD plays into `hw:Loopback,0,0` and BruteFIR reads `hw:Loopback,0,1`. The `snd-aloop` module is loaded at boot by `etc/modules-load.d/`.

`snd-aloop` has no clock of its own and by default invents one, an hrtimer. That makes **two** independent drift pairs on Linux (CD <-> loopback timer, and loopback timer <-> DAC), the second present even in plain MPD playback. It is normally invisible only because both BruteFIR stanzas in `etc/open-media-drc/brutefir_defaults.linux.conf` set `ignore_xrun: true`.

`etc/modprobe.d/omdrc-snd-aloop.conf` removes the second pair by pointing the module's `timer_source` at the DAC card:

```
options snd-aloop index=1 id=Loopback pcm_substreams=2 timer_source="hw:0,0,0"
```

The loopback then advances at the DAC's rate, so CD-to-DAC drift is the only drift left — a prerequisite for the CD correction in chapter 12, not an optimisation. `omdrc-config-helper` rewrites the marked line to whichever DAC is selected on `/configuration`; the module loads at boot, so a change takes effect on the next boot.

11.5 Audio roles

The source of truth is the USB identity selected on `/configuration`, not an ALSA card number and not a DAC name in `host.cmake`. Three files hold it:

File	Contents and lifetime
`$PREFIX/etc/open-media-drc/audio-roles.conf`	Persistent DAC and capture USB identities (`vid:pid[:serial]`)
`/run/omdrc/audio.roles`	Current-boot ALSA card numbers and descriptions, regenerated during Apply and hotplug reconcile

File	Contents and lifetime
`$PREFIX/etc/open-media-drc/browser-alsa.conf`	Current ALSA card IDs used by browser playback and capture

On **Apply**, `omdrc-config-helper` resolves the chosen identities against the attached cards and updates BruteFIR, MPD, the runtime roles and the browser ALSA configuration as one operation, then restarts the DRC lifecycle service. On boot or USB hotplug, `omdrc-audio-roles.service` runs the same reconcile: ALSA indexes may move from `card0` to `card2`, but the persistent identity is resolved again and every generated file receives the current card. Selecting another DAC therefore needs no CMake reconfiguration, no `.asoundrc` edit and no attempt to pin the DAC at card 0.

The known-device list of section 6.4.1 is the `OMDRC_AUDIO_DAC` / `OMDRC_AUDIO_CAPTURE` value in `audio-roles.conf`, applied by `linux_pick` in the helper. If two identical cards expose no serial, unplug one before Apply.

11.6 Browser audio: the managed ALSA default

The No DRC launchers (section 4.6) stop DRC and hand the DAC to a small ALSA mixer:

```
browser -> ALSA default -> plug -> dmix (48 kHz, S32_LE) -> selected DAC
```

MPD and BruteFIR name raw `hw:` devices explicitly, so they bypass this default and its mixer: the browser configuration inserts no resampling into music or the DRC chain. `dmix` shares browser streams with each other but not the DAC with MPD or BruteFIR while either holds the raw device. ALSA is used directly; no sndio daemon, PulseAudio or PipeWire is needed.

Installation. `cmake/browser-alsa-linux.cmake` installs `$PREFIX/share/open-media-drc/asoundrc.linux.co`
A live `make install` renders the saved roles into `$PREFIX/etc/open-media-drc/browser-alsa.conf`
and adds this marked block after the audio user's existing ALSA settings:

```
# BEGIN open-media-drc browser ALSA
</usr/local/etc/open-media-drc/browser-alsa.conf>
# END open-media-drc browser ALSA
```

The destination is `~/.config/alsa/asoundrc` when it exists (alsa-lib loads it after `~/.asoundrc`), otherwise `~/.asoundrc`. Existing content is kept byte for byte, a one-time backup `.omdrc-before-browser-alsa` is made, reinstallation replaces the marked block instead of duplicating it, and a malformed or hand-edited partial block is refused rather than guessed at. With `DESTDIR` only the template is staged and no home directory is touched. Set `OMDRC_INSTALL_BROWSER_ALSA=OFF` to leave desktop ALSA unmanaged (this does not remove an earlier block).

Device selection. The generated file names the DAC by ALSA card ID (for example `hw:CARD=DAC8STEREO,DEV=0`), independent of the boot's numeric index, and is regenerated by Apply and hotplug reconcile. If the saved DAC is absent the output is an unavailable sentinel — never card 0, which could be an unrelated capture interface or HDMI. Without a capture card, browser capture uses ALSA's `null` PCM. The mixer is fixed at stereo, 48000 Hz, `S32_LE`, so the DAC must support that mode; `plug` converts ordinary browser formats and rates before `dmix` (intentional for desktop audio, outside the bit-perfect paths).

Backends. Chromium should use its ALSA backend and the `default` PCM (`--alsa-output-device=default` makes it explicit). Firefox needs ALSA support; if it keeps choosing an absent sound server, set

`media.cubeb.backend` to `alsa` in `about:config`. Restart the browser fully after selecting another DAC.

Verification and recovery. Close every browser, launch through **No DRC**, play two tabs, then:

```
cat /usr/local/etc/open-media-drc/audio-roles.conf
cat /run/omdrc/audio.roles
sed -n '1,120p' /usr/local/etc/open-media-drc/browser-alsa.conf
cat /proc/asound/<selected-card-id>/pcm0p/sub0/hw_params
```

The last file must show a two-channel 48000 Hz `S32_LE` stream on the selected DAC, and the capture interface's playback PCM must stay closed. If the browser is silent, check in order: it was launched through **No DRC** with no earlier process left; `audio.roles` and `browser-alsa.conf` name the DAC selected on `/configuration`; `/proc/asound/*/pcm*p/sub*/hw_params` shows MPD, BruteFIR or something else holding the raw DAC; the browser was restarted after Apply or a reconnect; the backend selection above. To stop managing ALSA, restore the backup or delete only the three-line marked block.

11.7 The panel on Linux

- **Service**: `omdrcctrl` is a systemd **system** unit running as the audio user (`sudo systemctl restart omdrcctrl`); the renderer switch drives `systemctl --user`.
- **DAC feed** in the MPD panel is the ALSA `hw_params` read from `/proc/asound`.
- **BruteFIR CPU** is matched by `argv[0]`, because on Linux brutefir renames its `comm`.
- **Spectrum from the CD**: `alsaloop` is opaque and the supervisor never sees a sample, so omdrcctrl reads the *same capture device* a second time through an ALSA `dsnoop` and writes the FIFO itself. It is an independent reader on purpose: a stalled analyzer misses samples instead of stalling the CD.
- **Glitch detection** (section 16.4) is implemented for FreeBSD; on Linux its scripts run but the FreeBSD-specific sources stay quiet.
- **Bit-perfect tap**: `bitperfect-tap-linux.sh` reads usbmon's binary interface, so it has no capture-size limit; it escalates with `sudo -n`.
- `scripts/systemd-user-install.sh` is a legacy helper that links and enables a `systemd --user drc.service`.

Chapter 12

Linux: CD input with alsaloop

Everything in this chapter is built and reasoned but has not run on the Linux box yet — see "What has not been measured" at the end before trusting it on real hardware. Full detail: `doc/CDIN-LINUX.md`. The concept, the exclusive source rule and the web card are in chapter 10.

```
CD player --S/PDIF 44.1k--> ESI U24 XL --USB--> hw:<cap>,0
                                                  |  alsaloop
                                                  v
                                        hw:Loopback,0,0   (snd-aloop)
                                                  |
                                                  v
                                        hw:Loopback,0,1
                                                  |  BruteFIR
                                                  v
                                              hw:0,0      (Okto DAC8)
```

Linux ships the clock reconciler FreeBSD lacks, so no daemon is written: `alsaloop(1)` from `alsa-utils` is supervised by `omdrc-cdin.service`. With the loopback pinned to the DAC clock (section 11.4), CD-to-DAC is the only drift pair left.

Drift correction uses alsaloop's `playshift` mode, which steers `snd-aloop`'s `PCM Rate Shift 100000` control so the loopback consumes at exactly the rate the CD delivers — **no resampler is in the data path**. The supervisor reads the shift control back every stats interval and reports it as the `drift` field in ppm, a direct measurement of the correction applied. Other `--sync` modes (`OMDRC_CDIN_SYNC`) are for diagnosis: `samplerate` (libsamplerate, **not** bit-perfect, the fallback if the shift control misbehaves), `simple` (insert/drop samples), `none` (measure raw drift). `captshift` is useless here: a USB interface has no rate-shift control.

Exclusive source. `hw:Loopback,0,0` is a **single substream**, so alsaloop and MPD's `DRC-native`/`DRC-resamp` outputs cannot both hold it; whichever opens second gets `EBUSY`. Consequently:

- `drc.sh cdin` disables every MPD output, brings the chain up at 44.1 kHz and starts `omdrc-cdin.service`. MPD has no output while CD input is selected; that is the correct state, and the direct `OKTO-DAC` output is no help because BruteFIR holds the DAC.
- Any rate action (`drc.sh 44100`, `192000`, `resamp`, ...) records `music`, stops the bridge, **waits for the `alsaloop` process itself to be gone** (`systemctl stop` returns before the kernel

has closed the substream, and reopening too early is an `EBUSY` that surfaces as "MPD will not play"), then enables the MPD output.
- The unit is **not** enabled at boot; `drc.sh restore`/`reconcile` bring back the saved source.
- `drc.sh off`/`stop` return to MPD direct and leave the bridge down (FreeBSD moves its bridge to the DAC instead; here the DAC is single-open and MPD's direct output needs it). Re-select CD with `drc.sh cdin`.

When the DRC chain is down, the bridge writes straight to the DAC instead of the loopback. The output is settled once at startup (alsaloop negotiates a format against it and cannot re-decide while running), so the service is restarted whenever the chain moves.

12.1 The ESI input selector on Linux

Choose the capture interface in the `/configuration` Audio hardware section, as for the DAC (section 11.5). The selection persists in `audio-roles.conf`; `/run/omdrc/audio.roles` holds the resolved card number for this boot.

The card's input selector does not survive a reboot. Its Linux equivalent is an `amixer` capture-source switch, for example `amixer -c <card> cset name='PCM Capture Source' 1`. The control name is card- and kernel-dependent; put the command in an `ExecStartPre=` drop-in on `omdrc-cdin.service` so it is reapplied on every start.

12.2 Requirements and what has not been measured

Requirements: `alsa-utils`; `snd-aloop` with `timer_source` support (mainline since 4.x); a capture interface that takes its clock from the incoming S/PDIF carrier, as the U24 XL does.

Not yet measured: whether `timer_source="hw:0,0,0"` is accepted in that spelling and actually pins the loopback's `hw_ptr` to the DAC rate, and whether `--sync=playshift` finds the shift control on real hardware (the supervisor logs a warning naming the control if not; `--sync=samplerate` is the documented fallback).

Chapter 13

Linux: Blu-ray decryption with MakeMKV

The video launchers and the web remote are described for FreeBSD in section 17; on Linux the same scripts run with the ALSA settings of `video/lib/drc-audio.sh` and the tuning recorded in `video/BLURAY-PLAYBACK-TUNING.md`. This chapter covers the one part that differs in kind: how encrypted Blu-rays are decrypted, and the maintenance that choice brings.

13.1 libbluray, libaacs and KEYDB.cfg

A commercial Blu-ray is AACS-encrypted, and some titles add BD+. Neither mpv nor Kodi decrypts anything itself: both read discs through **libbluray**, which loads the decryption code at runtime as two plugin libraries.

Plugin	Job	Stock library	Overridden by
AACS	drive authentication, disc keys, title decryption	`libaacs`	`LIBAACS_PATH`
BD+	BD+ fix-ups on some titles	`libbdplus`	`LIBBDPLUS_PATH`

The two variables name the library libbluray opens (`libmmbd` is enough; the loader finds `libmmbd.so.0`). They are read by **libbluray**, not by the player, so they act the same way on mpv, Kodi and `bd_list_titles`.

With the stock plugins, decryption depends on `~/.config/aacs/KEYDB.cfg`, the FindVUK key database, which the web remote's **Blu-ray check** helps install. On this setup that is not enough, for two independent reasons:

1. **Bus encryption: the drive rejects libaacs's host certificate.** Before the drive releases the disc's volume ID, the host must authenticate with an AACS host certificate. The certificates libaacs can use are public and revoked, and a drive with a recent firmware MKB refuses them. mpv logs `Host key / Certificate ... has been revoked by your drive` and the

picture stays black **even when `KEYDB.cfg` holds the disc's key**. This is what happened on 2026-10-06 with the Samsung SE-506CB USB drive on the Linux box.

2. **Key coverage: `KEYDB.cfg` only knows discs that someone has submitted.** For a recent disc there is no public processing key for its MKB version, so libaacs needs that disc's own VUK in the file. A title missing from FindVUK does not decrypt, however fresh the download. Example: *Pink Floyd — Pulse* on Blu-ray is not in the current database.

13.2 Why MakeMKV, and why it is required here

MakeMKV ships **`libmmbd`**, which exports the libaacs and libbdplus interfaces, so libbluray can load it in place of either. Behind those it runs MakeMKV's own engine (`makemkvcon`): it authenticates to the drive in a way current firmware accepts, and it derives the disc keys itself instead of looking them up in `KEYDB.cfg`. It also covers BD+.

For **physical discs on this drive** MakeMKV is therefore required, not a convenience: without it, reason 1 blocks every AACS disc, and even on a drive that accepted the libaacs certificate, reason 2 would block every disc missing from FindVUK. The costs, accepted deliberately:

- MakeMKV is proprietary; the Linux build is free while "in beta", but only with a **beta key that expires** (section 13.4).
- `libmmbd` needs raw SCSI access to the drive (`/dev/sg*`): the player's account must be in the group that owns it (normally `optical` or `cdrom`).
- Decryption depends on a closed binary that has to be kept current.

`KEYDB.cfg` keeps its role on a box without MakeMKV: libbluray then loads the stock libaacs and the key file, as before.

13.3 How the players are told to use libmmbd

`video/lib/makemkv-env.sh` holds the switch, shared by every player launcher:

```
if [ "$(uname)" = "Linux" ] && [ -z "${LIBAACS_PATH:-}" ]; then
    for _omdrc_mmbd in /usr/lib/libmmbd.so.0 /usr/local/lib/libmmbd.so.0; do
        if [ -e "$_omdrc_mmbd" ]; then
            export LIBAACS_PATH=libmmbd LIBBDPLUS_PATH=libmmbd
            break
        fi
    done
    unset _omdrc_mmbd
fi
```

- **Linux only** — FreeBSD keeps the stock environment and `KEYDB.cfg`, by design (section 17).
- **Only when MakeMKV is installed** — `/usr/lib/libmmbd.so.0` is where the Arch `makemkv` package puts it. Without it, nothing changes.
- **Never over an explicit choice** — a `LIBAACS_PATH` already set is kept, so `LIBAACS_PATH=libaacs play-bluray.sh` still tests the stock path.

mpv. `video/lib/drc-audio.sh` sources it, and is itself sourced by all three mpv launchers (`play-bluray.sh`, `play-media.sh`, `webremote/mpv-idle.sh`). The idle mpv inherits the variables

when it starts, so a disc started from the web remote is decrypted by `libmmbd` too. Restart the idle mpv after installing MakeMKV.

Kodi. The panel's **Kodi** button runs `<prefix>/lib/omdrcvideo/kodi.sh` (`commands.conf` section `[kodi]`), which sources `makemkv-env.sh` and then executes `kodi`; on FreeBSD or without MakeMKV it is plain `kodi`. Kodi started another way (application menu, autostart) does not pass through the wrapper: start `kodi.sh` instead, or set the variables for the whole Plasma session with `export LIBAACS_PATH=libmmbd LIBBDPLUS_PATH=libmmbd` in `~/.config/plasma-workspace/env/makemkv.sh` of the desktop user, then log in again.

Avoid the common recipe of replacing `libaacs.so.0` and `libbdplus.so.0` with symlinks to `libmmbd.so.0`: it changes every program on the box, and a package update silently undoes it. To confirm that Kodi uses MakeMKV, play a disc missing from `KEYDB.cfg` (*Pulse*, for example); only `libmmbd` can play it.

13.4 Renewing the MakeMKV beta key

The free Linux MakeMKV works only with a **beta registration key**, and each key carries an expiry date, typically a month or two ahead. When it expires, `makemkvcon` refuses to work, `libmmbd` returns no keys and **every Blu-ray stops playing again**, with the same black screen or AACS error as without MakeMKV.

The current key is published by the MakeMKV author in the forum thread *"MakeMKV is free while in beta"* (https://forum.makemkv.com/forum/viewtopic.php?t=1053), in a sentence such as *"The current beta key is T-... and is valid until end of October 2026."* A new key appears before the old one expires.

The key belongs to **the account that runs the player**: `libmmbd` reads that account's `~/.MakeMKV/settings.conf`, line `app_Key = "T-..."`. On these boxes that is the desktop user who runs the idle mpv, Kodi and the `omdrcvideo` web remote.

13.4.1 From the video web remote

Installing a key is always a click, never automatic.

- **Blu-ray check** (menu, top right). Its first entry is the **MakeMKV beta key**. Every time the check opens it fetches the forum page, showing the progress in that entry, then lists the installed key's expiry and days left, the key currently published and its expiry, and any complaint `makemkvcon` itself raises about the key or an outdated release (it is not asked while a disc is playing). When the installed key is missing, expired, within **2 days** of its expiry, or superseded by a published key with a later date, the entry offers **Download and apply the new key**; **Check again** repeats the lookup.
- **Update MakeMKV key** (same menu) fetches the published key and applies it at once.
- A **banner on the video page** appears from **7 days before the installed key expires**, or when no key is installed, on boxes whose players use `libmmbd`. Tapping it opens the Blu-ray check.
- All of this is **Linux-only**. On FreeBSD the key entry and menu item are hidden and the key API is refused (section 17).

How it works (`video/webremote/src/lib/makemkv_key.py`):

- The page is read with **regular expressions**, so small changes of wording are tolerated: the key is the `T-...` string (preferably in the post's code box), the expiry the phrase after *valid until / till / through*. Accepted dates: *end of October 2026* or *October 2026* (the last day of that month), *October 31, 2026*, *31st of October 2026*, *2026-10-31*. A key found with an unreadable date is reported with the phrase quoted; a page without a key fails with a message.
- **Every lookup saves the expiry of the key it saw**, by key fingerprint, in `makemkv-key.json` in the web remote's cache directory (default `~/.cache/omdrc-video`). The installed key's expiry is then known without the network, which is what the banner reads.
- A later lookup that finds a **different key with a later date** means a new key is out, and the entry offers it. Once applied, its recorded date becomes the next expiry. A key installed by hand and never seen by a lookup has an unknown expiry; the published one is offered when it differs.
- **Applying** rewrites only the `app_Key` line of `settings.conf` (or creates the file), atomically and with mode `0600`, keeping the old file as `settings.conf.previous`.

The same can be done by hand: MakeMKV GUI **Help -> Register** as that user, or the `app_Key` line above. `makemkvcon -r info disc:9999`, as the same user, prints MakeMKV's version and registration state among its first messages.

MakeMKV releases also age out: an old build can refuse to run until it is updated, independently of the key. If a new key does not help, update the package (Arch: `makemkv` from the AUR). A purchased MakeMKV licence makes the registration permanent; updating a release that aged out still applies.

13.5 The Blu-ray check and KEYDB.cfg

Where the players use `libmmbd`, the web remote's **Blu-ray check** reports **MakeMKV (libmmbd)** as the decryption path, no longer requires `libaacs` and `libbdplus`, and shows `KEYDB.cfg` as **AACS keys (fallback)**: its problems are listed but do not count, and the KEYDB.cfg links and upload move into a collapsed *Fallback* section. On Linux without MakeMKV the check warns that decryption falls back to libaacs + `KEYDB.cfg`.

Source: `video/BLURAY-DECRYPTION.md`.

Part III — FreeBSD

Everything a FreeBSD (15.1) host needs beyond Part I: packages, rc.d and devd integration, stable sound-device roles, the OSS/`virtual_oss` audio stack, the panel's FreeBSD behaviour, video, the CD daemon, known issues, kernel patches and the port plan. **On Linux, skip Part III entirely**; nothing in Part I depends on it.

Chapter 14

FreeBSD: installation

Do the common build (chapter 3) with these FreeBSD specifics. Services and hotplug are in chapter 15.

14.1 Packages

Component	`pkg` package
libnpupnp, libupnpp	`curl libmicrohttpd expat2`
upmpdcli	`jsoncpp libmpdclient`
upmpdcli Qobuz plugin	`python3 py311-requests`
MPD	**`musicpd`**
BruteFIR	`fftw3 fftw3-float`
Loopback	`virtual_oss` (+ the `cuse` kernel module)
omdrc-ctrl	`py311-flask py311-Markdown py311-numpy`

Naming. MPD is `audio/musicpd`: the binary is `musicpd`, the service is `service musicpd ...` and the bundled client is `musicpc` (aliasing `mpc`). Service names, rc.d filenames, rc.conf keys and hook names use underscores (`omdrc_audio`, `omdrc_audio_enable`); standalone devd files use hyphens (`omdrc-audio.conf`). The separators are not interchangeable in `service`, `rcorder` or `PROVIDE/REQUIRE` tokens. Services are enabled with `sysrc <name>_enable=YES` and run manually with `service <name> onestart|onestop`.

```
pkg install bash brutefir virtual_oss musicpd mpc
sysrc kld_list+="cuse"
kldload cuse
```

The login audio user must be in the groups that grant access to sound and USB devices. BruteFIR must never run as root: an interactive `drc.sh` could not stop a root-owned instance.

14.2 Installing the project

Set the box values in `host.cmake`, then:

```
mkdir -p build && cd build
cmake .. -C ../host.cmake
make && sudo make install
make user-install          # as the audio user, after the system install
```

For a package layout use the FreeBSD port `freebsd/audio/open-media-drc`; the installed `omdrc_audio` points at `/usr/local/libexec/omdrc/drc.sh`, the run-from-repository script at this checkout.

Early-boot files must be regular copies in system paths, not symlinks into a possibly separate `/home`:

```
install -m 755 etc/rc.d/omdrc_audio /usr/local/etc/rc.d/omdrc_audio
install -m 644 etc/devd/omdrc-audio.conf /usr/local/etc/devd/omdrc-audio.conf
sh scripts/prepare-musicpd-rc-conf-dir.sh /usr/local/etc/rc.conf.d/musicpd
install -m 644 etc/rc.conf.d/musicpd/omdrc_audio \
  /usr/local/etc/rc.conf.d/musicpd/omdrc_audio
```

Copy the other enabled rc.d scripts from the inventory (section 15.1) the same way, and refresh the copies after every update. `rc.subr` accepts `rc.conf.d/musicpd` as a file *or* a directory; the helper makes the directory form, moving an existing file unchanged (mode preserved) to `musicpd/00-local.conf`, so an administrator's MPD settings survive. It is idempotent, and the Make and CMake installers and the package's `PRE-INSTALL` script all run it.

Remove obsolete lifecycle files once the new service is tested; they must not coexist with `omdrc_audio`: `/usr/local/etc/rc.d/{drc_usb_audio,brutefir_drc,omdrc_sndlink}`, `/usr/local/etc/devd/omdrc-sndlink.conf`, `/usr/local/libexec/omdrc-hotplug`.

14.2.1 BruteFIR defaults and MPD configuration

```
install -d -o AUDIO_USER -g AUDIO_GROUP /home/AUDIO_USER/.config/BruteFIR
install -m 644 etc/open-media-drc/brutefir_defaults.conf \
  /home/AUDIO_USER/.config/BruteFIR/brutefir_defaults.conf      # OSS I/O
```

Merge the three named outputs from `mpd/musicpd.conf.in`: `OKTO-DAC`, `DRC-native`, `DRC-resamp`. All FreeBSD physical output paths must use `/dev/dsp.dac`, never a numbered `/dev/dsp0` (section 15.2).

14.3 Configuring rc.conf

A core installation with the controller and renderer restore service:

```
musicpd_enable="YES"
musicpd_config="/home/giacomo/open-media-drc/mpd/musicpd.conf"

omdrc_audio_enable="YES"
omdrc_audio_user="giacomo"
omdrc_audio_dac="0x152a:0x88c5"       # strongly recommended with >1 card
omdrc_audio_capture="ESI U24XL"       # omit when CD input is unused
```

```
omdrc_audio_capture_recsrc="auto"

omdrc_renderer_enable="YES"
upmpdcli_enable="NO"
qobuzconnect2mpd_enable="NO"
qobuzconnect2mpd_user="giacomo"
qobuzconnect2mpd_group="giacomo"
qobuzconnect2mpd_homedir="/var/db/qobuzconnect2mpd"

omdrcctrl_enable="YES"
omdrcctrl_user="giacomo"
omdrcvideo_enable="YES"
omdrcvideo_user="giacomo"
```

Remove any numbered default-device assignment such as `hw.snd.default_unit=0` from `/etc/sysctl.conf`: its **value is a pcm unit number**, so only the role resolver can know it (section 15.2). Genuinely global sound tunables may stay there. For CD input also set `omdrc_cdin_enable="YES"` and `omdrc_cdin_user="giacomo"`; the knobs are in section 18.5.

Project key families:

Key family	Purpose
`musicpd_enable`, `musicpd_config`	MPD boot and configuration
`omdrc_audio_enable`, `omdrc_audio_user`, `omdrc_audio_drcsh`, `omdrc_audio_statussh`	master audio lifecycle and user boundary
`omdrc_audio_dac`, `omdrc_audio_capture`	stable card identities
`omdrc_audio_dac_sysctls`, `omdrc_audio_capture_sysctls`, `omdrc_audio_capture_recsrc`	per-role settings reapplied after every attach
`omdrc_audio_rundir`, `omdrc_audio_lockfile`, `omdrc_audio_statefile`	root boot-lifetime device transaction state
`omdrc_cdin_*`	optional CD bridge, fully listed in section 10
`omdrc_renderer_enable`, `omdrc_renderer_prefix`, `omdrc_renderer_script`, `omdrc_renderer_statedir`	restore the last selected renderer
`upmpdcli_enable`, `upmpdcli_user`, `upmpdcli_homedir`, `upmpdcli_config`, `upmpdcli_pidfile`, `upmpdcli_logfile`, `upmpdcli_flags`	UPnP renderer worker
`omdrcctrl_enable`, `omdrcctrl_user`, `omdrcctrl_env`, `omdrcctrl_pidfile`, `omdrcctrl_logfile`	web controller
`omdrcvideo_enable`, `omdrcvideo_user`, `omdrcvideo_env`, `omdrcvideo_pidfile`, `omdrcvideo_logfile`	video web remote

The old `drc_usb_audio_*`, `brutefir_drc_*` and `omdrc_sndlink_*` families are accepted by `omdrc_audio` only as one-release migration fallbacks: copy their values to the new keys and remove them. Enabling an old copied script creates a second lifecycle owner and is unsupported.

14.4 Validate ordering and activate

```
rcorder /etc/rc.d/* /usr/local/etc/rc.d/* | \
  egrep 'devd$|omdrc_audio$|musicpd$|omdrc_cdin$|omdrc_renderer$'
service devd restart
service omdrc_audio roles
service omdrc_audio status
service omdrc_audio reconcile
sysctl hw.snd.default_unit       # must equal the pcm unit reported as dac
mpc outputs                      # desired output enabled after musicpd starts
```

`omdrc_audio` requires `devd`, so the cold-plug scan runs after devd is listening; a card that finishes attaching later produces a pcm event. It deliberately does not require MPD: a slow MPD must not stop the physical DRC chain from becoming healthy.

14.5 Network: keep DHCP on every interface

Do not give an Ethernet port a static address in `rc.conf` on a box that is sometimes wired and sometimes wireless. libupnpp chooses its interface once, at startup: the first that is UP+RUNNING+MULTICAST *and has an address*. `em(4)` keeps `RUNNING` set with no carrier, so a statically configured wired port stays fully qualified with no cable in it:

```
ifconfig_em0="inet 192.168.1.9 netmask 255.255.255.0"   # applied regardless
em0: flags=8843<UP,BROADCAST,RUNNING,...>  status: no carrier
```

upmpdcli then binds the dead port and its SSDP advertisements never leave the host. The failure is quiet: upmpdcli **starts, connects to MPD and keeps driving it**, so `ps` and the panel's renderer switch report it healthy; only the control points stop listing it. A box whose wired port *used to be* the live one breaks the same way without being touched. The tell is one line in `/tmp/upmpdcli.log`, sometimes followed by an intermittent `UPNP_E_INVALID_HANDLE` / `Device would not start`:

```
LibUPnP: Using IPV4 192.168.1.9 port 49152     <- not the address you serve on
```

The fix is in `rc.conf`, not `upmpdcli.conf`:

```
ifconfig_em0="DHCP"       # NOT "inet 1.2.3.4 ..."
```

An unplugged DHCP interface gets no lease and so no address; libupnpp skips it and picks the connected one with `upnpiface` unset, and plugging the cable back in works because FreeBSD's `/etc/devd/dhclient.conf` starts `dhclient` on `LINK_UP`. Pinning `upnpiface` is the wrong tool — the pinned value goes stale the moment the box changes network — and stays a last resort.

14.5.1 Leave `defaultrouter` unset when any interface uses DHCP

`rc.d/routing` installs `defaultrouter` **unconditionally at boot**, so with no cable the default route still points at the wired gateway and Wi-Fi's DHCP-supplied route is overridden. With both

interfaces on DHCP the connected one supplies the route itself:

```
# /etc/rc.conf
ifconfig_em0="DHCP"
wlans_iwm0="wlan0"
ifconfig_wlan0="WPA  DHCP"
#defaultrouter="192.168.1.1"      # leave unset; DHCP provides it
```

`synchronous_dhclient` defaults to `NO`, so an unplugged DHCP interface does not delay boot.

Chapter 15

FreeBSD: services, device roles and lifecycle

This chapter is the reference for every FreeBSD init and devd artifact the repository owns. Port templates and their rendered copies are one logical script, so each appears once.

15.1 Service inventory

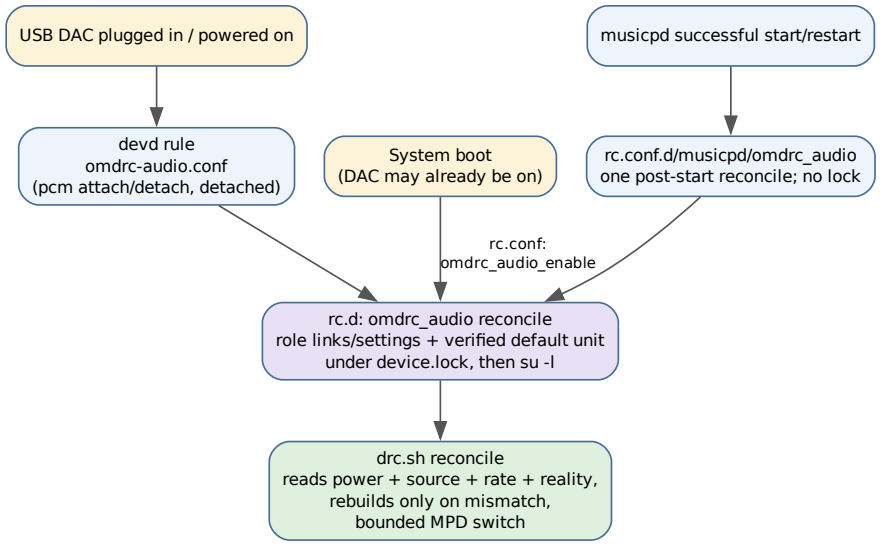


Figure 15.1: FreeBSD hotplug: devd fires on the `pcm` device and a level-triggered reconcile follows; a successful `musicpd` start triggers one late reconcile.

Script/configuration	rcorder relation or event	Function	Enable directly?
`musicpd`	`REQUIRE: mixer LOGIN avahi_daemon`	Starts MPD with the repository's FreeBSD configuration	Yes
`rc.conf.d/musicpd/omdrc_audio`	`rc_cmd musicpd start_postcmd`	Issues one bounded audio reconcile after MPD is actually available	No; sourced by `musicpd`
`omdrc_audio`	`REQUIRE: FILESYSTEMS devd; shutdown`	Single owner of card roles and DRC lifecycle	Yes
`omdrc_cdin`	`REQUIRE: omdrc_audio; shutdown`	Optional continuous S/PDIF capture bridge	Yes, only with CD input
`omdrc_renderer`	`REQUIRE: NETWORKING FILESYSTEMS musicpd; shutdown`	Restores whichever renderer the UI last selected	Yes
`upmpdcli`	`REQUIRE: NETWORKING FILESYSTEMS musicpd; shutdown`	UPnP/OpenHome worker controlled by `omdrc_renderer`	No when renderer restore is used
`omdrcctrl`	`REQUIRE: NETWORKING LOGIN; shutdown`	Starts the web controller as the audio user	Yes when installed
`omdrcvideo`	`REQUIRE: NETWORKING LOGIN; shutdown`	Starts the video web remote; it does not start mpv	Yes when installed
`omdrc-audio.conf`	devd `pcm[0-9]+` attach and detach	Detaches one level-triggered `omdrc_audio reconcile` request	Installed in devd; no rcvar

15.1.1 musicpd

The project `musicpd` script selects the rendered `musicpd_config`, has rc.subr derive the pidfile from it, and launches the FreeBSD `musicpd` binary; MPD drops to the user/group declared in its own configuration. It owns no DRC transition and no project lock. The dependency token is `musicpd`: both `omdrc_renderer` and `upmpdcli` require the name this script actually provides.

15.1.2 rc.conf.d/musicpd/omdrc_audio

Not another daemon: a service-specific `rc.subr` fragment that sets `musicpd`'s `start_postcmd` to a function running only after MPD started successfully:

```
omdrc_musicpd_poststart()
{
```

```
    checkyesno omdrc_audio_enable 2>/dev/null || return 0
    /usr/sbin/service omdrc_audio reconcile ||
        warn "musicpd: omdrc_audio reconcile failed; retry it manually"
    return 0
}
```

`omdrc_audio` normally runs before `musicpd` and deliberately does not wait for it, so the physical chain can be healthy while its bounded MPD output selection is still `pending`. A successful MPD start is the earliest factual readiness signal, so the hook retries once at that event instead of using a delay, endless polling or a readiness gate. It runs after a manual `service musicpd restart` too, when MPD may have forgotten its outputs. The edge is strictly one-way:

```
musicpd successful start
  -> service omdrc_audio reconcile
  -> roles transaction (device.lock, then release)
  -> drc.sh reconcile (drc.lock, bounded mpc)
```

No `omdrc_audio` path starts `musicpd`, so there is no service cycle. The hook takes no lock and creates no process; a reconcile failure is reported but the hook still returns success, since MPD itself is running; with `omdrc_audio_enable` off it is a no-op. The FreeBSD CMake branch, the direct Make target and the package all install it.

15.1.3 omdrc_audio

`omdrc_audio` replaced the former three-service chain; `omdrc_audio_enable` is the only master switch. Verbs:

Verb	Meaning
`start`	boot cold-plug role pass, then full reconcile
`roles`	root-only role links/settings transaction, no DRC transition
`reconcile`	role transaction, release device lock, then user DRC reconcile
`stop`	transient teardown; preserve desired power, rate, design, and source
`status`	show role resolution and actual chain status

It keeps `su -l` from the old `brutefir_drc` (correct HOME and login PATH, and BruteFIR owned by the user who runs interactive commands) and the master rcvar and boot/hotplug entry of `drc_usb_audio`. The unreliable `/var/run/drc_usb_audio.active` marker is gone: actual processes, config paths, rates, nodes, role links and saved intent are authoritative.

Never re-enter an rc.d script through `$0`. `/etc/rc` *sources* each script, so during boot `$0` is `/etc/rc`; the old `omdrc_sndlink` re-entered itself under `lockf` with `$0` and so ran `/bin/sh /etc/rc oneupdate`, a second complete rc pass that duplicated network and service startup and destabilised the boot. `omdrc_audio` re-enters for its short locked `roles` step with `/bin/sh "$rc_service" oneroles` (rc.subr sets `rc_service` to the absolute script path) and falls back to the script path only outside rc.

15.1.4 omdrc_cdin, omdrc_renderer, upmpdcli

- `omdrc_cdin` runs as the audio user. Before replacing `virtual_oss`, `drc.sh` stops the bridge and waits to a fixed deadline for its *process* to exit — process exit is the release acknowledgement, not a logfile that may be rotated. Failure aborts CUSE teardown and tries to restore MPD's direct output. After a successful transition a bridge that was running is restarted with `onestart`, so a panel-started instance survives even with the rcvar off. The `release` extra command only sends `SIGHUP` for diagnostics.
- `omdrc_renderer` reads `last_renderer` and starts exactly one renderer, upmpdcli or qobuz-connect2mpd, via `onestart`/`onestop`; it keeps the selection at shutdown. Both worker rcvars stay `NO`: enabling one independently races the owner and may leave two front-ends driving MPD.
- `upmpdcli` is a worker: it supplies the audio user's HOME and a PATH containing `/usr/local/bin`, creates the user-owned pid directory and captures plugin stderr for the panel's log view. Its `REQUIRE` token is `musicpd`, not `mpd`.

15.1.5 omdrcctrl and omdrcvideo

Both run as the configured non-root user through `daemon(8)`; rc.subr drops privileges via `${name}_user`, and a `start_precmd` creates user-writable pid/log directories (a plain `/var/run/*.pid` would be root-only). The environment sets HOME, PATH, DISPLAY and optionally the shared `OMDRC_STATE_DIR`. `omdrcctrl` reads `/var/run/omdrc/audio.roles` without spawning a status command. `omdrcvideo` starts only the HTTP/API process; the persistent idle mpv belongs to the graphical login session.

Their identity does not depend on the caller: `/var/run/omdrcctrl/omdrcctrl.pid` (and `/var/run/omdrcvideo/omdrcvideo.pid`) always names the `daemon(8)` supervisor, never a `TMPDIR` path. An earlier non-root branch chose `${TMPDIR:-/tmp}/omdrcctrl-USER.pid`, so an ordinary status probe reported the root-started service as stopped and `onestart` could start a second instance; it was removed. `daemon(8)` creates the pidfile and the log 0600, and its `-M` mode option is not present in every base system – on an older one the service fails to start with `daemon: invalid option -- M`. So `start_precmd` pre-creates both 0644 (only when missing, to keep the log's history), which leaves the PID readable for diagnostics on every release. Use `sudo service ... start|stop|restart` as the system interface; a development process needs a distinct port and direct launcher.

15.1.6 omdrc-audio.conf (devd)

The only project devd rule matches the `pcm` device at attach and detach:

```
attach 100 {
    device-name "pcm[0-9]+";
    action "/usr/sbin/daemon -f /usr/sbin/service omdrc_audio reconcile";
};
```

The detach rule has the identical action. Matching `pcm` is a safety boundary: a UAC2 device exposes several USB interfaces but one sound card, and matching USB class events ran several lifecycle runs per plug (a broad USB detach rule would also react to a keyboard or a disk). Matching the USB device instead fires *before* its `pcm` child exists and would need a retry loop. At the `pcm` event the kernel has already allocated the unit and created its OSS nodes, so role resolution needs no settle sleep.

15.2 Stable device roles

`pcm` units are handed out in attach order, and USB attach order is port order. On the reference box the ESI U24 XL sits on a lower root-hub port than the DAC and wins the race at every boot, so a chain that addressed the DAC by unit played into the S/PDIF interface while `cdin` captured from the DAC.

There is no declarative way to pin the number. A unit hint (`hint.pcm.1.at="uaudio0"`) needs `BUS_HINT_DEVICE_UNIT`, which only `acpi(4)`, `pci(4)` and `isa(4)` implement; `uaudio(4)` ignores it silently, and `devclass_alloc_unit()` even skips any unit carrying an `at` hint, so hinting `pcm0` would take unit 0 from the DAC. `hw.snd.default_unit` only selects among existing units, and `devd` sees an event after the kernel has already acted and has no `NAME=`/`SYMLINK=`. So the number is not used. `devfs` accepts symlinks, and `omdrc_audio` maintains two pairs by **role**:

link	is	created
`/dev/dsp.dac`, `/dev/mixer.dac`	the DAC everything plays to	always
`/dev/dsp.capture`, `/dev/mixer.capture`	the CD/S-PDIF input	only when a capture card is named

Everything opens the DAC by name: BruteFIR's output, MPD's `OKTO-DAC` output, `cdin --out`, the mpv launchers, `verify-bitperfect.sh`. A one-card box needs no configuration; a two-card box stops caring which enumerated first. Links use relative targets (`/dev/dsp.dac -> dspN`); a pre-existing non-symlink at a role name is never overwritten, and a detach pass removes only project-owned links whose role is now unfilled. Role publication is an atomic rename to `/var/run/omdrc/audio.roles`.

```
sysrc omdrc_audio_enable=YES
sysrc omdrc_audio_capture="ESI U24XL"   # only if you use the CD input
service omdrc_audio status              # prints the roles, exits 1 if unfilled
```

Roles are decided by identity, never by number. Capability cannot tell a DAC from a capture interface (an OKTO DAC8 reports `play/rec` like one), so an explicit match always wins, as a USB id or a substring of the `/dev/sndstat` description:

```
sysrc omdrc_audio_dac="0x152a:0x88c5"          # vendor:product
sysrc omdrc_audio_dac="0x152a:0x88c5:000483"   # ...:serial, for two identical DACs
sysrc omdrc_audio_dac="OKTO RESEARCH"          # or just the name
```

With no match configured, playback-capable cards are ranked: a pure-playback USB DAC beats a USB play/record interface, which beats non-USB playback. If several candidates remain the service says it guessed and prints the `omdrc_audio_dac` lines that would pin the intended card. IDs come from `dev.pcm.N.%parent` -> `dev.uaudio.N.%pnpinfo`.

Two triggers, one code path: the rc.d service does the cold-plug pass, and the devd rule handles later attach/detach. Both run the same complete level reconcile, so one rule covers both roles, detach, a moved card and coalesced or reordered events.

The one thing a symlink cannot cover is a sysctl OID. `dev.pcm.<unit>.*` is keyed by the number being avoided, so the unit is read back off the link:

```
t=$(readlink /dev/dsp.dac)                    # -> "dsp0"
sysctl -n "dev.pcm.${t#dsp}.feedback_rate"
```

`drc.sh` has this as `dac_unit()`, the panel as `_dac_unit()`, and `glitch-usbtap.sh` uses it to find the DAC's `uaudio` parent. Everything that does not need the number uses `dac_dev()`: `/dev/dsp.dac` when the link exists, else `/dev/dsp0`.

Per-role `pcm` settings ride with the service:

```
omdrc_audio_dac_sysctls="bitperfect=1 play.vchans=0"
omdrc_audio_capture_sysctls="bitperfect=1 rec.vchans=0"
```

`/etc/sysctl.conf` is the wrong home: `dev.pcm.<unit>.*` is keyed by the unstable number, `/etc/rc.d/sysctl` runs long before anything knows which card is which, and a re-attach re-creates the whole `dev.pcm.<unit>.*` tree from driver defaults — which is why they are reapplied on every attach, so a replugged DAC has `bitperfect=1` again before BruteFIR reopens it. Global `hw.snd.*` and `hw.usb.uaudio.*` tunables survive a re-attach and stay in `/etc/sysctl.conf`.

`hw.snd.default_unit` is the deliberate exception. The role transaction sets it to the DAC's unit with the absolute `/sbin/sysctl`, reads it back, logs a change or failure, and makes `service omdrc_audio status` fail when the readback differs from the resolved DAC. It matters to applications outside the project that open bare `/dev/dsp`: on a two-card box, leaving the default at pcm0 can route an unrelated application into the ESI capture interface while BruteFIR correctly holds the DAC. Project components keep using `/dev/dsp.dac`. Never put a literal value in `/etc/sysctl.conf`: it runs near the start of rc, before USB attach order is known, and would create a second owner encoding yesterday's enumeration.

Apply and the known-device list. On FreeBSD the web page's Apply updates the two `omdrc_audio_*` role keys, reconciles `omdrc_audio` and verifies that the resulting `/dev/dspX` nodes exist. The known-device list of section 6.4.1 is the comma-separated `omdrc_audio_dac` / `omdrc_audio_capture` value in `/etc/rc.conf` (`"0x22e8:0xdac4,0x152a:0x88c5"`), reconciled by `audio_pick` in `omdrc_audio`; the automatic ranking above applies only when `omdrc_audio_dac` is empty.

15.3 What devd serializes — and what it does not

FreeBSD 15.1's `devd` (`sbin/devd/devd.cc::my_system()`) forks `/bin/sh -c command` for a direct action and waits for that child. A synchronous `action "/usr/sbin/service omdrc_audio reconcile"` would therefore block devd until the whole reconcile returned, and two events would queue rather than overlap. That is an implementation fact, not an API promise: `devd.conf(5)` does not guarantee it.

The installed action changes the lifetime on purpose. `daemon(8)` detaches the worker (its `-f` means *close inherited descriptors*, not "foreground"), so devd and the shell wait only for the short launcher:

```
devd -> sh -c "daemon -f service omdrc_audio reconcile"
          -> daemon launcher -> detached service omdrc_audio reconcile
```

devd thus serializes the launchers but not the workers, and two reconciles can overlap. That is intended: a reconcile may wait on locks, MPD, BruteFIR, virtual_oss/CUSE and hardware

verification, and running it inline would freeze the machine-wide event loop (USB, network, input, storage, ACPI).

The device lock makes the overlap safe because it is taken *before* a worker scans pcm state, so a waiter never publishes a snapshot older than its wait:

```
A takes device.lock; scans ESI only
OKTO attaches; B starts and waits
A publishes capture-only state; releases device.lock
B takes device.lock; scans ESI + OKTO; publishes both roles and links
```

The detach case is symmetric: without the lock an old pre-detach scan could recreate `/dev/dsp.dac` after a newer worker removed it. Every request rebuilds complete level state; no attach or detach edge is read as an instruction to start or stop DRC.

A **rejecting singleton** (a pidfile wrapper that exits when a worker exists) would be wrong: A scans with the DAC present, the DAC detaches, B's launcher runs but is refused, and A publishes its stale "present". A coalescing singleton would need a dirty flag and a rescan protocol; the lock simply queues the waiter, so the second pass scans fresh.

References: FreeBSD 15.1 devd.cc, devd(8), devd.conf(5), daemon(8), lockf(1).

15.4 Locking and bounded waits

Two locks remain, with a strict non-nesting rule:

Lock	Owner	Protects	Lifetime
`/var/run/omdrc/device.lock`	`omdrc_audio roles`	role discovery, four links, sysctls, recsrc, role publication	one short role transaction
`STATE_DIR/drc.lock`	audio-user `drc.sh`	saved intent reads/writes and the physical chain transition	one mutating DRC command

Both use `lockf -k -s -t ...`; `-k` keeps one inode after release, which `lockf(1)` recommends for concurrent callers, and file existence does not mean locked. `/var/run` suits root boot-lifetime state; the DRC lock lives beside persistent user state and is never derived from `TMPDIR`, so a desktop session and a boot login shell cannot choose different locks. The lock is released when the orchestration command exits; BruteFIR and virtual_oss never become owners. The order is:

```
devd or rc
  -> omdrc_audio roles       [take device.lock; update facts; release]
  -> su -l AUDIO_USER
  -> drc.sh reconcile        [take drc.lock; converge; release]

musicpd successful start
  -> the same omdrc_audio reconcile path (the hook owns no lock)
```

No path takes `drc.lock` and then asks for `device.lock`, and `omdrc_audio` releases `device.lock` before entering the user reconciler. Several simultaneous pcm events wait for the short role transaction,

then for `drc.lock`; the first repairs state and later calls become no-ops.

"devd serializes, so delete the locks" is wrong: `daemon -f` deliberately releases devd before the worker finishes; boot rc, the MPD hook, administrators, the UI and direct `drc.sh` calls are not serialized by devd; a rejecting singleton can drop a requested reconcile; and the manuals do not promise devd's current `wait4()` behaviour. Merging the two locks would hold a short root transaction across the long audio-user transition. Two non-nested locks are the simplest safe design.

Every external wait taken under `drc.lock` is bounded. Each `mpc` call goes through a timeout wrapper, so a slow or absent MPD blocks neither boot nor a verified physical chain; the pending selection is logged and the MPD-start hook retries it. BruteFIR startup and exit, virtual_oss readiness, DAC warm-up, verification and CD release all use explicit poll caps, and service calls that restart the CD bridge have a deadline and use non-interactive sudo.

Syslog is evidence, not state: `omdrc_audio` publishes role state directly and `drc.sh` appends to the persistent `STATE_DIR/drc.log`; neither decides from syslog. (During one incident syslogd held a bound but unlinked `/var/run/log`, so new `logger` calls failed silently.) Repair syslogd before using log absence as evidence.

Chapter 16

FreeBSD: the OSS audio stack, panel and diagnostics

16.1 OSS, virtual_oss and cuse

FreeBSD's native audio API is OSS. The loopback is **`virtual_oss`**, a userland OSS mixing/routing daemon from the base system that creates character devices through the **`cuse(3)`** kernel facility. `drc.sh` starts it per rate with a play node `/dev/dsp.play` (MPD writes) and a synchronized loopback node `/dev/dsp.loop` (BruteFIR reads; the `-L` loopback). The `cuse` module must be loaded (`kld_list`). BruteFIR's OSS I/O is built in; the fork's OSS fixes matter here.

Key sysctls for the bit-perfect direct path:

```
bitperfect=1     # first opener's format becomes the hardware format
play.vchans=0    # no virtual-channel mixer/resampler
```

They are applied to the DAC's unit by `omdrc_audio` on every attach (`omdrc_audio_dac_sysctls`, section 15.2). They also make the DAC **single-open**: exactly one client at a time — BruteFIR when DRC is on, otherwise MPD's direct output or a browser.

16.2 DAC priming

The OKTO DAC routes silence on the *first* stream opened at a new sample rate; a second open fixes it. On a detected rate change `drc.sh` therefore opens BruteFIR once, tears it down, then starts it for real. The kernel-level fix is the clock-before-alt patch (section 20.1); with it installed, `DAC_PRIME_CYCLES` defaults to 0.

16.3 The panel on FreeBSD

- **Service**: `sysrc omdrcctrl_enable=YES && service omdrcctrl start`. The rc.d details (`daemon(8)`, pidfile, privilege drop) are in section 15.1.
- **DAC feed** in the MPD panel is the `virtual_oss` rate.
- **Audio Devices** card: `/dev/sndstat` with `fmt 0x...` bitfields decoded to `AFMT_*`/`PCM_CAP_*` labels (collapsible). **Advanced** card: `sysctl dev.pcm.<DAC unit>` (resolved from `/dev/dsp.dac`) and `sysctl hw.usb.uaudio` diagnostics.

- **Renderer switch** runs `sudo service ... onestart/onestop`.
- **Spectrum from the CD**: `omdrc-cdin` tees the samples itself from the period it is about to write, into a FIFO opened non-blocking that drops rather than ever delaying a write to the DAC. The reader's presence is the whole protocol, so nothing happens until the panel opens the FIFO.
- **Glitch Debug card**: section 16.4.

The spectrum **meter-timing hold-back** on FreeBSD counts the filter's peak index divided by the sample rate and one BruteFIR `filter_length` partition, minus the margin. The `virtual_oss` rings (`-s 200ms` blocks, constant across rates), BruteFIR's I/O buffers and the OSS/DAC/USB output are not estimated: they are left to each screen's calibrated delay, and the DAC buffer with DRC off is not read (that term is Linux-only), so with the chain down the box holds back nothing and the screen's delay covers the OSS buffer. The built-in `dirac pulse` has zero group delay but still pays the convolver partition. Hold-back is anchored to elapsed time rather than to the last byte received, so the read point drains the buffered tail when a writer stops and non-blocking CD-FIFO drops cannot walk the display out of sync; after a silence gap retained PCM is discarded before the source resumes.

16.4 Glitch detection

One global switch — `glitch-debug.sh on|off|status|analyze|usbtap|tail|clear` — also exposed as the Debug card in omdrc-ctrl.

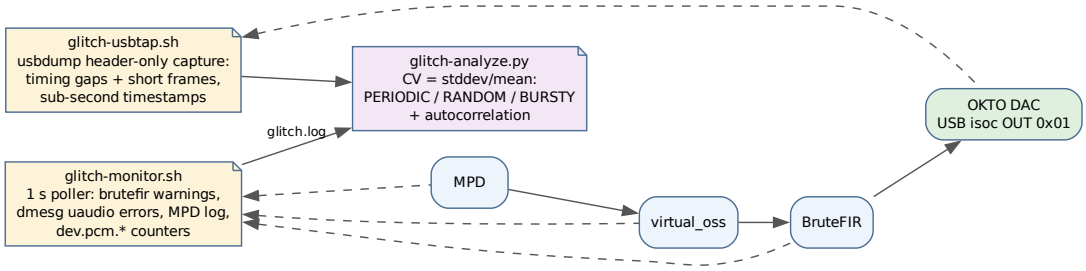


Figure 16.1: The glitch-detection layers and where each taps the chain.

- **`glitch-monitor.sh`** (always-on, lightweight): polls every second and logs new anomalies from four sources — BruteFIR warnings (missed real-time deadlines), kernel `uaudio`/USB errors, the MPD log, and any increasing `dev.pcm.*` under/over/err/xrun counter — into a unified `glitch.log`.
- **`glitch-usbtap.sh`** (definitive, heavier, CLI-only): taps the OKTO's isochronous OUT endpoint 0x01 with `usbdump`, downstream of every software stage. Header-only analysis scales to multi-minute captures; it flags **timing gaps** ($>$ 2.5x the nominal ~4 ms interval) and **short frames** (SLEN $<$ 0.5x nominal), while the constant +-few-samples feedback wobble of asynchronous USB is counted separately and never flagged. Blind spot: a full-length block of zeros (silence insertion) needs payload inspection — use `verify-bitperfect.sh` for that.
- **`glitch-analyze.py`**: classifies inter-event intervals per stage by the coefficient of variation — CV ~ 0 **PERIODIC** (a buffer/clock cycle), CV ~ 1 **RANDOM/Poisson** (CPU/scheduling),

CV > 1.5 **BURSTY** (something waking up) — plus autocorrelation and correlation against DRC rate switches in `drc.log`.

16.5 Verifying bit-perfect on FreeBSD

`scripts/verify-bitperfect.sh` *proves* the DAC receives bytes unchanged. Two levels:

1. **Structural** (kernel-certified): with `hw.snd.verbose=2`, `/dev/sndstat` must show the play channel `BITPERFECT` with the feeder graph exactly `{userland} -> feeder_root -> {hardware}` — any `feeder_rate`/`feeder_volume`/format node means the kernel is altering bytes. Preconditions: `bitperfect=1` and `play.vchans=0` on the DAC's unit — which is what `omdrc_audio_dac_sysctls` asserts on every attach.
2. **Empirical wire tap** (gold standard): play a deterministic test signal (near-silent ~-90 dBFS per-sample counter in the low 16 bits, distinct L/R — maximally sensitive to truncation, dither, volume, resampling, channel swap) while capturing the USB isochronous OUT endpoint 0x01 with `usbdump`; decode, align, and byte-compare. The embedded OSS writer aborts loudly if the kernel coerces format/channels/rate. Verified on this host at 44.1/48/88.2 kHz: hundreds of kB contiguous identical bytes, and on the live MPD both direct (**BIT-PERFECT**) and through `virtual_oss` (**VALUE-EXACT**, 0 slips).

The subtle part is clock domains: a producer must be **flow-controlled by the sink's clock** (blocked writes). MPD is; a free-running test writer is not, and drifts. `virtual_oss` itself is bit-transparent with a flow-controlled producer. The one caveat: BruteFIR bridges the loopback's software clock to the DAC crystal without resampling, so an inaudible one-sample slip occurs every several minutes on the DRC path — values are never altered. The DRC path is *intentionally* not byte-equal (that is the correction); to test its plumbing, use a unit-impulse filter with attenuation 0.

`bitperfect-tap-freebsd.sh` resolves the DAC's `uaudio` unit from the play device (`/dev/dsp.dac` to `pcm<N>.%parent`), as `glitch-usbtap.sh` does. It once tapped `dev.uaudio.0`, which on the reference box is the ESI U24 XL (the DAC8 is `uaudio1`), and reported `NO CAPTURE` against a healthy chain.

Measured through the panel on the OKTO DAC8 (FreeBSD 15.1-RELEASE-p2, `usbus0` devaddr 3):

Path	Material	Verdict
`aplay`	44100 / 32-bit	**BIT-PERFECT** — tap WAV file hash identical to the input
`mpd`	44100 / 32-bit	**BIT-PERFECT**
`mpd-http`	44100 / 32-bit	**BIT-PERFECT**
`mpd-http`	96000 / 24-bit FLAC	**BIT-PERFECT**, DAC clock followed (`feedback_rate` 96002)
`upnp`	44100 / 32-bit, driven through upmpdcli over OpenHome	**BIT-PERFECT**

The 96 kHz FLAC run carries the most information: it proves the FLAC decode is transparent *and* that the reference's 24-to-32 promotion matches MPD's own, the one place the two could have disagreed. The `upnp` run certifies the renderer itself: the whole upmpdcli-to-MPD-to-DAC path with upmpdcli choosing what MPD plays.

16.6 Browser audio on FreeBSD

The No DRC launchers (section 4.6) stop DRC so that a browser can play straight to the DAC. On Linux this is ordinary ALSA (section 11.6). On FreeBSD it is not simple, and Chromium in particular **does not play at all** on this box unless the launcher steps in. This section explains why, what the launcher does about it, and how to check it. The implementation is `browser-nodrc/lib.sh`, `browser-nodrc/chromium-nodrc.sh`, `browser-nodrc/chrome-nodrc.sh` and `cmake/browser-audio.cmake`.

In short:

Browser	Audio path on FreeBSD during a No DRC session
Firefox	its own OSS backend, straight to `/dev/dsp` (the DAC)
Chromium, Chrome	Chromium's sndio backend -> a `sndiod` started by the launcher for this session -> `rsnd/<unit>` (the DAC)
(not used)	PulseAudio and PipeWire: deliberately absent; the installer prevents PulseAudio from starting

16.6.1 Why Chromium is silent on this box without help

FreeBSD's native audio API is OSS (`/dev/dspN`). MPD, BruteFIR, the CD bridge and Firefox (through its cubeb OSS backend) all use it directly.

Chromium has no OSS backend at all. The FreeBSD Chromium port (installed as `chrome`; there is no `chromium` command) can produce sound through only three back ends, which it probes in this order:

1. **PulseAudio**, the sound server;
2. **sndio**, the small OpenBSD sound server (package `audio/sndio`);
3. **ALSA**, which on FreeBSD is only a userland library (`alsa-lib`) with no kernel part: its single PCM plugin, `libasound_module_pcm_oss.so`, opens `/dev/dspN` again.

On an ordinary FreeBSD desktop, Chromium plays through **PulseAudio**, or through PipeWire's PulseAudio-compatible server. A desktop session (KDE Plasma, for example) starts that server, and Chromium finds it first. That is why "Chromium needs PulseAudio or PipeWire on FreeBSD" is the usual advice, and why Chromium works on a normal desktop and is silent here.

This box cannot run PulseAudio or PipeWire. The DRC chain is bit-perfect: `omdrc_audio` sets `bitperfect=1` and `play.vchans=0` on the DAC's `pcm` unit, which removes the kernel's mixer and virtual channels (section "OSS, virtual_oss and cuse" above, and section 15.2), so **the DAC can be opened by only one process at a time**. A sound server is a long-lived daemon that opens the default unit (`hw.snd.default_unit`, which is the DAC) and keeps it:

- while DRC is on, BruteFIR could no longer open the DAC, and music would stop;
- during a No DRC session it would still hold the DAC when the browser exits, so the launcher could not restore DRC;
- it resamples and mixes everything that passes through it, and nothing in this project starts or stops it.

PipeWire has the same problem: it is the same kind of daemon holding the same device.

Without a sound server, the other back ends fail on their own:

- **ALSA.** With the stock configuration, Chromium's `default` PCM is `plug:oss`, i.e. whatever `/dev/dsp` is, at **a rate Chromium picks**. With no virtual channels, opening the device at a new rate reprograms the DAC's clock, and the OKTO routes silence on a cold open after a rate change (section 16.2). The ALSA backend is also not chosen by the probe: it has to be selected per profile in `chrome://flags`.
- **Single open.** Worse, Chromium opens and closes the output around each playback. A second tab, a page sound, or simply the end of one stream overlapping the start of the next needs **a second, concurrent open** of the DAC. There is none: the open fails with `PcmOpen: <dev>,Device busy`, **Chromium never retries**, and the browser is silent for the rest of the session, with no message to the user.

Any working Chromium path on this box therefore needs something that **mixes** Chromium's streams into one, **holds the DAC only while the browser runs**, and **does not change the DAC's rate**.

16.6.2 The launcher's answer: a private sndiod for the browser session

`sndiod` is that something. It is a small server that opens the device once, mixes all its clients into one stream and converts rate and format. It does not start itself, so the launcher can start it and stop it. Chromium already has a sndio backend, and when there is no PulseAudio its probe reaches sndio on its own: **no command-line flag and no `chrome://flags` setting is needed.**

The No DRC launcher for Chromium (and Chrome) does the following:

1. Saves the DRC state (`omdrc session`) and arms a trap that restores it on any exit, including a crash or a kill (section 4.6).
2. Runs `omdrc off`: BruteFIR stops and the DAC is free.
3. **Reads the DAC's current rate** from `dev.pcm.<unit>.feedback_rate` and snaps it to the nearest standard rate (the USB feedback figure reads, for example, 44101 for 44100). If it cannot be read, it falls back to `BROWSER_ALSA_RATE` (default 48000).
4. **Starts sndiod** on the DAC:

   ```
   sndiod -r <current rate> -f rsnd/<unit> -m play -s default
   ```

   - `-r <current rate>` keeps the DAC at the rate it is already running, so its clock is never switched and there is no cold-open silence; sndiod converts Chromium's 44.1 or 48 kHz in software. This is lossy streaming material outside the bit-perfect paths, so the conversion costs nothing that matters.
   - `-f rsnd/<unit>` names the DAC by unit number (`rsnd/3` is `/dev/dsp3`), taken from the `/dev/dsp.dac` role link, so the right card is used on a box with several cards.
   - `-m play -s default` makes the server **playback only**. sndiod's default is play+record, which would also open a duplex USB DAC for capture; on FreeBSD's `uaudio` that lets the capture stream replace the DAC's own feedback endpoint as the playback clock, and wastes USB bandwidth. The order matters: per-device options come before `-f`, per-sub-device options before `-s`.

5. Waits until nothing holds the DAC (`fuser`, up to `DAC_FREE_WAIT` seconds, default 8), so Chromium's first open cannot lose a race with a previous browser that is still shutting down, or with MPD's direct output. If the DAC is still held, it names the process that holds it.
6. Runs Chromium in the foreground. Every tab and page sound now goes to sndiod, which mixes them; no "device busy" can happen. (Tested with two tabs playing at once: no errors.)
7. When Chromium exits, it **stops the sndiod it started** and waits for it to release the DAC, and only then restores DRC, because BruteFIR has to open the DAC itself.

Details and limits:

- **Only an instance the launcher started is stopped.** If a `sndiod` is already running (started by someone else, or as a system service), it is reused and left running. Its settings are then not the ones above. By default sndiod releases the device when it has no clients, so a system instance does not block DRC between browser sessions. Enabling it as a service is still not recommended on this box.
- `chrome://flags` -> ***Audio Backend.*** A choice made there overrides the probe. A profile set to ALSA, for example by the older ALSA route below, bypasses sndiod, opens the raw DAC that sndiod holds, gets `EBUSY` and is silent. The setting is stored per profile and cannot be changed from outside the browser. The launcher warns when it finds one in `~/.config/chromium/Local State` (`~/.config/google-chrome` for Chrome). Set it back to *Default* and relaunch.
- **An already-running Chromium.** A second `chrome` command passes its URL to the running browser and exits at once. The launcher therefore leaves DRC alone and only opens the URL when Chromium is already running. Quit Chromium completely before using its No DRC entry.
- **Missing `sndiod`.** Without `audio/sndio` the launcher says so and starts the browser anyway, without sound. Install it with `pkg install sndio`.

16.6.3 The PulseAudio guard

Removing PulseAudio is not practical: the package is pulled in by the desktop (`plasma6-plasma-pa`, `plasma6-kinfocenter`, `qt6-multimedia`, `speech-dispatcher`), and two of its defaults start a server **on the DAC**:

- its **XDG autostart entry**, which starts a server with every desktop login;
- **`autospawn = yes`** in `client.conf`, which lets *any* program that uses the PulseAudio library start a server when it needs one. That includes Chromium's backend probe: without this guard, the **first launch of Chromium would itself put a PulseAudio server on the DAC**.

On FreeBSD, `make install` turns off both without editing any package-owned file:

File installed	Effect
`<pulse etc>/client.conf.d/10-omdrc-no-autospawn.conf`	`autospawn = no`; read after the package's `client.conf`
`~<audio user>/.config/autostart/pulseaudio.desktop`	a same-named user entry with `Hidden=true`, which masks the system autostart entry

`<pulse etc>` is found automatically (`/usr/local/etc/pulse` on a normal install) or set with

-DOMDRC_PULSE_ETC_DIR. If PulseAudio is not installed, the drop-in is put in place anyway, so the guard is ready if PulseAudio is added later. The autostart mask is never written over a pulseaudio.desktop the user made themselves (one without the project's marker); the installer keeps it and prints what to add. Under DESTDIR (package builds) the mask is not written into a home directory; the installer prints the command to run on the target. The guard takes effect from the next login and does nothing to a server that is already running. The installer warns if one is running; stop it with pulseaudio --kill.

With PulseAudio unable to start, Chromium's probe finds no server and moves on to sndio, which is exactly what the launcher needs.

16.6.4 Firefox

Firefox has its own OSS backend (cubeb) and opens the DAC directly; it needs neither sndiod nor any other setup. It runs with --no-remote, so the launcher always owns a new instance.

16.6.5 The ALSA route (kept as a fallback, not recommended)

Before sndiod, the launcher drove Chromium through ALSA. That code is still in lib.sh (browser_alsa_begin) and works, but **it does not mix**, so it has the single-open problem described above: one stream at a time, and a collision makes the whole session silent. To use it, change BROWSER_AUDIO=sndio to BROWSER_AUDIO=alsa in the launcher script (an environment variable does not override it) and, once per profile, set chrome://flags -> *Audio Backend* -> *ALSA*. The command line cannot select the ALSA backend; the launcher warns if the flag is missing.

What it does:

- writes an ALSA configuration **for this run only** to a temporary directory and passes it to the browser alone through ALSA_CONFIG_PATH. No system ALSA file is touched, and MPD, BruteFIR and the CD bridge do not use libasound, so the configuration cannot reach the bit-perfect paths;
- defines pcm.omdrc_dac as plug over the OSS plugin on the DAC, **set to the DAC's current rate** (as for sndiod) and to S32_LE;
- replaces default through a second @hooks entry, because alsa.conf loads /usr/local/etc/asound.conf, whose default is plug:oss, *after* the top-level file;
- passes --audio-backend=alsa --alsa-output-device=omdrc_dac.

The format has to be fixed. Left to negotiation, the OSS plugin offers S24_LE; for a 16-bit browser stream plug picks it, the device runs 32-bit, and the samples arrive **256 times (48 dB) too quiet**. That sounds like silence, with no error anywhere. A DAC that cannot take S32_LE needs BROWSER_ALSA_FORMAT.

16.6.6 Checking it and fixing silence

Run the launcher from a terminal to see its messages (the menu entry hides them):

```
/usr/local/libexec/omdrc/browser-nodrc/chromium-nodrc.sh
```

A healthy start prints:

```
browser-nodrc: disabling DRC for direct DAC output
browser-nodrc: playback-only sndiod mixing on rsnd/<unit> at 44100 Hz (pid NNNN)
```

and, after Chromium is closed:

```
browser-nodrc: sndiod stopped
browser-nodrc: restoring DRC (<mode>)
```

While a page plays, in a second terminal:

```
pgrep -lx sndiod                      # the launcher's sndiod is running
pgrep -lx pulseaudio                  # must print nothing
fuser /dev/dsp.dac                    # only sndiod's pid holds the DAC
sysctl dev.pcm.<unit>.feedback_rate   # still the rate from before the launch
```

If Chromium is silent, check in this order:

1. **Was Chromium already running?** The launcher said "already running — DRC left unchanged". Quit every Chromium window (and any Chromium still running in the background) and launch again.
2. **Is PulseAudio running?** `pgrep -lx pulseaudio`. Stop it with `pulseaudio --kill`, then check that both guard files above exist.
3. **Is an *Audio Backend* set in `chrome://flags`?** Set it back to *Default* and relaunch.
4. **Is the DAC held by something else?** The launcher prints the holder after `DAC_FREE_WAIT`: usually a previous browser still shutting down, or MPD playing through its direct output. Stop it and relaunch.
5. **Is sndiod there?** "sndiod not found" means `pkg install sndio`; "sndiod failed to start" usually means the DAC was busy (item 4).
6. **Does the DAC play at all?** Close the browser, run `omdrc off`, and play a test tone directly: `sox -n -t ossdsp /dev/dsp.dac synth 2 sine 440 vol 0.1`. If that is silent too, the problem is the DAC or its rate (section 16.2), not the browser.

After the browser exits, `omdrc session` should show the power and mode it had before the launch, and `pgrep -x sndiod` should print nothing, unless a system sndiod was already running.

Chapter 17

FreeBSD: video — mpv playback and the phone web remote

Video is documented here for FreeBSD (`video/README.md`). The scripts are the same on Linux; Blu-ray decryption with MakeMKV, which is Linux-only today, is in section 13.

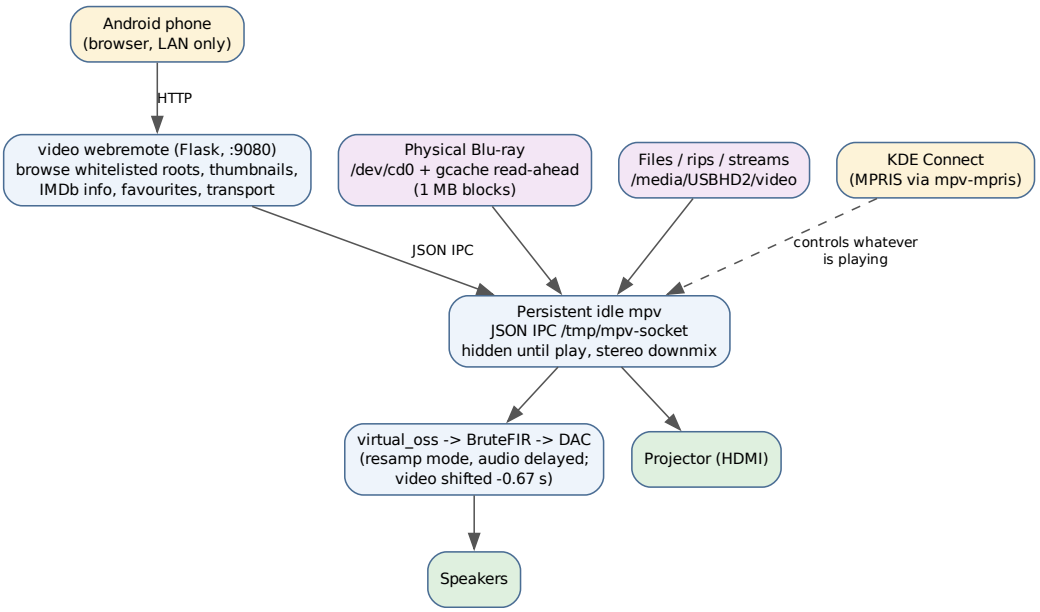


Figure 17.1: Video playback and control paths.

17.1 Playback launchers

- **`play-bluray.sh`** — physical Blu-ray discs. Kodi cannot read a physical BD on FreeBSD (raw `/dev/cd0` wants 2048-byte-aligned reads; the kernel cannot mount UDF 2.50), so mpv + libbluray read the raw device. Because the USB drive only sustains full speed in ~1 MB

chunks and raw `cd0` has no kernel read-ahead, the script fronts the drive with a **GEOM cache** (`gcache create -b 1048576 -s 268435456 bd cd0` -> `/dev/cache/bd`), and probes `bd_list_titles` to play the *genuinely longest* title (mpv has no BD menu support; `e`/`E` cycle titles at runtime).
- **`play-media.sh`** — local files, playlists, and network/stream URLs (m3u8, yt-dlp sites); same DRC audio routing, no gcache.

Both source `lib/drc-audio.sh`, which ensures the chain is in **resamp mode** before playing — necessary because the direct DAC is bit-perfect (`bitperfect=1` on the DAC's unit): a 48 kHz movie on a higher-clocked DAC would play ~2x fast. With DRC up, mpv plays to `oss//dev/dsp.play` and delays the **video** by `DRC_VIDEO_DELAY` (default **0.67 s**) to match the audio-path latency; subtitles ride with the picture automatically.

The 0.67 s is derived, not guessed (full derivation in `video/AV-SYNC-DELAY.md`): the FIR filter's impulse peak sits at sample 96000 of 524288 taps at 192 kHz = **0.500 s group delay** (the coefficient list *is* the impulse response; the peak is when a transient emerges), plus one BruteFIR partition (32768 samples at 192 kHz = **0.171 s**), plus a little `virtual_oss` buffering.

DVDs are simpler: they mount fine (UDF 1.x) and are low-bitrate, so `mpv dvd:// --dvd-device=/dev/cd0`; commercial discs need `libdvdcss`.

Remote control: the `mpv-mpris` package auto-loads into every mpv, so KDE Connect's Android *Media control* (and `playerctl`, Plasma widgets) drive whatever is playing — play/pause/seek/volume/metadata.

17.2 The web remote (`video/webremote/`)

KDE Connect controls what is *already playing*; the web remote is what *starts* a title. A separate Flask app (port 9080, LAN-only, rc.d service `omdrcvideo`) serving a phone UI to:

- **Browse** the whitelisted media roots (realpath containment — no `..` or symlink escape), with entries classified server-side: folder, Blu-ray rip (`BDMV/index.bdmv`), DVD rip (`VIDEO_TS`), or playable file. Grid (poster thumbnails) or compact list.
- **Thumbnails** via ffmpeg frame grabs, disk-cached by path+mtime, with a background prewarm thread and bounded concurrency.
- **IMDb info** — title deduced from the name and, with an OMDb API key, verified and enriched (year, director, cast, plot, rating).
- **Play** on a **persistent idle mpv** over its JSON IPC socket (`/tmp/mpv-socket`) — hidden until something plays, DRC audio configured once at startup, `audio-channels=stereo` so 5.1/7.1 sources downmix. Blu-ray rips get the longest-title probe; a **Play Blu-ray disc** button reuses the gcache lifecycle for physical discs, loading into the same mpv.
- **Transport** — seek, +-10/30 s, play/pause, mute, stop, audio and subtitle track menus; **favourites** pinned to the main page.

The idle mpv is autostarted by the KDE/Plasma session, from `~/.config/autostart/mpv-idle.desktop` (linked to the installed entry by `make user-install`); a `git pull` + `service omdrcvideo restart` is the whole update path.

17.3 Video-related FreeBSD constraints

- Physical Blu-ray: no kernel UDF 2.50 mount; raw `/dev/cd0` needs sector-aligned reads and has no read-ahead — hence mpv + libbluray + gcache. Kodi's internal player cannot do it.
- Kodi's OSS sink does not enumerate cuse userspace devices at all — the in-tree Kodi patch fixes that (section 20.3).
- Decryption on FreeBSD is stock libaacs + `~/.config/aacs/KEYDB.cfg`, **by design**. The MakeMKV route of section 13 cannot work here: the FreeBSD port (`multimedia/makemkv`) builds MakeMKV with the Linux toolchain for the Linux ABI layer, so its `libmmbd.so.0` is a Linux library that the native libbluray of mpv and Kodi cannot load. Every MakeMKV switch is therefore Linux-only: `makemkv-env.sh` exports nothing, the panel's Kodi wrapper runs plain `kodi`, the web remote hides the MakeMKV key entry and refuses its key API, and the Blu-ray check reports *Decryption: libaacs + KEYDB.cfg*. FreeBSD keeps both libaacs limits: discs missing from FindVUK do not decrypt, and a drive that rejects libaacs's revoked host certificate reads no AACS disc (mpv log: `has been revoked by your drive`). Keep `KEYDB.cfg` current through the Blu-ray check's upload.
- The Linux-only playback tuning of `drc-audio.sh` (ALSA buffer and periods, `--autosync`, hardware decoding, the read-ahead options) and the drive-speed request in `disc.sh` are confined to the Linux branch; FreeBSD gets no extra mpv options and keeps the gcache path.

Chapter 18

FreeBSD: CD input with omdrc-cdin

FreeBSD ships nothing that reconciles two free-running audio clocks, so `omdrc-cdin` (`cdin/`) was written: it bridges the S/PDIF capture device into the same `virtual_oss` entry point MPD uses. The concept, the exclusive-source rule and the web card are in chapter 10.

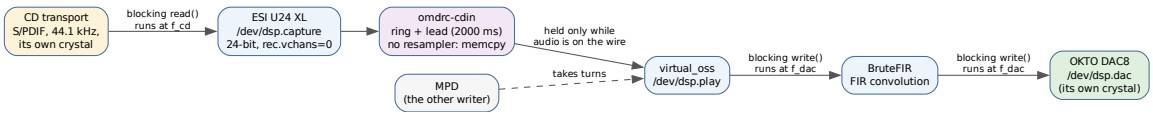


Figure 18.1: The CD path. Both ends of the bridge block on their own device, so the ring fill between them is the drift signal; there is no resampler anywhere in it.

18.1 Why a bridge is needed at all

Capture is slaved to the CD's crystal, the DAC runs on its own, and the two differ by a few ppm **forever**. The missing piece was a tool, not a kernel facility: a blocking `read()` runs at the CD's clock and a blocking `write()` at the DAC's, so **the ring fill between them is the drift signal**. No OSS clock ioctl is needed to measure it.

There is no resampler. The data path is a `memcpy`, so what reaches BruteFIR is bit-identical to what the transport sent — verifiable with `scripts/verify-bitperfect.sh`, which would be meaningless with a resampler in the path. Drift is absorbed by the *lead* instead.

18.1.1 The lead is the only number that matters

The lead — how much audio is buffered ahead of the output — is simultaneously three things:

- the **drift margin**: how long before the buffer runs out;
- the **startup delay**: you cannot pre-fill a lead you have not waited for;
- the **transport lag**: every Play/Stop/Skip is heard this much later.

They cannot be tuned separately. The upper bound is arithmetic: `time-to-splice = lead / drift`, so at a pessimistic **50 ppm** a **2000 ms** lead covers about **11 hours** of continuous gapless audio. A disc is at most 80 minutes, therefore **drift cannot cause a discontinuity inside a disc**.

The *lower* bound is not set by drift at all. It is set by transport seeks and USB stalls: some players briefly **drop carrier** across a pregap or index boundary, which is a sub-second input stall that the lead has to absorb. That, not drift, is why the default is 2000 ms rather than the ~50 ms drift alone would need. Below ~250 ms the daemon warns that nothing is left to absorb a seek.

Calibrate it on the real transport with a full disc:

```
omdrc-cdin --in /dev/dsp.capture --out /dev/dsp.play --lead 2000 -d -s 10 \
    -l /tmp/cdin.log
```

`starves` must stay 0 — each one is an audible dropout. After about five minutes the `drift` field reports the measured ppm and the projected headroom, which replaces the 50 ppm assumption with the actual hardware. Step `--lead` down (1500, 1000, 750…) until the first value that produces **any** starve; that is below the floor the transport imposes, so go back up one step and keep a margin. Record the value: it becomes `omdrc_cdin_lead`.

18.2 The three states, and the two very different tenancies

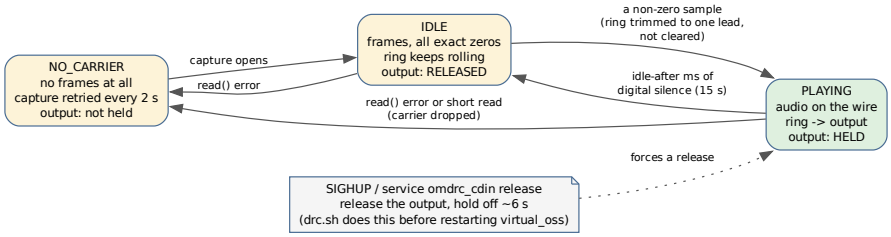


Figure 18.2: The daemon's state machine. The output device is held only in PLAYING; the capture device is held for the whole session.

State	On the wire	The daemon	`/dev/dsp.play`
`NO_CARRIER`	no frames at all	retries the capture device	not held
`IDLE`	frames, all exact zeros	counts the silence	**released**
`PLAYING`	audio	ring -> output	held

The **capture** device is held for the life of a session: it is the interface's own node, nobody else wants it, and it is the only thing that can tell whether a carrier exists. The **output** device is `virtual_oss`'s client node and is taken only while music plays.

That asymmetry is not politeness. `drc.sh` restarts `virtual_oss` on every rate change, and an open cuse client handle at that moment wedges the teardown *permanently*: `cuse_server_free()` spins uninterruptibly, SIGKILL does not touch it, and only a reboot recovers (section 20.2). A daemon holding `/dev/dsp.play` around the clock would put that hazard under every rate change. (The deployed source policy is stricter still: MPD's audible output stays disabled while the bridge is selected, even in silence, because `virtual_oss` mixes clients.)

For the remaining exposure — a rate change while the bridge runs — `drc.sh` stops the service and takes process exit as the release acknowledgement:

```
service omdrc_cdin onestop
# wait until pgrep -x omdrc-cdin no longer finds the process
```

After the chain is rebuilt the bridge is restarted, including one started with `onestart` while its rcvar is off. If the bridge does not exit, teardown is refused and `drc.sh` restores MPD's direct output so the machine is not left silent.

Choosing `--idle-after` (default 15000 ms) has one failure mode at each end and a wide safe band between them: too short and Red Book's 2 s inter-track pause releases the device mid-disc, so every track change costs a lead to resume; too long and a stopped player keeps holding the chain. `--idle-after 0` disables the gate entirely, which is occasionally useful when measuring. Note this is silence *on the wire*: a player that drops carrier instead of sending zeros never reaches the gate at all — the read fails and it lands in `NO_CARRIER`, which is the state that reopens the device.

Resuming does not lose the first note. The ring keeps rolling through the silence, so an episode begins by *trimming* it to one lead rather than clearing it and waiting for a fresh pre-fill. The music still emerges one lead later, but the period that carried the first sample is still in the buffer.

18.3 What the transport does to the stream

A CD player's S/PDIF output is **always 44.1 kHz**. Transport actions change *what* is sent, never *how fast*, so there is no re-lock, ever:

Action	On the wire	Daemon behaviour
Pause	carrier alive, digital silence (most players)	plays the silence through; lead unchanged
Skip track	brief mute (0.1–1 s), then audio	a short silence, heard one lead later
Fast fwd / rewind	chopped scan snippets or mute, rate unchanged	ordinary audio or ordinary silence
Stop / tray / power	carrier drops	`read()` stalls or errors, session ends, device reopened

Only the carrier drop matters, because the wall clock keeps running while no frames arrive: the lead drains by exactly the dropout's length and never recovers.

18.3.1 Every row of that table is testable without a CD player

Point `--in` at a **directory** and its `*.wav` files become the tracks of a disc, played in name order; a single file is a one-track disc. Between tracks the rig emits `--gap` ms of exact digital silence (default 2000, Red Book's inter-track pause), which is what the silence gate looks for. `--transport` scripts the buttons as `AT:EVENT` pairs, `AT` being seconds into the stream:

```
omdrc-cdin -i DISC -o /dev/dsp.dac -d -s 5 \
    --transport "20:skip,35:pause=4,55:dropout=800,70:seek=+30,105:stop"
```

`dropout=N` drops the carrier for N ms; `--in-ppm` offsets the simulated source's clock, making the design's central claim testable in seconds instead of the day real hardware needs. The two failure modes behave as the arithmetic predicts:

- **lead exhausted** — `starves` increments, then `in` and `out` converge to the *same* wrong rate. That equality is the backpressure signature: with no buffer left, the DAC is paced by the source instead of its own clock;
- **ring saturated** — `lead` pins at the ring capacity and `drops` climbs at the drift rate; audio is discarded, one discontinuity per drop.

The rig emulates the CD player's clock, not the disk's seek time, so the disc is prefetched on its own thread (4 s deep). A schedule that slips more than 100 ms is shifted forward rather than firing every overdue deadline at once; both effects are counted and shown as `rig stalls N slips N` only when non-zero.

18.4 Reading the stats line

```
[stats] lead 1635 ms (min 1625, max 1649)  drift +38.7 ppm (+/-387.0),
        ring fills in 46 h  in 44100.206 Hz  out 44088.817 Hz
        frames 4054016/3980288  drops 0 B  starves 0  silence 0%  up 90 s
```

- **`lead` is the ring only.** It settles *below* `--lead`, because the pre-fill hands the first few hundred ms straight to the output device's own buffer. End-to-end latency is this figure plus that buffer, plus `virtual_oss`'s 200 ms and BruteFIR's filter group delay; the startup delay actually waited is `--lead`.
- **`drift` is measured from the change in `lead`**, not from the frame counters: those carry each device's constant buffer offset, which at ppm scale would swamp the figure and which cancels in a difference. The `+/-` is the period quantisation over elapsed time — while it exceeds the estimate, the estimate means nothing. It needs minutes and tightens for hours.
- **`in` / `out` are measured from the instant playback began**, and the window restarts at every discontinuity, so a dropout does not leave the cumulative average reading low for the rest of the session.
- **`starves`** counts events, not periods: one continuous starvation is 1.
- **`drift ref dropped (lead jumped)`** means the estimate was thrown away and restarted because the lead moved for a reason that is not the clocks. A 3.3 s jump inside a 60 s window once read as `+54361 ppm`, which is a stall wearing a drift figure's clothes.

18.5 Running it as a service

```
# /etc/rc.conf
omdrc_cdin_enable="YES"
omdrc_audio_capture="ESI U24XL"   # names the ESI; that is what creates
                                  # /dev/dsp.capture, which cdin then uses
```

rc.conf variable	Default	Meaning
`omdrc_cdin_in`	`/dev/dsp.capture`	capture device (or a WAV file/directory, for the rig); the link comes from `omdrc_audio_capture`

rc.conf variable	Default	Meaning
`omdrc_cdin_out`	`/dev/dsp.play`	playback device; `/dev/dsp.dac` writes the DAC directly, bypassing BruteFIR
`omdrc_cdin_bits`	`24`	source width — the U24 XL's capture endpoint is 24-bit and nothing else (see below)
`omdrc_cdin_lead`	`2000`	lead in ms; drift margin, startup delay and transport lag at once
`omdrc_cdin_idle_after`	`15000`	digital silence before the output device is released; `0` disables the gate
`omdrc_cdin_logfile`	`/tmp/omdrc-cdin.log`	**must match** `log_file` in `commands.conf`'s `[cdin]` — this file *is* the web card
`omdrc_cdin_stats`	`10`	seconds between `[stats]` lines
`omdrc_cdin_user`	`AUDIO_USER`	run user; the same one that owns BruteFIR and MPD, because they take turns on the same devices
`omdrc_cdin_flags`		anything else: `--out-bits`, `--period`, `--retry`, `-v`

The rc script is installed by the CMake superproject (`cdin/CMakeLists.txt`, a subproject of the top-level build). It uses `daemon(8)`, drops privileges via the standard `rc.subr` `${name}_user`, and adds one non-standard verb, `release` (the diagnostic `SIGHUP` above). A `--` separates the daemon supervisor's arguments from the bridge's arguments; the public `omdrc_cdin_flags` value is moved out of rc.subr's reserved `${name}_flags` namespace before startup. Started unprivileged with `service omdrc_cdin onestart` it clears `${name}_user` — so `rc.subr` does not try to `su` to it — and uses a pidfile under `/tmp`.

The panel owns this service even when `omdrc_cdin_enable="NO"`. Selecting the CD source explicitly may start it with `onestart`; an incidental rate change never starts a bridge that was already stopped. When a running bridge must follow a rebuilt virtual output, `drc.sh` uses bounded `onestop`, waits for the process to disappear, then uses bounded `onestart`. This preserves a panel-started instance: `onerestart` would stop it and then let the disabled rcvar reject the start half. The timeout runs in foreground mode so its process-group cleanup cannot kill daemon(8)'s successfully detached supervisor.

Log lines are a **contract**, not just prose: the web panel parses them, so `state <name>: <why>` and `<device> <path>: <available|unavailable|acquired|released>` keep their shape. Availability and holding are separate axes: `unavailable` means nothing can play and is the red light, while `acquired`/`released` is the ordinary rhythm of a daemon doing its job and is never a fault.

18.5.1 The panel card on FreeBSD

The exit status of `service ... onestart` is not trusted — it forks a `daemon(8)` and returns before the daemon can die on a missing device — so the panel polls with `pgrep -x` until it agrees. The Start/Stop button needs a `sudoers` grant; without one, set `control = no` rather than leave a button that can only fail:

```
omdrcctrl ALL=(root) NOPASSWD: /usr/sbin/service omdrc_cdin onestart, \
    /usr/sbin/service omdrc_cdin onestop
```

18.6 The ESI U24 XL on FreeBSD: three traps

The hardware facts are in chapter 10. Everything below was observed on FreeBSD 15.1-RELEASE-p2 and is documented at length in `cdin/ESI-U24XL.md`. Role links and the `omdrc_audio_capture` setting are in section 15.2.

18.6.1 Trap 1: the S/PDIF input is called `pcm2`, and must be selected

The card comes up on the *analog RCA* input after a boot, so `cdin` records silence from a healthy transport until the recording source is switched:

```
mixer -f /dev/mixer.capture pcm2.recsrc=set   # mixer.capture pairs with dsp.capture
mixer -f /dev/mixer.capture -s                # print just the active source
```

Only devices flagged `rec` can be a source; here `line` (selector position 1, analog RCA) and `pcm2` (position 2, digital S/PDIF) are the two positions of the card's single USB selector unit (`sysctl dev.pcm.<unit>.mixer.selector_0`), mutually exclusive, so `set`, `add` and `toggle` all end with one source. The name is not arbitrary: `uaudio` maps `UATE_SPDIF` to `SOUND_MIXER_ALTPCM`, whose index in `SOUND_DEVICE_NAMES` is `pcm2`. The `dig1..3` names are fallbacks for colliding selector pins; the pre-14 syntax `mixer =rec dig1` no longer exists.

The setting survives neither a reboot nor a replug (`/etc/rc.d/mixer` saves state only for `mixer0` unless `mixer_enable="YES"`, and needs the U24 XL attached at boot). `omdrc_audio` therefore asserts it on every attach, right after linking the card:

```
omdrc_audio_capture_recsrc="auto"   # the default
```

`auto` prefers the digital input *by inspection*: `mixer(8)` tags every possible source `rec` and the active one `src`, so the service looks for `pcm2`, then `dig1..3`, among the sources the card offers and leaves a card with none of them alone. Name a device (`line`) to force analog, or `none` to keep hands off. The command is idempotent and costs one USB control transfer.

18.6.2 Trap 2: `pcm2 = 0.00:0.00` is not a muted capture gain

`mixer -f /dev/mixer.capture` shows `pcm2` at `0.00` beside three devices at `0.75`, which reads as a zeroed record level. It is not: there is no gain to raise. Sweeping it moves no hardware node, and there is no software fallback (`sys/dev/sound/pcm/mixer.c` applies feeder volume only to `SOUND_MIXER_PCM`, the playback path). The `0.00` is a display artifact — there is no `[SOUND_MIXER_PCM2]` entry in `snd_mixerdefaults[]`. The capture level on S/PDIF is whatever the transport sends, bit for bit.

18.6.3 Trap 3: a "44100 Hz" open that is neither 44.1 kHz nor bit-perfect

Every layer reports success. By default FreeBSD puts a **virtual channel** in front of the card: the hardware runs at `dev.pcm.N.rec.vchanrate` (48000, always) and a kernel `feeder_rate` resamples to whatever the application asked for. `SNDCTL_DSP_SPEED` returns 44100 and every ioctl succeeds, while the card actually delivers 44100 frames per second into a stream the kernel believes is 48000:

44100 x 44100/48000 = **40517 Hz** arriving at the application.

`cdin` reads 40.5k frames/s and writes 44.1k to the DAC, so the lead drains at 3.6k frames/s: a 2000 ms lead is gone in about 25 seconds and the DAC then underruns continuously. It looks exactly like catastrophic clock drift and is not. The only place it shows:

```
sysctl hw.snd.verbose=2 && cat /dev/sndstat

[dsp1.record.0]: spd 48000 ...                              <-- the hardware
dsp1.record.0[dsp1.virtual_record.0]: spd 44100/48000 ...
  ... -> feeder_rate(q:4  48000 -> 44100) -> {userland}    <-- the lie
```

The cure is `omdrc_audio_capture_sysctls` (`rec.vchans=0 bitperfect=1`), after which the same command prints `{hardware} -> feeder_root(0x00210000) -> {userland}`: one `memcpy` from the USB transfer to the read buffer, at the rate the transport really runs.

18.6.4 The consequence: capture is 24-bit

With the format feeder gone, the card's own width is the only one it accepts, and the U24 XL's capture endpoint offers exactly one — 24-bit S-LE at 44100 or 48000 Hz. So `omdrc_cdin_bits` is **24**, and the two ends of the bridge do not share a format: downstream runs S32_LE (`virtual_oss -b 32`, BruteFIR `sample: "S32_LE"`), and with `bitperfect=1` the output device refuses a width instead of converting it. `cdin` negotiates and converts itself (`cdin/src/convert.c`): the output is opened at the **source** width first; if refused, a wider one is tried, **never** a narrower one. Widening is left-justification, pure byte placement for little-endian PCM (`24 -> 32` adds a zero byte), so there is no rounding or dither and the bit-perfect claim survives. `--out-bits N` forces the width.

24-bit capture also needed a fix inside `cdin`. `SNDCTL_DSP_SETFRAGMENT` encodes the fragment size as an exponent, so only powers of two are expressible, and a 24-bit stereo frame is 6 bytes: every period is `6N` bytes, never a power of two. The fix was to stop treating the fragment as the transfer size — it is the device's *interrupt granularity*. `cdin` now asks for the largest power of two that fits inside the period (4096 bytes for a 6144-byte period) and keeps reading its own 1024-frame periods. Not one byte of audio changes.

18.6.5 Checklist when the CD input captures silence

0. `service omdrc_audio status` — the U24 XL must hold the `capture` role with `bitperfect=1 rec.vchans=0`, and the DAC the `dac` role. It also prints the links and the active recording source. If the stream is not silent but *distorted*, and the lead drains to zero in about 25 s, it is Trap 3, not the selector.
1. `ls -l /dev/dsp.capture` — it must exist and point at the U24 XL's unit. If it does not, `omdrc_audio_capture` does not match the card's name.
2. `mixer -f /dev/mixer.capture -s` — must print `pcm2`. If it prints `line`, the card is on the analog input and `omdrc_audio_capture_recsrc` was set to `none` or to `line`.
3. Ignore `pcm2 = 0.00:0.00`; it is cosmetic.

4. Only then look at the transport, the cable and lock.

18.7 Status and what is still owed

The bridge, its state machine, the lazy output open, the width negotiation, the rc.d service and the web card are in place. The simulated transport has run a 12-track 16-bit disc end to end against a bit-perfect `/dev/dsp0` with `starves 0`, `drops 0` and the lead inside a one-period band — across track gaps, skips, seeks, a 4 s pause and an 800 ms carrier dropout (which cost exactly 800 ms of lead and starved nothing). Three unit-test suites cover places where a bug would be silent: `test_ring`, `test_convert` (the widening, pinned by the *value* relation `src << (dst_bits - src_bits)` plus canary bytes) and `test_gate`.

Not yet proven with a real transport attached:

- **carrier loss**: does `read()` block, short-read or error when the CD stops or the coax is unplugged? The state machine assumes a short read or error means `NO_CARRIER`; if a stopped player blocks the read forever, the daemon needs a read timeout;
- **the real drift**: `omdrc-cdin --in /dev/dsp.capture --out none -d -s 30` against the OKTO's `feedback_rate` gives the drift the lead must cover;
- **loopback pacing**: `virtual_oss` runs with `-f /dev/null` and owns no hardware clock; confirm a writer to `/dev/dsp.play` is throttled by BruteFIR draining `/dev/dsp.loop`, and see what happens when BruteFIR is *not* running (writes may block forever);
- **the shared-clock patch with two USB audio devices streaming** (section 20.1); ideally put the ESI on a different root hub.

Drift resync during inter-track silence (phase 2b) is not needed inside a disc — drift cannot cause a discontinuity in 80 minutes — but is needed for a session that never stops. The seams are marked `TODO(phase2b)` in the source.

Chapter 19

FreeBSD: known issues and their status

- **Static address on a wired port with no cable**: libupnpp binds the dead port and upmpdcli becomes undiscoverable while still running; a static `defaultrouter` compounds it. Both are `rc.conf` bugs: section 14.5.
- **OKTO 44.1 kHz-family flicker** (bug #295933): the DAC continuously drops and re-acquires USB streaming lock on 44.1/88.2/176.4/352.8 kHz while the 48 kHz family is stable (Linux plays everything). The device has one UAC2 Clock Source **shared** by playback and capture, and `uaudio(4)` lets the vestigial capture side reprogram it to 48 kHz under active playback. Fixed by the shared-clock patch (section 20.1).
- **Rate-change cold-open silence**: on the first open after *any* rate change the DAC shows the right rate and healthy USB but routes no audio; a second open fixes it. Stock `uaudio` programs the clock *after* selecting the alt-setting (Linux does the opposite). Worked around by `drc.sh` priming (section 16.2); fixed by the clock-before-alt patch.
- **virtual_oss livelock** ("155% CPU", frozen chain): a `SNDCTL_DSP_SETTRIGGER` on a read-only fd could strand the synchronized engine in a wait nothing wakes. Fixed by the settrigger patch (section 20.2).
- **cuse teardown wedge** (bug #296291): stopping virtual_oss could leave it unkillable in `D<E`, pinning `cuse.ko` until reboot — a kernel refcount leak in `cuse_client_open()`'s `is_closing` path (regression from `634e578ac7b0`, in 15.1). Kernel fix written (section 20.2).
- **`pcm` unit numbers are attach order** and nothing declarative can change them, so with a second USB audio device the DAC can lose `/dev/dsp0`. Handled by not depending on the number: section 15.2.
- **A capture open can succeed at the wrong rate**: with a record virtual channel in front of the card the hardware runs at `rec.vchanrate` (48000) while every ioctl reports the requested rate, so a 44.1 kHz S/PDIF source arrives at ~40517 Hz and looks like catastrophic drift. `rec.vchans=0` plus `bitperfect=1` is the cure (section 18.6).
- **musicpd 100% CPU on HTTP streams**: a libcurl leftover-fd spin — curl registers an event fd into MPD's I/O loop and never removes it. **Audio is unaffected**; MPD upstream has triaged it third-party (#2244/#2229); the venue is libcurl.
- **One wire format per attach**: `uaudio` fixes s32le at attach and pads 16-bit content to 32 bits, so the DAC panel shows 24 bits on 16-bit tracks. Bit-perfect (zero-padding is lossless), just unlike Linux's per-stream alt switching. Knobs: `hw.usb.uaudio.default_bits`/`default_rate`.

Chapter 20

FreeBSD: kernel and userland patches

Local fixes kept in-tree while the official FreeBSD fixes are pending. All `/usr/src` patches apply with `-p1` against `releng/15.1`.

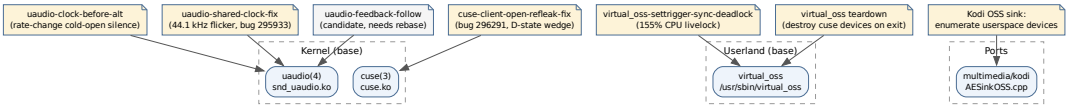


Figure 20.1: The patched layers at a glance.

> **Upgrade caveat (every kernel/userland patch below):** `freebsd-update` or `make installkernel` overwrites patched binaries with stock ones. After any OS/kernel update, re-apply the patches and rebuild. A `.ko` is ABI-specific to its kernel — never keep prebuilt binaries.

20.1 uaudio(4) patches (`freebsd-uaudio-patch/`)

Target device: OKTO RESEARCH DAC8 STEREO (USB `0x152a:0x88c5`, Thesycon UAC2 firmware, USB High-Speed). Patches 1 and 2 apply in order on stock `releng/15.1`; patch 3 is built but not installed; patch 4 is an unbuilt candidate.

20.1.1 1. `uaudio-clock-before-alt.c.patch` — rate-change cold-open silence

Problem. On the first open after any sample-rate change the DAC shows the correct rate and healthy USB (feedback present, no underruns) but routes silence; a second open at the same rate fixes it, which is why `drc.sh` primed.

Root cause. Stock `uaudio_configure_msg_sub()` does `SET_INTERFACE` to the streaming alt-setting (arming the device), *then* `SET_CUR` of the rate on the UAC2 Clock Source (possibly a crystal switch under the armed interface), then starts transfers. Linux does the opposite. The Thesycon firmware latches its stream configuration at `SET_INTERFACE`, so FreeBSD arms the stream against the *old* clock. This also explains why open/close priming works: every close parks at alt 0.

Fix. For UAC2 devices in `CHAN_OP_START`: park at alt 0; program the clock while idle (legal — the UAC2 clock lives on the AudioControl interface); on a genuine rate change sleep `hw.usb.uaudio.clock_settle_ms` (default 100 ms, tunable, clamped to 2000) for crystal relock; then `SET_INTERFACE` and start. UAC1 devices are untouched.

Status. Built and installed since 2026-07-06; different-rate tracks lock first try. It replaces the host-side `DAC_PRIME_CYCLES` prime for all clients. If a crossing is ever silent, raise `clock_settle_ms`.

20.1.2 2. `uaudio-shared-clock-fix.c.patch` — the 44.1 kHz flicker (bug #295933)

Root cause. The DAC exposes **one UAC2 Clock Source shared between playback and capture**. On async playback `uaudio` auto-starts the record channel purely as a jitter-information source at its own nominal rate (48 kHz), and its `SET_CUR` clobbers the shared clock under active 44.1 kHz playback.

Fix — three cooperating, device-agnostic changes: (a) **rate-align the jitter record stream** so the rec-side `SET_CUR` is a same-value no-op and its framing stays valid; (b) a **shared-clock guard** that skips a `SET_CUR` to a clock shared by both directions when the other direction runs at a different rate; (c) **always submit the explicit-feedback SYNC transfer**, so `dev.pcm.<unit>.feedback_rate` stays live for `drc.sh`'s chain-sanity check. A guard-only version was rejected: without (a) the rec channel expects 48 kHz framing while the device delivers 44.1, the jitter estimate pins at its negative clamp and the play callback strips samples continuously. This fix replaces the retired VID/PID-gated capture-disable workaround; `pcm0 (play/rec)` in sndstat is the expected state again.

Status. Applied 2026-07-07, `-Werror`-clean alone and on top of patch 1. Upstream: bug #295933 / PR 2323, landed as `755685dd665e` (MFC `6886e8a9a0aa`); superseded by patch 3.

20.1.3 3. `uaudio-clock-transaction.c.patch` — residual 44.1 kHz silent open

Problem. The landed guard skips a `SET_CUR` only on a rate *mismatch*, and its own jitter-stream alignment guarantees a *match*, so the capture pass issues a second same-value `SET_CUR` right after playback is armed (confirmed with `hw.usb.uaudio.debug=6`: two `SET_CUR` to clock 41 and one settle per open). Upstream lacks the clock-before-alt reorder, so its second write lands under two armed interfaces. Separately measured: `uaudio` auto-streams the OKTO's vestigial capture interface for every playback — 251.9 isochronous completions/s against 125.4 playback interrupts/s, half the device's isochronous bandwidth — to recompute a number the explicit feedback endpoint already reports.

Fix. (a) Never write a UAC2 clock another stream owns; (b) `GET_CUR` first and skip a redundant write, as Linux does; (c) stop borrowing the vestigial capture stream for jitter when the playback alt has an explicit feedback endpoint (`hw.usb.uaudio.prefer_feedback`); (d) do not start the jitter capture at a stale rate. **Not claimed:** that this causes the intermittent silent open — it did not reproduce in 42 controlled cycles (21 into 44.1).

Status. Built `-Werror`-clean with and without `USB_DEBUG` on top of 1 and 2; **not installed, not listening-tested.** Split for upstream as `upstream-series/uaudio-upstream-0001-shared-clock-write-discip` and `...-0002-prefer-explicit-feedback.c.patch` (`git am`-clean on `main` after `755685dd665e`; a GitHub PR committed by `christos@`, CC `hselasky@`). Audit and A/B plan: `uaudio-clock-transaction.md`; open items: `SUBMISSION-295933.md`; `bench/` holds a DAC lock bench and `bench/uaudio-affects.py`, which decides from USB descriptors whether a device is affected.

20.1.4 4. `uaudio-feedback-follow.c.patch` — candidate (unbuilt)

A sketch to make playback follow the device's feedback rate smoothly (Linux-style). It touches the same sync-callback region as patch 3, so it needs rebasing, and becomes live only once the vestigial capture stream stops being auto-started.

20.1.5 Applying and building

```
cd /usr/src
patch -p1 < uaudio-clock-before-alt.c.patch
patch -p1 < uaudio-shared-clock-fix.c.patch   # applies with offsets; also on pure
↪  stock
patch -p1 < Makefile.patch                     # adds -DUSB_DEBUG

cd /usr/src/sys/modules/sound/driver/uaudio
make clean && make
```

Install the module (back up stock first, refresh the backup after every OS update so the revert path matches the running ABI):

```
OBJ=/usr/obj/usr/src/amd64.amd64/sys/modules/sound/driver/uaudio/snd_uaudio.ko
sudo cp -f /boot/kernel/snd_uaudio.ko /boot/kernel/snd_uaudio.ko.orig
sudo service musicpd stop                  # release the DAC
sudo cp -f "$OBJ" /boot/kernel/snd_uaudio.ko
sudo kldunload snd_uaudio                  # devd auto-reloads on attach
UG=$(usbconfig | awk '/DAC8STEREO/{print $1}' | tr -d ':')
sudo usbconfig -d "$UG" reset              # clean re-enumeration
sudo sysctl -f /etc/sysctl.conf            # restore buffer_ms baseline
sudo service musicpd start
```

Verify: `grep pcm0 /dev/sndstat` shows (play/rec); `sysctl hw.usb.uaudio.clock_settle_ms` exists; `sysctl dev.pcm.$(readlink /dev/dsp.dac | tr -dc 0-9).feedback_rate` tracks the playback rate. Then run the listening matrix (both crystal directions, same-family changes, MPD-direct mixed-rate queue, browsers) — machine signals cannot verify these fixes; only listening counts. Revert with the `.orig` copy + kldunload/kldload.

20.2 virtual_oss / cuse patches (`freebsd-virtual-oss-patch/`)

20.2.1 1. Runtime livelock: `virtual_oss-settrigger-sync-deadlock.patch`

Symptom. Minutes after a chain (re)start, playback freezes: MPD stops advancing, BruteFIR starves, `virtual_oss` burns 150–200% CPU and both clients sit in `cuse-cli` waits.

Root cause (procstat + gdb on the live process), two userland bugs in `usr.sbin/virtual_oss`: (1) `SNDCTL_DSP_SETTRIGGER` ignores the fd's open mode, so BruteFIR's one call triggering both directions flips `tx_enabled = 1` on the read-only `/dev/dsp.loop` fd that can never write; (2) the synchronized loopback engine's wait loops check `tx_enabled` **once, outside the wait**, and the trigger/halt paths never wake the engine, so once it parks waiting for play data from a client that will never write, nothing re-evaluates the premise. The arming window is microseconds wide against a 200 ms block cadence, hence intermittent.

Fix (four changes, upstreamable): SETTRIGGER honours the open mode (`fflags`); trigger/halt ioctls `atomic_wakeup()` the engine; the sync wait loops re-check `tx_enabled/rx_enabled`; and a `tx_written` latch stops the engine sleeping for a client that has never written.

20.2.2 2. Teardown deadlock: userland device destroy + kernel refleak

Symptom. Stopping `virtual_oss` intermittently wedges it in `D<E` / `MWCHAN W` (SIGKILL-immune), pins `cuse.ko` and blocks recreating the devices until reboot. Every DRC rate change was a reboot risk.

- **Userland**: `virtual_oss` never destroyed its cuse devices on exit, so the kernel's `cuse_server_free()` busy-waited for client refs never released. `virtual_oss-teardown-int.h.patch` + `virtual_oss-teardown-main.c.patch` keep each `cuse_dev_create()` handle and call `cuse_dev_destroy()` on all DSP/WAV/loopback devices before exit (upstream committed an equivalent as `0bd5ef6b4363`).
- **Kernel** (the real fix, bug #296291): a regression from `634e578ac7b0` (Nov 2025, in 15.1). In `cuse_client_open()` the `is_closing` / `si_drv1==NULL` error path returns with `pcs->refs` incremented, the client still linked in `hcli` and no destructor registered, so **every open racing into the teardown window leaks one server ref** and `cuse_server_free()` waits forever (proven live with a diagnostic `cuse.ko`, dtrace and kgdb). `cuse-client-open-refleak-fix.patch` calls `cuse_client_free(pcc)` on both error paths; committed on branch `fix/cuse-client-open-refleak-296291`, compile-tested, `Fixes: 634e578ac7b0`, `PR: 296291`. The userland destroy mitigates normal exit but not `kill -9`; the kernel leak stays latent without this.
- A diagnostic-only `cuse-teardown-diag.c.patch` logs the stuck refcount every ~5 s (remove once the kernel fix lands).

20.2.3 Applying and building

```
cd /usr/src
patch -p1 < virtual_oss-settrigger-sync-deadlock.patch
patch -p1 < virtual_oss-teardown-int.h.patch
patch -p1 < virtual_oss-teardown-main.c.patch
patch -p1 < cuse-teardown-diag.c.patch        # optional diagnostic

# userland:
cd /usr/src/usr.sbin/virtual_oss && make && sudo make install

# kernel module (diagnostic or refleak fix):
cd /usr/src/sys/modules/cuse && make && sudo make install
```

Reboot first if a wedged virtual_oss is pinning `cuse.ko` — a new module cannot load and the devices cannot be recreated until then. Validation: churn `drc.sh <rate>` / `off` cycles ~20x; `virtual_oss` must stay near-idle CPU and always exit cleanly (no `D<E` in `ps -o pid,stat,mwchan`, `cuse.ko` unloadable).

20.3 Kodi OSS sink patch (`kodi-virtual-oss-patch/`)

Problem. Kodi's settings only ever offered the hardware DAC: `CAESinkOSS::EnumerateDevicesEx()` counts kernel PCM cards via `SNDCTL_SYSINFO`, and cuse userspace devices are not kernel cards, so `/dev/dsp.play` was never listed. The settings filler also silently *resets* any value it cannot match, so editing `guisettings.xml` never stuck.

Fix. After the kernel-card loop, parse the *"Installed devices from userspace:"* section of `/dev/sndstat` (the only place cuse devices are advertised) and add each node, probed like a kernel card (`SNDCTL_ENGINEINFO`, non-blocking open, graceful fallbacks, dedup). Generic: any userspace OSS device is listed. Verified on `multimedia/kodi` 22.0a3: *"dsp.play virtual_oss device"* appears in Settings > System > Audio, selecting it persists and Kodi feeds the DRC chain. `virtual_oss` must be running when Kodi initialises audio.

Apply through the ports tree:

```
sudo cp patch-xbmc_cores_AudioEngine_Sinks_AESinkOSS.cpp \
        /usr/ports/multimedia/kodi/files/
cd /usr/ports/multimedia/kodi
make config && make patch && make build
sudo make deinstall && sudo make install   # same version -> reinstall is a no-op
strings /usr/local/lib/kodi/kodi.bin | grep -c "from userspace"   # must print 1
```

Upstream: the FreeBSD port (`files/` patch) and/or Kodi (`AESinkOSS.cpp`; be ready to answer "why parse sndstat text" — cuse devices are unreachable via the mixer ioctls).

Chapter 21

FreeBSD: the port plan and a clean image

Status: **plan only, nothing applied.** Source: `doc/FREEBSD-PORT-PLAN.md`; Linux packaging is out of scope for now.

21.1 Why the repository cannot be ported as-is

A port installs identical, immutable files under hier(7) paths from a tagged tarball. Run-from-repo violates that on every axis, deliberately:

1. **Files are rendered per host**: CMake bakes `@AUDIO_USER@`, `@AUDIO_HOME@` and `@REPO_DIR@` from `host.cmake`; a package must install the same bytes everywhere and configure at runtime.
2. **The tree is written at runtime** (`last_arg`, `last_power`, `drc.log`): `pkg check -s` flags it; state belongs in `/var/db/`.
3. **Room data is mixed with software** (`configs/120.blue`, ~50 MB of `filters/*`): a port ships neutral defaults. `OMDRC_SITE_DATA_DIRS` / `OMDRC_SITE_ROOT` already resolve room data from a separate checkout.
4. **rc.d scripts shadow other ports** (`musicpd`, `upmpdcli`): the stock scripts' rc.conf knobs must be used instead.
5. **A personal BruteFIR fork** must become a port dependency.
6. **Kernel/userland patches**: a port cannot patch the base system, nor another port (`virtual_oss`); they must land upstream first.
7. **Packaging basics are missing**: no LICENSE, tagged releases only just begun (the `v*` series), and the tarball would ship debugging journals and kernel patches.

Run-from-repo does not have to die: the repository becomes a normal upstream that *also* supports `make install PREFIX=... DESTDIR=...`, stays the development/appliance mode, and the port is a thin consumer of tagged releases.

21.2 Phase 0 — Upstream the out-of-tree pieces (prerequisite, in flight)

- **uaudio patches** (shared-clock, clock-before-alt) to FreeBSD base: bug 295933 / PR 2323, in progress.
- **cuse refleak fix** to base: bug 296291, in progress.
- **virtual_oss SETTRIGGER patch** to upstream `hselasky/virtual_oss`, so `audio/virtual_oss` inherits it.
- **BruteFIR fork**, in order of preference: (a) upstream the delta to Anders Torger (dormant, unlikely); (b) `files/` patches to `audio/brutefir` (viable if the delta stays small); (c) its own project and port (`audio/brutefir-omdrc`), only if (b) is refused.
- **kodi-virtual-oss-patch**: upstream to Kodi or to `multimedia/kodi`'s `files/`, or drop from the tarball.

21.3 Phase 1 — Make the repository package-friendly

Worth doing even without a port.

- **Engine / site-data split.** Ship the engine (drc.sh, omdrc-ctrl, webremote, browser-nodrc, rc.d/devd glue, sample snippets, docs); keep room data (`configs/`, `filters/`, plots) in a separate overlay.
- **FLAT default filters** (first — small and independent): a `configs/flat/` geometry using BruteFIR's built-in `filename: "dirac pulse"` with `attenuation: 0.0`, so no binary filters are needed and `verify-bitperfect.sh` can pass through it as a plumbing self-test.
- **Runtime configuration instead of render-time baking**: read `$OMDRC_CONF` -> `${PREFIX}/etc/open-media-drc/omdrc.conf` -> `<script-dir>/config.env`; render BruteFIR configs on the fly into a state-dir tempfile so packaged configs are host-neutral.
- **State out of the tree**: `last_arg`, `last_power`, `drc.log` and the `/tmp` pid/output files to `/var/db/omdrc/` in installed mode.
- **Install target** (`DESTDIR/PREFIX`): scripts to `${PREFIX}/libexec/omdrc/` with a `${PREFIX}/bin/omdrc` wrapper; sample config and flat configs to `${PREFIX}/etc/open-media-drc/`; snippets to `share/examples/`; docs to `share/doc/open-media-drc/`; our rc.d scripts via `USE_RC_SUBR`.
- **Housekeeping**: a LICENSE (BSD-2-Clause fits); tagged releases whose tarballs exclude `freebsd-*-patch/`, journals and site data (`git archive` + `export-ignore`); split the README into a quickstart and `doc/DEVELOPMENT.md`.

21.4 Phase 2 — The port itself

`audio/open-media-drc` (working name): `USE_GITHUB=yes`, tagged `DISTVERSION`, `NO_BUILD` for the shell core, `USES=python:run shebangfix`. `RUN_DEPENDS`: brutefir (per Phase 0), virtual_oss, musicpd, sox/soxr. `OPTIONS_DEFINE`: `CTRL` (py-flask/Markdown/numpy), `VIDEO` (mpv), `UPNP` (upmpdcli). `USE_RC_SUBR` for `omdrc_audio` (and `omdrcctrl` with `CTRL`) — **not** musicpd or upmpdcli; a pkg-message documents the rc.conf lines that point the stock scripts at our configs. Validate with `portlint -AC`, `portclippy`, `poudriere testport` and `pkg check -s` after a service run.

21.5 Phase 3 — Submission and maintenance

Submit as a Bugzilla PR (Ports & Packages) with `MAINTAINER= delleceste@gmail.com`; niche integration ports are accepted when clean and maintained. Expect review rounds on rc style, sample handling and the brutefir dependency; merged Phase 0 fixes are the strongest argument that the stack works on stock FreeBSD. Ongoing: bump per release and watch fallout from musicpd/upmpdcli/virtual_oss updates.

Order of value: Phase 0 upstreaming (helps every FreeBSD USB-audio user, and a prerequisite anyway); the flat filters; the rest of Phase 1; Phases 2–3 only once the BruteFIR question is settled and there is an audience — a port is a maintenance promise.

21.6 A clean installable image (planned, not built)

`doc/USB-APPLIANCE-IMAGE-PLAN.md` scopes a USB stick that installs FreeBSD with `open-media-drc` preinstalled — no room filters, no per-geometry configs, no personal data, no git history, no build artifacts — ready for a room's `configs/<geo>` + `filters/<geo>` to be dropped in. Target: a fanless Intel/AMD-integrated-audio box dedicated to the appliance; **bee is out of scope** and stays the dev/measurement machine (root UFS 99% full, a legacy partition layout).

It builds on work already in flight: `OMDRC_SITE_DATA_DIRS` + `GEOMETRY=flat` gives a generic mode with no filter files; `.gitattributes export-ignore` already strips filters, per-room configs, the patch trees and the journals from `git archive` output; and the install target with runtime config (Phase 1) is what should land on the image, not bee's live CMake install with its baked `host.cmake` values. Nothing has been built from it yet.

Appendix A — Bit-perfect test assets and cross-OS comparison

Chapter 7 covers the single-host proof. This appendix documents the test assets and the procedure that proves the Linux and FreeBSD boxes send the DAC the *very same bytes*. It is the one place where both systems are compared side by side.

21.7 Test assets (`tests/`)

Two deterministic, near-silent (~ -90 dBFS) signals; every sample is uniquely determined, so any truncation, dither, volume change, resampling or channel swap shows up immediately.

- **Short asset (committed)** — `bitperfect-test-44100-s32-stereo.wav`: S32_LE, 2 ch, 44100 Hz, 100000 frames (~2.27 s). Per-sample counter in the low 16 bits, `L = i & 0xFFFF`, `R = (i*40503) & 0xFFFF`. Its PCM payload is byte-identical to the reference `.raw` (the WAV is the same bytes plus a 44-byte header — MPD cannot play headerless raw).
- **Cross-OS asset (generated, not committed)** — `bitperfect-test-44100-s32-stereo-30s.wav`: S32_LE, 2 ch, 44100 Hz, 1323000 frames (30 s), 10.6 MB. Here *every* `(L,R)` pair is unique over the whole file (`R = (i*40503 + (i >> 16)) & 0xFFFF` folds the block index in, breaking the 65536-frame period), so capture alignment is unambiguous at any length. It is too large to commit; regenerate it byte-identically on any OS:

```
python3 tests/gen-bitperfect-wav.py \
    tests/bitperfect-test-44100-s32-stereo-30s.wav
# sha256 88d365eeaccb1fa830bb1a2726b0f29bb545885824351080e0c5b4cbc9602348
```

21.8 Other rates and sample widths

The generator takes `--rate`, `--bits` (16/24/32) and `--frames` (= seconds x rate). Both tap scripts read rate and width from the WAV header, so a different file is the whole configuration:

```
python3 tests/gen-bitperfect-wav.py --rate 192000 --bits 24 --frames 5760000 \
    tests/bitperfect-test-192000-s24-stereo-30s.wav
./scripts/bitperfect-tap-linux.sh tests/bitperfect-test-192000-s24-stereo-30s.wav
```

The sample *values* are the same counter at every width (never above `0xFFFF`), so only the container changes. A 16/24-bit asset therefore also exercises the **lossless promotion to the 32-bit USB wire container** (`<<8` for 24-bit, `<<16` for 16-bit) that any bit-perfect player must perform for a

DAC accepting only 32-bit containers; `prep` promotes first, so the player always emits S32_LE. Consequences: **each format has its own sha256**, and a cross-OS comparison must use the **same width on both machines**.

Rate	Bits	Frames	Size	sha256 (first 16)
44100	32	1323000 (30 s)	10584044	`88d365eeaccb1fa8`
44100	24	1323000 (30 s)	7938044	`e2702c119606cdf8`
192000	24	5760000 (30 s)	34560044	`58dd87f3560334fb`
192000	24	1920000 (10 s)	11520044	`01317af6523ec67f`
96000	24	960000 (10 s)	5760044	`b572faabdee3b623`

The generator prints the sha256 of whatever it writes: generate once, note the hash, match it on the other machine (full hashes in `tests/README.md`). `.gitignore` excludes the generated assets by name; drop other WAVs under `bp-results/` (ignored except for `*.txt`).

21.9 Verification status

Both taps are executed and passing:

Host	Runs	Result
Linux (DacMagic 100, kernel 7.1.5-arch1)	44100/32-bit x 30 s, 44100/24-bit x 30 s, 192000/24-bit x 10 s	all **BIT-PERFECT**, 0 truncated events, 0 usbmon drops
FreeBSD 15.1-RELEASE (same DAC, `usbus0` devaddr 2)	same three	all **BIT-PERFECT** (exit 0); the 192 kHz run confirms the DAC clock followed (`dev.pcm.0.feedback_rate` = 191994) and costs ~24 s wall clock

`bitperfect-compare.py` reports **MATCH** across the two hosts for the 44100/32-bit asset; the 24-bit pairs have no committed Linux counterpart yet, so they stand as per-host proofs. The comparator has been exercised on every input pair (wav/wav, wav/txt, txt/txt, refusal of raw/txt) plus a deliberately bit-flipped payload, which it reports as MISMATCH at the exact offset.

21.10 Cross-OS byte comparison

Each OS taps its own USB isochronous OUT endpoint while playing the locally regenerated common WAV, then the reports are compared. Only the tiny `bp-results/*.txt` reports are committed — they carry the tap payload's length and sha256, proving byte-identity without moving 10 MB streams through git.

```
# Linux box:
./scripts/bitperfect-tap-linux.sh \
    tests/bitperfect-test-44100-s32-stereo-30s.wav
```

```
git add bp-results/*-linux.txt && git commit && git push

# FreeBSD box (free the DAC first: ./drc.sh off):
./scripts/bitperfect-tap-freebsd.sh \
    tests/bitperfect-test-44100-s32-stereo-30s.wav
git add bp-results/*-freebsd.txt && git commit && git push

# then on either box:
git pull
./scripts/bitperfect-compare.py \
    bp-results/bitperfect-test-44100-s32-stereo-30s-linux.txt \
    bp-results/bitperfect-test-44100-s32-stereo-30s-freebsd.txt
```

Identical length and sha256 on both reports proves the two systems deliver bit-identical audio to the DAC. The mismatch-forensics path is in `scripts/README.md` and `doc/BIT-PERFECT-VERIFICATION.md`. **A single run already proves the local path**: each tap compares its capture against the reference derived from the input file on that machine and exits 0 on **BIT-PERFECT**; the comparison is the optional further step.

What the report records. Each `PREFIX.txt` names and hashes every stage (`input file`, `ref bytes`, `wire raw`, `tap wav`, `verdict`). Only `input file`, `ref bytes` and `tap wav` are reproducible; **`wire raw` is not** — it is the untrimmed capture, whose length varies between identical runs and legitimately differs between Linux and FreeBSD (10584816 vs 10772776 bytes at 44100/32-bit). It is provenance only; the comparator uses `tap wav`. `PREFIX.wav` equals the input WAV only for a **32-bit** input: 16/24-bit input yields the promoted 32-bit container, so the invariant is the `tap wav` payload hash, not the file hash.

What surrounds the audio. The capture is always longer than the reference, and everything outside is measurably all-zero: a head of stream-priming zeros — **exactly 16 ms at both 44100 and 192000 Hz**, a fixed-duration buffer prime — and a tail of the 500 ms pad plus ~19 ms the kernel keeps transmitting after the writer closes. On a **BIT-PERFECT** verdict both capture boundaries fell outside the audio; when they fall inside, the tool says so (`HEAD LOST`, `INCOMPLETE`). "Inaudible" describes the sample *values*: opening or closing an isochronous stream, and any rate change around it, can still cause an audible artifact from the DAC's analogue side (mute relay, PLL relock — see `OKTO-DAC8-FreeBSD-44k1-flicker.md`), from stream start/stop and not from the zeros.

FreeBSD tap cost. The FreeBSD tap decodes `usbdump -vv` *text*, so parsing scales with the capture: 30 s at 44100 Hz is ~10.5 MB of payload as tens of MB of hex-dump text (fine); 30 s at 192 kHz is ~46 MB as several hundred MB of text — slow, and it stresses the pcap capture. Prove the path at 44100 first, then shorten the high-rate run (`--frames 1920000` = 10 s at 192 kHz). The Linux tap reads usbmon's binary interface and has no such limit.

Appendix B — Source document index

This manual is a synthesis of the repository's Markdown documentation and the CMake build, grouped by the same split as the manual itself.

Common (Part I and appendices)

Topic	Source document
Chain overview, install, drc.sh, hotplug	`README.md`
Install / build (CMake superproject, host values)	`CMakeLists.txt`, `host.cmake.sample`
Build modules: engine + site data; DAC hotplug + brutefir services; MPD + upmpdcli renderers; dependency audit; per-user setup	`cmake/core-drc.cmake`, `cmake/hotplug.cmake`, `cmake/renderers.cmake`, `cmake/dependencies.cmake`, `cmake/user-install.sh.in`
Web-UI subproject builds	`omdrc-ctrl/CMakeLists.txt`, `video/webremote/CMakeLists.txt`
Browser launchers	`cmake/browser-audio.cmake`
Filter/config layout, drc.sh modes, agent rules	`FILTERS_AND_DRC.md`
Filter provenance, hashes, verification, the design scripts	`doc/FILTER_PROVENANCE_AND_RESPONSE.md`
Site-data split (`OMDRC_SITE_DATA_DIRS` / `OMDRC_SITE_ROOT`)	`scripts/README.md`, `host.cmake.sample`, `cmake/core-drc.cmake`
Helper scripts	`scripts/README.md`, `README.md`
Web control panel and `/configuration` page	`omdrc-ctrl/README.md`, `omdrc-ctrl/src/configuration.py`
The kiosk and the Android app	`omdrc-ctrl/src/kiosk/README.md`, `android/omdrc-app/README.md`
The KDE Plasma widget	`kde/omdrc-plasmoid/README.md`
Qobuz album search	`omdrc-ctrl/README.md` (Qobuz album search), `omdrc-ctrl/src/qobuz_words.txt`
Ask AI, listening guide, AI settings, AI prompts	`omdrc-ctrl/README.md` (`[qobuz_search]`), `omdrc-ctrl/src/prompts/*.txt`, `omdrc-ctrl/src/qobuz_ai.py`
Spectrum analyzer	`omdrc-ctrl/SPECTRUM_ANALYZER.md`
Bit-perfect verification, the `/bitperfect` page and its implementation	`doc/BIT-PERFECT-VERIFICATION.md`, `scripts/README.md`, `omdrc-ctrl/README.md`
Test signal	`tests/README.md`
VBA vs all-pass comparison	`doc/xtras/FVBA.vs.ALLPASS.md`

Linux (Part II)

Topic	Source document
Browser ALSA management	`browser-nodrc/README.md`, `cmake/browser-alsa-linux.cmake`
CD / S-PDIF bridge (alsaloop)	`doc/CDIN-LINUX.md`
Blu-ray decryption (MakeMKV `libmmbd`, beta key, Kodi)	`video/BLURAY-DECRYPTION.md`, `video/BLURAY-PLAYBACK-TUNING.md`, `video/webremote/README.md`
Service and module glue	`etc/systemd/`, `etc/modules-load.d/`, `etc/modprobe.d/`

FreeBSD (Part III)

Topic	Source document
Stable sound-device names and lifecycle	`etc/rc.d/omdrc_audio`, `etc/devd/omdrc-audio.conf`
Browser audio (Chromium via sndiod, PulseAudio guard)	`browser-nodrc/lib.sh`, `cmake/browser-audio.cmake`, `etc/pulse/client.conf.d/10-omdrc-no-autospawn.conf`
CD / S-PDIF bridge	`cdin/README.md`
ESI U24 XL configuration and traps	`cdin/ESI-U24XL.md`
Video playback + Blu-ray	`video/README.md`
A/V sync delay derivation	`video/AV-SYNC-DELAY.md`
Web remote (install/API, design)	`video/webremote/README.md`, `video/webremote/ARCHITECTURE.md`
Glitch detection	`doc/GLITCH-DETECTION.md`
FreeBSD port plan	`doc/FREEBSD-PORT-PLAN.md`
USB appliance image plan	`doc/USB-APPLIANCE-IMAGE-PLAN.md`
uaudio patches (index + install)	`freebsd-uaudio-patch/README.md`
44.1 kHz flicker analysis	`freebsd-uaudio-patch/FreeBSD-uaudio-shared-clock-bug`
Shared-clock fix design	`freebsd-uaudio-patch/uaudio-shared-clock-fix.md`
Clock-before-alt analysis	`freebsd-uaudio-patch/uaudio-clock-before-alt.md`
Feedback-follow audit	`freebsd-uaudio-patch/uaudio-feedback-follow.md`
Residual 44.1 kHz silent-open audit (patch 3)	`freebsd-uaudio-patch/uaudio-clock-transaction.md`
Upstream follow-up submission for #295933	`freebsd-uaudio-patch/SUBMISSION-295933.md`
virtual_oss patches (index)	`freebsd-virtual-oss-patch/README.md`
SETTRIGGER livelock root cause	`freebsd-virtual-oss-patch/ROOTCAUSE-settrigger-sync-e`
cuse refleak root cause	`freebsd-virtual-oss-patch/ROOTCAUSE-cuse_client_open-`
cuse teardown bug report	`VIRTUAL_OSS_CUSE_DEADLOCK.md`
Cold-open silence audit	`OKTO-DAC8-silent-first-open.md`
44.1 kHz flicker observations	`OKTO-DAC8-FreeBSD-44k1-flicker.md`
MPD/curl CPU spin	`MPD-CURL-CPU-SPIN-FreeBSD.md`
Kodi OSS sink patch	`kodi-virtual-oss-patch/README.md`

21.11 Keeping this manual up to date

This manual is a **synthesis**, not a transclusion: rebuilding the PDF does not pull in changes to the source `.md` files. When a source document changes:

1. Update the corresponding section of `doc/pdf/open-media-drc-manual.md` (the tables above are the section-to-source mapping).
2. If the chain topology changed, adjust `doc/pdf/diagrams/*.dot`.
3. Re-run `doc/pdf/build-pdf.sh` (pandoc, pdflatex or Chromium, graphviz) to regenerate `doc/open-media-drc-manual.pdf`.

Keep the OS split. A fact that holds on one OS only goes in Part II (Linux) or Part III (FreeBSD), never inline in Part I, which points to it instead. Editing constraints (pdflatex): keep the file ASCII, with no box-drawing characters or Unicode arrows; diagrams are added as `![caption](build/<name>.pdf){width=NN%}`. Details in `doc/pdf/README.md`.

Appendix C — Glossary

Terms in alphabetical order. **OS** shows where the term applies: *both*, *Linux* or *FreeBSD*. References point to the section that explains it.

Term	OS	Meaning	See
alsaloop	Linux	`alsa-utils` tool that bridges the CD capture card into `snd-aloop`, correcting drift with the rate-shift control	12
AACS	both	Blu-ray encryption; decrypted by libbluray's plugin, stock `libaacs` or MakeMKV's `libmmbd`	13
analysis file	both	Precomputed response traces of a design (`analysis/<design>.json`), shown only when the bundle verifies	5, 5.9
attenuation	both	Per-config BruteFIR gain reduction that prevents clipping where a filter has gain above 0 dB; computed by `headroom_calc.py`	4
bit-perfect	both	The DAC receives the source bytes unchanged: no resampling, volume, dither or format conversion	7, 16.5
`bitperfect=1`	FreeBSD	`dev.pcm` sysctl: the first opener's format becomes the hardware format, so no kernel feeder alters bytes	16, 15.2

Term	OS	Meaning	See
bounded wait	both	Every external call under a lock has a timeout, so a slow MPD cannot stall boot or the chain	15
browser-nodrc	both	Launchers that stop DRC, run a browser and restore the exact prior state	4.6
BruteFIR	both	The float64 FIR convolution engine (delleceste fork) that applies the room filters	3
bundle / `bundle_id`	both	One design's self-contained, hash-bound set of files; the id is the SHA-256 of its canonical identity	5
capture role	both	The card carrying the CD/S-PDIF input, chosen by USB identity	6.4.1, 15.2, 11.5
CD input (`cdin`)	both	Second source: a CD transport's S/PDIF captured through the ESI U24 XL into the loopback	10
`omdrc-cdin`	FreeBSD	Purpose-written capture daemon; a `memcpy` ring whose lead absorbs drift	18
`config.env` / repo mode	both	A `config.env` beside `drc.sh` makes it run from the checkout instead of the installed tree	3
cuse	FreeBSD	Kernel facility (`cuse(3)`) through which `virtual_oss` creates character devices; its teardown bug is a known wedge	16, 20.2

Term	OS	Meaning	See
`default` design	both	The reserved historical design that keeps un-suffixed paths and cannot be removed	5
design / **`@design`**	both	One immutable filter revision inside a geometry, with a provenance manifest	5
devd	FreeBSD	FreeBSD's device event daemon; the project rule fires on `pcm` attach/detach	15.1
`dmix`	Linux	ALSA software mixer that lets browser streams share the DAC	11.6
DR (dynamic range)	both	TT Dynamic Range value of a master; *Measure DR* measures the stream itself, *Estimate DR* the passage playing now; the DR log keeps it per album	8, 8.3, 8.4, 9
DRC	both	Digital Room Correction: FIR filtering applied before the DAC	4
`drc.sh`	both	The single control point of the DRC pipeline	4
`drc.lock` / **`device.lock`**	FreeBSD	The two non-nested locks guarding the chain transition and the role transaction	15
ESI U24 XL	both	USB S/PDIF capture interface used for CD input	10, 18.6, 12.1
geometry	both	A physical setup (speaker and listening position); a directory under `configs/` and `filters/`	5
glitch detection	FreeBSD	Monitor, USB tap and analyzer that classify dropouts	16.4

Term	OS	Meaning	See
`host.cmake`	both	Initial CMake cache holding every box-specific value; read only by `-C` on a fresh build directory	3
hotplug	both	Reacting to DAC plug/unplug: udev + systemd (Linux), devd + rc.d (FreeBSD)	11.3, 15.1
`KEYDB.cfg`	both	FindVUK key database read by stock `libaacs`; covers only submitted discs	13, 17
known-device list	both	Cards previously applied to a role; drives automatic DAC swapping	6.4.1
lead	FreeBSD	Audio buffered ahead of the output: drift margin, startup delay and transport lag at once	18
loopback	both	Device MPD writes and BruteFIR reads: `snd-aloop` (Linux), `virtual_oss` (FreeBSD)	11.4, 16
MakeMKV / `libmmbd`	Linux	Proprietary Blu-ray engine; `libmmbd` replaces `libaacs`/`libbdplus` via `LIBAACS_PATH`/`LIBBDPLUS_PATH` (`makemkv-env.sh`, for mpv and Kodi); needs a beta key that the video web remote looks up and renews	13, 13.4
manifest	both	JSON of hashes and metadata for a design; written last as the commit marker	5
MPD / `musicpd`	both	The player; `mpd` on Linux, `musicpd` on FreeBSD	4.3, 14.1

Term	OS	Meaning	See
MPD outputs	both	`OKTO-DAC` (direct), `DRC-native`, `DRC-resamp`	4.3
`omdrc-config-helper`	both	Privileged helper that installs verified designs and pins audio roles	6.4
omdrc-ctrl / `omdrcctrl`	both	The Flask web control panel (port 9090)	6, 11.7, 16.3
`omdrc_audio`	FreeBSD	The rc.d service that owns device roles and the DRC lifecycle	15.1, 15.2
priming	FreeBSD	Opening the DAC once at a new rate so the real open does not route silence	16.2
PulseAudio guard	FreeBSD	`autospawn = no` drop-in plus a masked autostart entry, so no sound server can take the single-open DAC	16.6
provenance	both	The hash chain from REW exports to the coefficients BruteFIR loaded	5
reconcile	both	Level-triggered comparison of saved intent with reality; repairs only a mismatch	4, 15
REW	both	Room EQ Wizard, the measurement tool the filters are designed in	5
roles	both	Stable DAC and capture assignments by USB identity, never by card number	11.5, 15.2
`snd-aloop`	Linux	ALSA loopback kernel module, pinned to the DAC clock with `timer_source`	11.4
`sndiod`	FreeBSD	Playback-only sndio server used for Chromium during a No DRC session	16.6

Term	OS	Meaning	See
site data / site root	both	Room `configs/` and `filters/`, kept in a separate repository via `OMDRC_SITE_DATA_DIRS` / `OMDRC_SITE_ROOT`	5
source policy (`last_source`)	both	Persistent choice of `music` or `cdin`; CD input is exclusive	10, 4
spectrum analyzer	both	Panel card fed from a FIFO of MPD or CD audio	6
`uaudio(4)`	FreeBSD	Kernel USB audio driver; the patches fix the OKTO clock behaviour	20.1, 19
udev rule	Linux	`99-usb-audio-drc.rules` copied to `/etc/udev/rules.d`	11, 11.3
upmpdcli	both	UPnP/OpenHome front end that makes MPD a renderer	3, 14.5
`/bitperfect` page	both	Panel page that runs the USB tap through five playback paths and shows the bytes	7.1, 7.2
variant	both	Legacy config-filename suffix selecting an alternate filter set; always unverified	5
virtual_oss	FreeBSD	Userland OSS mixer/router providing `/dev/dsp.play` and `/dev/dsp.loop`	16
wire tap	both	Capture of the USB isochronous OUT endpoint (usbmon on Linux, usbdump on FreeBSD)	7, 7.2